

My Tool Box

Multi-Agent Reinforcement Learning Environments
and Research Utilities

User Manual, version 1.0

Dongming Wang

September 27, 2026

Contents

1	Introduction	1
1.1	What the package provides	1
1.2	Who this package is for	3
1.3	Design principles	3
1.4	Package layout	4
1.5	Conventions used in this manual	4
1.6	How this manual is organised	5
1.7	Licence and citation	5
2	Installation	6
2.1	Requirements	6
2.2	Installing from the repository	6
2.2.1	Optional extras	7
2.2.2	PyTorch	7
2.2.3	Tk	7
2.3	Development installation	8
2.4	Verifying the installation	8
2.5	Optional dependency matrix	8
2.6	Upgrading and uninstalling	9
2.7	Building this manual	9
I	Environments	10
3	Environment Library Overview	11
3.1	Environments at a glance	11
3.2	Package structure	11
3.3	Creating environments	12
3.4	Common API conventions	13
3.4.1	Gymnasium interface	13
3.4.2	Joint multi-agent spaces	13

3.4.3	Termination and truncation	14
3.4.4	Seeding and reproducibility	14
3.4.5	Reset options	15
3.5	Rendering	15
3.6	Recording episodes	16
3.7	Registered environments	17
4	Kuramoto Oscillator Networks	18
4.1	Model	18
4.2	Coupling modes and topologies	19
4.3	Spaces	20
4.4	Reward and episode end	20
4.5	Parameters	21
4.6	Info dictionary	22
4.7	PyTorch backend	22
4.8	Rendering	23
4.9	Registered ids	23
4.10	Usage	23
4.11	Reproducibility and performance	25
4.12	Changes in 1.0	25
5	Networked Communication Environments	26
5.1	LineMsg	26
5.1.1	Model	26
5.1.2	Spaces	27
5.1.3	Parameters and info	27
5.1.4	Rendering	28
5.1.5	Usage	28
5.2	WirelessComm	29
5.2.1	Model	29
5.2.2	Spaces	30
5.2.3	Parameters and info	30
5.2.4	Rendering	30
5.2.5	Usage	31
5.3	Registered ids	32
5.4	Reproducibility and performance	32
5.5	Changes in 1.0	32

6	Pistonball	34
6.1	Arena and physics	34
6.2	Spaces	34
6.3	Reward and episode end	35
6.4	Parameters	36
6.5	Keyboard control	36
6.6	Rendering	37
6.7	Registered id	38
6.8	Usage	38
6.9	Reproducibility and performance	38
6.10	Changes in 1.0	39
7	Consensus and Formation Control	40
7.1	Model	40
7.2	Spaces	41
7.3	Reward and episode end	42
7.4	Parameters	42
7.5	Info dictionary and properties	43
7.6	Baseline controller	44
7.7	Rendering	44
7.8	Registered ids	45
7.9	Usage	45
7.10	Reproducibility and performance	46
8	AJLATT: Active Joint Localisation and Target Tracking	47
8.1	Scenario and motion model	47
8.2	Sensing, communication and estimation	48
8.3	Spaces	49
8.4	Reward and episode end	49
8.5	Configuration	50
8.6	Target motion	53
8.7	Info dictionary and episode statistics	53
8.8	Team reward wrapper and legacy factory	53
8.9	Maps	54
8.9.1	Bundled maps	54
8.9.2	Map files	54
8.9.3	Map API	55
8.9.4	Building maps	56
8.9.5	Plotting maps	57

8.10	Rendering	58
8.11	Registered id	58
8.12	Usage	58
8.13	Reproducibility and performance	59
8.14	Changes in 1.0	60
II	Toolkit	61
9	Toolkit Overview and Parameter Management	62
9.1	Structure of the toolkit	62
9.2	parakit: managing argparse parameters	63
9.2.1	Quick start	63
9.2.2	Headless API	63
9.2.3	File format	64
9.2.4	Parsing rules	65
9.2.5	Recommended workflow in training scripts	66
9.2.6	The ParameterTuner window	66
10	plotkit: Publication-Quality Plots	69
10.1	Overview	69
10.2	Input rules	70
10.3	Common parameters	71
10.4	Curves with uncertainty bands	71
10.4.1	plot_shadow_curve	71
10.4.2	plot_learning_curves	73
10.5	Lines, scatter plots, bars and histograms	73
10.5.1	plot_bar	73
10.5.2	plot_histogram	73
10.6	Heatmaps	73
10.7	Styles	74
10.8	Colour palettes	76
10.9	Saving figures	77
10.10	Recipes	77
10.10.1	Reinforcement-learning training curves	77
10.10.2	Comparing methods, including unfinished runs	78
10.10.3	Several panels with consistent colours	78
10.10.4	Distinguishing series without colour	79
10.10.5	Bar chart of final performance	79

10.10.6 Confusion and correlation matrices	79
10.10.7 Distributions of returns	79
10.10.8 Plotting tensors and data frames	80
10.11 Command-line gallery	80
11 neural_toolkit: Networks and Tabular Tools	81
11.1 Conventions	81
11.2 Building blocks	81
11.3 Network families	82
11.3.1 Constructor arguments	82
11.3.2 Architectures	82
11.4 Factories	85
11.5 NetworkUtils	86
11.6 Tabular tools	86
11.7 Example: a DQN agent for LineMsg	89
11.8 Changes in version 1.0	91
III Practice	92
12 Examples, Benchmarks and Development	93
12.1 Example scripts	93
12.2 Writing an evaluation loop	95
12.3 Recording episodes	95
12.4 Benchmarks	96
12.5 Testing, linting and continuous integration	98
12.5.1 Running the tests	98
12.5.2 Linting and formatting	98
12.5.3 Continuous integration	98
12.5.4 Conventions for contributions	99
13 Troubleshooting	100
13.1 Rendering on machines without a display	100
13.2 No window appears in human mode	100
13.3 Missing optional dependencies	101
13.4 “Out of date” warnings for -v0 environment ids	102
13.5 AJLATT deprecation warnings	102
13.6 PyTorch, GPUs and devices	102
13.7 Video export	103
13.8 Reproducibility and seeding	104

13.9 Performance	104
13.10 Other common errors	105
A API Reference	106
A.1 env_lib	106
A.1.1 Package functions	106
A.1.2 Common environment interface	106
A.1.3 KuramotoOscillatorEnv and KuramotoOscillatorEnvTorch	107
A.1.4 LineMsgEnv and WirelessCommEnv	108
A.1.5 PistonballEnv	108
A.1.6 ConsensusEnv	109
A.1.7 AJLATTEnv and AJLATTConfig	110
A.1.8 AJLATT maps	110
A.2 env_lib.utils	111
A.3 toolkit.plotkit	112
A.4 toolkit.neural_toolkit	113
A.4.1 Layers	113
A.4.2 Policy and value networks	113
A.4.3 Q-networks	113
A.4.4 Encoders and decoders	114
A.4.5 Factories	114
A.4.6 NetworkUtils	114
A.4.7 Tabular tools	115
A.5 toolkit.parakit	116
B Migration from pre-1.0 versions	117
B.1 Checklist	117
B.2 Installation	117
B.3 Changes common to all environments	118
B.4 Kuramoto oscillators	119
B.5 LineMsg and WirelessComm	119
B.6 Pistonball	119
B.7 AJLATT	120
B.8 plotkit	121
B.9 neural_toolkit	122
B.10 parakit	122

Chapter 1

Introduction

My Tool Box is a Python package for research on multi-agent reinforcement learning (MARL) and networked control. It is distributed as a single installable project, `my-tool-box`, that provides two import packages:

- env_lib** A collection of multi-agent environments that follow the Gymnasium API: coupled oscillators, message passing and wireless access on graphs, a cooperative physics game, consensus and formation control, and multi-robot localisation and target tracking.
- toolkit** Research utilities that do not depend on the environments: `plotkit` creates publication-quality figures, `neural_toolkit` provides PyTorch network building blocks and tabular reinforcement-learning tools, and `parakit` manages `argparse` parameters, with an optional graphical editor.

This manual describes version 1.0 of the package. It explains how to install it, documents every environment and toolkit module, collects practical recipes and troubleshooting advice, and ends with an API reference and a migration guide for code written against earlier versions.

1.1 What the package provides

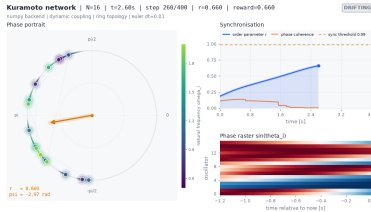
Table 1.1 lists the environments. Each one is available as a class in `env_lib` and under one or more registered Gymnasium ids; the version suffix of an id selects a preset configuration (for example `WirelessComm-v1` is the 4×4 grid), not a newer revision. Figure 1.1 shows the built-in visualisations.

The `toolkit` package contains three independent subpackages:

- `toolkit.plotkit` (Chapter 10): learning curves with confidence bands and smoothing, grouped bar charts, histograms, annotated heatmaps, style presets, colour-blind-safe palettes and multi-format export. It depends on `matplotlib` only.
- `toolkit.neural_toolkit` (Chapter 11): policy, value and Q-networks with MLP, convolutional, recurrent and transformer trunks, encoders and decoders (including a variational autoencoder), factories, network utilities and tabular reinforcement-learning tools. It requires `PyTorch`.
- `toolkit.parakit` (Chapter 9): saving, loading, validating and applying the defaults of an `argparse.ArgumentParser`, and a Tk window for editing them.

Table 1.1: Environments provided by `env_lib`. The last column names the optional dependency group (see Chapter 2) required in addition to the core installation.

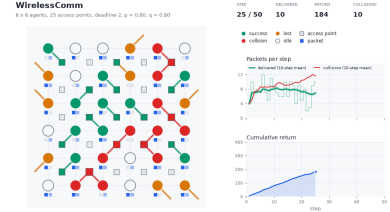
Registered ids	Task	Chapter	Extra
KuramotoOscillator-*	Drive a network of coupled phase oscillators to synchronisation (NumPy).	4	—
KuramotoOscillatorTorch-*	The same model on PyTorch tensors, with many systems simulated in a batch.	4	torch
LineMsg-v0	Relay a message along a line of agents with unreliable links.	5	—
WirelessComm-v0, -v1	Share wireless access points on a grid without collisions.	5	—
Pistonball-v0	Pistons cooperate to roll a ball to the goal (2-D physics).	6	pistonball
Consensus-v0, Formation-v0	Rendezvous or form a shape over a communication graph.	7	—
AJLATT-v0	Robots localise themselves and jointly track a moving target.	8	—



(a) Kuramoto oscillators



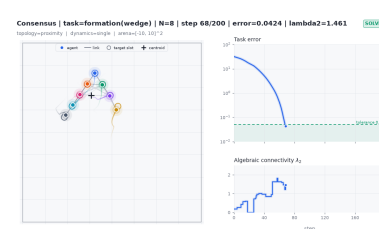
(b) LineMsg



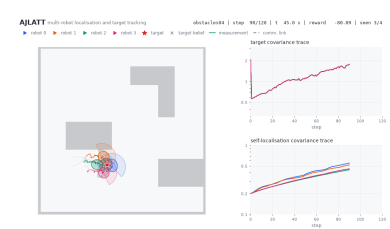
(c) WirelessComm



(d) Pistonball



(e) Formation control



(f) AJLATT target tracking

Figure 1.1: Frames rendered in "rgb_array" mode by the environments of `env_lib`. Each renderer is a dashboard that combines the scene with the quantities that matter for the task, such as the order parameter of the oscillators, per-agent outcomes or covariance traces.

1.2 Who this package is for

The package is written for researchers and students who

- need small, fast and well-specified multi-agent benchmarks with a standard interface, in particular for networked systems where agents observe only their neighbourhood;
- study synchronisation, consensus, distributed estimation or cooperative control and want environments whose dynamics and rewards are documented precisely enough to be cited in a paper;
- want consistent, reproducible figures and reusable network components without adopting a large framework.

The environments do not depend on any particular learning library; any algorithm that can drive a Gymnasium environment can use them.

1.3 Design principles

Gymnasium compatibility. Every environment subclasses `gymnasium.Env`. The method `reset` accepts a `seed` and an `options` dictionary as keyword arguments and returns the pair `(observation, info)`. The method `step` returns the observation, the reward, the flags `terminated` and `truncated`, and the `info` dictionary, where `terminated` signals a task-defined end (success or failure) and `truncated` the episode limit. Observations are `float32` arrays contained in `observation_space`. Multi-agent environments expose the whole team through joint spaces whose leading axis indexes the agents, and every environment reports per-agent rewards in `info["agent_rewards"]`. Each environment enforces its own episode limit through a constructor argument; the registry does not add a `TimeLimit` wrapper that could silently override it. Chapter 3 describes these conventions in detail.

Vectorised implementations. Dynamics, observations and rewards are computed with NumPy array expressions rather than Python loops over agents. The Kuramoto model additionally has a PyTorch backend that simulates many independent systems in one batch, on the CPU or a GPU. The networks of `neural_toolkit` are batch-first PyTorch modules. Section 12.4 reports measured step times.

Reproducibility. All randomness of an environment comes from its own random generator (`self.np_random`), which is seeded by `reset(seed=...)`. No library code seeds or draws from the global NumPy, Python or PyTorch random state, and plotting functions never modify the global matplotlib configuration unless asked to. Section 13.8 lists everything that must be seeded for an experiment to be repeatable.

Headless rendering. Rendering is optional and never performed unless requested. In `"rgb_array"` mode figures are drawn off-screen and returned as `(H, W, 3) uint8` arrays, so episodes can be recorded to GIF or MP4 on machines without a display, for example on a compute cluster or in continuous integration. In `"human"` mode an environment renders itself at the end of every `reset()` and `step()`, as Gymnasium prescribes.

Lazy optional dependencies. `import env_lib` and `import toolkit` load only the core dependencies (NumPy, SciPy, matplotlib, Gymnasium and PyYAML). PyTorch, pygame, pymunk,

imageio and Tk are imported only when a feature that needs them is used, and a missing dependency produces an error message that names the package extra to install.

1.4 Package layout

The repository uses the standard `src` layout:

My_Tool_Box/	
pyproject.toml	packaging metadata, optional extras, tool settings
CHANGELOG.md	release notes and migration notes
LICENSE	MIT licence
src/	
env_lib/	environments (import name env_lib)
registration.py	Gymnasium ids, env_lib.make(), env_lib.list_envs()
utils/	themes, renderers, GIF/MP4 recording
kos_env/	Kuramoto oscillators (NumPy and PyTorch)
linemsg_env/	LineMsg
wireless_comm_env/	WirelessComm
pistonball_env/	Pistonball
consensus_env/	Consensus and Formation
ajlatt_env/	AJLATT (configuration, estimation, maps, rendering)
toolkit/	research utilities (import name toolkit)
plotkit/	plotting
neural_toolkit/	PyTorch networks and tabular tools
parakit/	argparse parameter management
examples/	runnable, documented example scripts
benchmarks/	performance measurements
tests/	pytest suite
docs/manual/	this manual (LaTeX sources)

A first program creates an environment through the registry, runs the distributed controller that ships with the consensus environment and renders the final state:

```
import env_lib

env = env_lib.make("Consensus-v0", render_mode="rgb_array")
obs, info = env.reset(seed=0)
for _ in range(200):
    action = env.unwrapped.laplacian_policy() # baseline controller
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        break
frame = env.render() # (H, W, 3) uint8 image
print(info["error"], info["agent_rewards"].shape, frame.shape)
env.close()
```

1.5 Conventions used in this manual

Python identifiers are set in monospace, package and module names in sans-serif, and registered environment ids as `Consensus-v0`. Shell commands are meant to be run from the repository root in a POSIX shell; on Windows, replace `export VAR=value` by `set VAR=value (cmd)` or `$env:VAR="value"` (PowerShell). Code examples use the public API of version 1.0 and were

executed with it. Array shapes are written as tuples, for example `(n_agents, obs_dim)`; `B` denotes a batch size and `T` a sequence length.

1.6 How this manual is organised

- Chapter 2 covers requirements, installation with optional extras, verification and building this manual.
- Part I documents the environments: the common interface (Chapter 3), the Kuramoto oscillators (Chapter 4), LineMsg and WirelessComm (Chapter 5), Pistonball (Chapter 6), consensus and formation control (Chapter 7) and AJLATT (Chapter 8).
- Part II documents the toolkit: its structure and `parakit` (Chapter 9), `plotkit` (Chapter 10) and `neural_toolkit` (Chapter 11).
- Part III is practical: the example scripts, recording, benchmarks and the contributor workflow (Chapter 12), and troubleshooting (Chapter 13).
- Appendix A is a compact API reference and Appendix B explains how to migrate code written for versions before 1.0.

1.7 Licence and citation

My Tool Box is free software distributed under the MIT licence; the full text is in the file `LICENSE` at the root of the repository. The source code is hosted at https://github.com/EricDMW/My_Tool_Box, where bugs and feature requests can be reported as issues.

If the package contributes to published work, please cite it as follows:

```
@software{wang2026mytoolbox,
  author = {Wang, Dongming},
  title = {My Tool Box: Multi-Agent Reinforcement Learning
           Environments and Research Utilities},
  version = {1.0.0},
  year = {2026},
  url = {https://github.com/EricDMW/My_Tool_Box}
}
```

When results depend on a particular environment, please also cite the original description of the underlying model or benchmark (for example the Kuramoto model of coupled oscillators), as given in the corresponding environment chapter.

Chapter 2

Installation

My Tool Box is a standard Python project described by a single `pyproject.toml` file. It is installed from its source repository with `pip`; heavy dependencies are optional and grouped into *extras*.

2.1 Requirements

- Python 3.9 or newer. The package is tested with CPython 3.9 to 3.13; continuous integration runs the test suite on 3.9, 3.10, 3.11 and 3.12.
- Any operating system supported by the dependencies (Linux, macOS and Windows). Development and continuous integration use Linux.
- The core dependencies below, installed automatically by `pip`.

Table 2.1: Core dependencies (always installed).

Package	Minimum version	Used for
numpy	1.23	all numerical code
scipy	1.9	AJLATT estimation, plotkit smoothing
matplotlib	3.6	environment renderers, plotkit
gymnasium	0.29	environment API and registration
pyyaml	6.0	AJLATT map files and map specifications

2.2 Installing from the repository

The recommended procedure creates a virtual environment, clones the repository and installs the package with all optional features:

```
git clone https://github.com/EricDMW/My_Tool_Box.git
cd My_Tool_Box
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
python -m pip install --upgrade pip
pip install "[all]"
```

`pip` can also install directly from the Git URL without a local clone, which is convenient for a project that only uses the package:

```
pip install "my-tool-box[all] @ git+https://github.com/EricDMW/My_Tool_Box.git"
```

With `conda`, create the environment first and then use `pip` inside it:

```
conda create -n mytoolbox python=3.11
conda activate mytoolbox
conda install tk # only for Tk windows (see below)
pip install ".[all]"
```

2.2.1 Optional extras

The extras of Table 2.2 are selected in square brackets and can be combined, for example `pip install ".[pistonball,video]"`. The quotes prevent shells such as `zsh` from interpreting the brackets.

Table 2.2: Optional extras defined in `pyproject.toml`.

Extra	Installs	Enables
<code>pistonball</code>	<code>pygame</code> $\geq$ 2.1, <code>pymunk</code> $\geq$ 6.4	<code>PistonballEnv</code> (<code>pymunk</code> for the physics, <code>pygame</code> for rendering and keyboard control)
<code>torch</code>	<code>torch</code> $\geq$ 2.0	<code>KuramotoOscillatorEnvTorch</code> , <code>toolkit.neural_toolkit</code>
<code>video</code>	<code>imageio</code> $\geq$ 2.20, <code>imageio-ffmpeg</code> $\geq$ 0.4	MP4, WebM, AVI, MOV and MKV export in <code>save_animation</code> and <code>record_episode</code> (GIF export needs no extra)
<code>all</code>	the three extras above	every feature of the package
<code>dev</code>	<code>pytest</code> $\geq$ 7, <code>pytest-cov</code> $\geq$ 4, <code>ruff</code> $\geq$ 0.5	running the test suite and the linter; combine with <code>all</code>

2.2.2 PyTorch

The `torch` extra installs the default PyTorch wheel for the platform. To choose a specific build, install PyTorch first, following the selector at <https://pytorch.org/get-started/>, and then install the package; `pip` keeps the PyTorch that is already present. For example, a CPU-only build (much smaller, and the one used in continuous integration) is obtained with

```
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install ".[all]"
```

GPU use is covered in Section 13.6.

2.2.3 Tk

Tk is not installable with `pip`. It is needed only for the `parakit` editor window (Section 9.2.6) and for `render_mode="human"` with matplotlib's TkAgg backend; everything else, including off-screen rendering, works without it. Install it with the system package manager (`sudo apt-get install python3-tk` on Debian and Ubuntu, `sudo dnf install python3-tkinter` on Fedora,

`brew install python-tk` with Homebrew on macOS) or with `conda install tk`. The official Python installers for Windows and macOS already include Tk.

2.3 Development installation

Contributors install the package in *editable* mode, so that changes to the sources under `src/` take effect without reinstalling, together with the development tools:

```
git clone https://github.com/EricDMW/My_Tool_Box.git
cd My_Tool_Box
python -m venv .venv && source .venv/bin/activate
pip install -e ".[all,dev]"
```

The console scripts and the version number are generated at installation time; after changing `pyproject.toml` (for example adding an entry point), run the `pip install -e` command again. The testing and linting workflow is described in Section 12.5.

2.4 Verifying the installation

The following commands check, from quickest to most thorough, that the package is installed correctly:

```
# 1. Versions of both import packages
python -c "import env_lib, toolkit; print(env_lib.__version__, toolkit.__version__)"

# 2. Registered environment ids
python -c "import env_lib; print(env_lib.list_envs())"

# 3. Create every environment and take a few random steps
python examples/quickstart.py --steps 5

# 4. Full test suite (requires the dev extra)
pytest -q
```

The first command prints `1.0.0 1.0.0`. The quick-start script prints one line per environment and skips, with a message, environments whose optional dependencies are missing. The test suite skips tests of missing optional dependencies in the same way.

The installation also provides three console scripts:

Command	Purpose
<code>ajlatt-build-map</code>	build an AJLATT map from a YAML specification (Chapter 8)
<code>ajlatt-plot-map</code>	list and plot the bundled AJLATT maps (Chapter 8)
<code>plotkit-gallery</code>	gallery of plotkit figures (Section 10.11)

2.5 Optional dependency matrix

Table 2.3 summarises which features need which optional packages. Because the package imports optional dependencies lazily, a missing dependency only affects the feature that needs it; the error message names the extra to install.

Table 2.3: Features and their optional requirements.

Feature	Requires	Provided by
NumPy environments (Kuramoto, LineMsg, WirelessComm, Consensus, AJLATT), <code>rgb_array</code> rendering, GIF export	core only	core
PistonballEnv	pymunk	pistonball
Pistonball rendering and ManualPolicy	pygame	pistonball
KuramotoOscillatorEnvTorch	torch	torch
toolkit.neural_toolkit	torch	torch
MP4 and other video formats	imageio, imageio-ffmpeg	video
toolkit.plotkit	core only	core
Plotting pandas, PyTorch, TensorFlow or JAX data	the respective library	not required
toolkit.parakit headless API	standard library	core
parakit editor window	Tk and a display	system package
<code>render_mode="human"</code> (matplotlib environments)	an interactive backend (TkAgg, QtAgg, ...) and a display	system package or PyQt/PySide
Test suite and linting	pytest, pytest-cov, ruff	dev

2.6 Upgrading and uninstalling

To upgrade a clone, pull the latest sources and reinstall: `git pull` followed by `pip install ".[all]"` (or `pip install -e ".[all,dev]"` for an editable installation). `pip uninstall my-tool-box` removes the package. Users of versions before 1.0, which were installed from two separate project folders, should first uninstall the old distributions and then follow Appendix B.

2.7 Building this manual

The manual is written in LaTeX and lives in `docs/manual/`. Building it requires a TeX distribution that provides `pdflatex` and `latexmk`, such as TeX Live or MacTeX. On Debian and Ubuntu the packages `latexmk`, `texlive-latex-extra`, `texlive-fonts-recommended` and `lmodern` are sufficient. The document uses only standard packages (`listings`, `booktabs`, `tabularx`, `hyperref`, `fancyhdr` and similar).

```
cd docs/manual
./build.sh                                # writes output/main.pdf
# equivalently:
latexmk -pdf -interaction=nonstopmode -halt-on-error -outdir=output main.tex
latexmk -C -outdir=output                # remove build products
```

`latexmk` runs `pdflatex` as often as needed to resolve cross references. All build products are written to `docs/manual/output/`, which is ignored by Git. The file `docs/manual/README.md` describes the source layout of the manual.

Part I

Environments

Chapter 3

Environment Library Overview

The `env_lib` package provides multi-agent environments for research on cooperative control, networked decision making and multi-robot estimation. All environments implement the Gymnasium API, are registered with Gymnasium when `env_lib` is imported, and share one rendering and recording layer. This chapter describes what the environments have in common; the following chapters document each environment in detail:

- Kuramoto oscillator networks, NumPy and batched PyTorch backends (Chapter 4);
- LineMsg and WirelessComm, two discrete networked communication benchmarks (Chapter 5);
- Pistonball, a cooperative physics game (Chapter 6);
- Consensus and formation control on communication graphs (Chapter 7);
- AJLATT, active joint localisation and target tracking with a robot team, including its map tools (Chapter 8).

3.1 Environments at a glance

Table 3.1 summarises the environments in their default configuration. N is the number of oscillators, M the number of coupling edges and n the number of agents.

3.2 Package structure

The package lives in `src/env_lib` of the repository:

<code>src/env_lib/</code>	
<code>__init__.py</code>	<code>make()</code> , <code>list_envs()</code> , lazily imported environment classes
<code>registration.py</code>	Gymnasium registration of every id (<code>ENV_SPECS</code>)
<code>utils/</code>	themes, renderers, <code>record_episode</code> , <code>save_animation</code>
<code>kos_env/</code>	Kuramoto oscillators (NumPy and PyTorch backends)
<code>linemsg_env/</code>	LineMsg: message relay along a line of agents
<code>wireless_comm_env/</code>	WirelessComm: random access to shared access points
<code>pistonball_env/</code>	Pistonball: cooperative physics game (<code>pymunk</code> , <code>pygame</code>)
<code>consensus_env/</code>	consensus and formation control on graphs
<code>ajlatt_env/</code>	AJLATT: multi-robot localisation and target tracking
<code>maps/</code>	occupancy maps, map builder and map plotting

Table 3.1: Environments in their default configuration.

Environment	Agents	Action space	Observation	Reward	Requires
Kuramoto (NumPy)	1 controller, $N = 10$ oscillators	continuous $\text{Box}((N+M,))$, $N + M = 55$	$(75,)$, i.e. $3N + M$	scalar synchronisation measure, bonus on synchronisation	none
Kuramoto (PyTorch)	<code>n_agents</code> parallel systems (1)	as NumPy; batched (<code>n_agents</code> , $N+M$) accepted	$(75,)$ of system 0	per system in info	torch
LineMsg	$n = 10$	$\text{Discrete}(2**n)$ or $\text{MultiBinary}(n)$	$(10, 3)$	sum of per-agent rewards	none
WirelessComm	$n = 36$ (6×6)	$\text{MultiDiscrete}([5]*n)$	$(36, 18)$	packets delivered (sum)	none
Pistonball	$n = 20$ pistons	$\text{Box}(-1, 1, (n,))$ or $\text{MultiDiscrete}([3]*n)$	$(20, 7)$	sum of local rewards	pymunk, pygame
Consensus / Formation	$n = 8$	$\text{Box}(-1, 1, (n, 2))$	$(8, 16)$	mean of local rewards, success bonus	none
AJLATT	$n = 4$ robots	$\text{Box}((n, 2))$: $v \in [0, 2]$, $\omega \in [-\pi/4, \pi/4]$	$(4, 24)$	per-robot array	none

Every environment subpackage contains the environment module and a **rendering** module. The renderer is imported on the first call to `render()`, so an environment that is never rendered does not create any figure or window.

Lazy imports. `import env_lib` imports neither PyTorch nor pygame, pymunk or tkinter. The environment classes of Table 3.2 are attributes of the package that are imported on first access, for example `env_lib.ConsensusEnv`. An optional dependency is therefore only needed when the environment that uses it is created (Table 3.3).

Table 3.2: Environment classes exported by `env_lib`.

Class	Module	Chapter
KuramotoOscillatorEnv	<code>env_lib.kos_env.kuramoto_env</code>	4
KuramotoOscillatorEnvTorch	<code>env_lib.kos_env.kuramoto_env_torch</code>	4
LineMsgEnv	<code>env_lib.linemsg_env.linemsg_env</code>	5
WirelessCommEnv	<code>env_lib.wireless_comm_env.wireless_comm_env</code>	5
PistonballEnv	<code>env_lib.pistonball_env.pistonball_env</code>	6
ConsensusEnv	<code>env_lib.consensus_env.consensus_env</code>	7
AJLATTEnv, AJLATTConfig	<code>env_lib.ajlatt_env</code>	8

3.3 Creating environments

There are three equivalent ways to create an environment:

```
import gymnasium as gym
import env_lib

env = env_lib.make("WirelessComm-v0")           # registered id
env = gym.make("WirelessComm-v0")               # after importing env_lib
env = env_lib.WirelessCommEnv(grid_x=6, grid_y=6) # direct construction
```

Table 3.3: Optional dependencies of the environments. The extras are installed with `pip install -e ".[extra]"` from the repository root.

Extra	Packages	Needed for
pistonball	pymunk, pygame	Pistonball: <code>pymunk</code> to create the environment, <code>pygame</code> to render it and for keyboard control
torch	torch	the PyTorch backend of the Kuramoto environments
video	imageio, imageio-ffmpeg	<code>save_animation</code> to <code>.mp4</code> and other video formats (GIF export needs only Pillow)

```
print(env_lib.list_envs())
```

all registered ids

The function `env_lib.make` registers the environments if necessary and calls `gymnasium.make` with the id and the keyword arguments, which are passed to the constructor and override the configuration of the id, for example `env_lib.make("LineMsg-v0", num_agents=20, max_iter=100)`. `gymnasium.make` returns the environment inside Gymnasium's standard wrappers (order enforcement and, except for `AJLATT-v0`, the passive environment checker). Environment-specific methods such as `laplacian_policy()` or `get_batch_observations()` are reached through `env.unwrapped`.

3.4 Common API conventions

3.4.1 Gymnasium interface

Every environment implements

- `reset(*, seed=None, options=None)` returning `(observation, info)`;
- `step(action)` returning `(observation, reward, terminated, truncated, info)`;
- `render()` and `close()`; `close()` may be called several times.

Observations are `float32` NumPy arrays contained in `observation_space`. Calling `step()` before the first `reset()` raises `env_lib.ResetNeededError`, which derives from both `RuntimeError` and `gymnasium.error.ResetNeeded`, so either can be caught. Invalid actions (wrong shape, values outside a discrete range, NaN or infinite values) raise `ValueError`, and continuous actions are clipped to the action bounds. `AJLATT` is the exception: it only checks the action shape, and clipping is opt-in (`clip_actions=True`, Section 8.3).

3.4.2 Joint multi-agent spaces

Each environment exposes the whole team through a single Gymnasium interface. The action space covers all agents at once (for example `Box(-1, 1, (8, 2))` for eight planar agents, or `MultiDiscrete([5] * 36)` for 36 wireless agents), and observations have one row per agent. A multi-agent algorithm splits the joint arrays along the first axis; a centralised algorithm uses them directly.

The scalar `reward` returned by `step()` is the team reward. The per-agent rewards are always available as `info["agent_rewards"]`, a `float64` array of shape `(n_agents,)`, which every `step()` returns. How the team reward is formed from the per-agent rewards depends on the environment (Table 3.1): it is their sum for `LineMsg`, `WirelessComm`, `Pistonball` and `AJLATT`

and their mean (plus a success bonus) for Consensus. The Kuramoto environments control one or several complete oscillator networks, so their “agents” are systems: the NumPy backend reports `n_agents` copies of its reward and the PyTorch backend one reward per simulated system. AJLATT returns the per-robot rewards and per-robot collision flags directly as arrays; `TeamRewardWrapper` converts them to a scalar reward and a scalar `terminated` flag (Section 8.8).

The info dictionary returned by `reset()` contains `"agent_rewards"` (all zeros) for LineMsg, WirelessComm, Consensus and AJLATT. The Kuramoto and Pistonball environments do not include the key in their reset info.

3.4.3 Termination and truncation

`terminated` signals a task-specific terminal state; `truncated` signals the end of the time budget. Every environment enforces its own episode limit through a constructor argument (Table 3.4). The registrations do not add a `TimeLimit` wrapper (`env.spec.max_episode_steps` is `None`), so a limit passed to the constructor is never overridden by the registry. To change the episode length pass the constructor argument, for example `env_lib.make("Consensus-v0", max_steps=400)`; passing `max_episode_steps` to `gymnasium.make` would add a second, independent limit.

Table 3.4: Episode limits and termination conditions.

Environment	Limit argument	Default	<code>terminated</code> is set when
Kuramoto	<code>max_steps</code>	1000	the order parameter exceeds <code>sync_threshold</code>
LineMsg	<code>max_iter</code>	50	never (always <code>False</code>)
WirelessComm	<code>max_iter</code>	50	never (always <code>False</code>)
Pistonball	<code>max_cycles</code>	125	the ball reaches the left wall
Consensus	<code>max_steps</code>	200	the task error drops below <code>tolerance</code>
AJLATT	<code>max_episode_steps</code>	120	per robot: an obstacle is closer than <code>obstacle_collision_distance</code> (array of flags)

3.4.4 Seeding and reproducibility

- `reset(seed=s)` seeds the environment’s generator `self.np_random`. All randomness of an episode (initial state, stochastic transitions, noise) is drawn from this generator, and `reset()` without a seed continues its stream. With the same seed, the same actions and the same library versions an episode is reproduced exactly.
- No environment reads or modifies the global NumPy or PyTorch random state. The PyTorch Kuramoto backend owns a `torch.Generator` that is seeded from `self.np_random` at every reset.
- `env.action_space.sample()` uses the space’s own generator; seed it separately with `env.action_space.seed(s)`.
- Some structure is fixed at construction and has its own seed: `topology_seed` (random Kuramoto topology, default 42), `graph_seed` (Erdős–Rényi graph of the consensus environment) and `AJLATTConfig.seed` (seed of the first AJLATT reset when none is given).
- The legacy `env.seed()` methods of LineMsg, WirelessComm and Pistonball still work but emit a `DeprecationWarning`; use `reset(seed=...)` instead.

3.4.5 Reset options

`reset(options=...)` can override parts of the sampled initial state in the Kuramoto environments ("phases", "natural_frequencies", "coupling_strengths") and in the consensus environment ("positions", "velocities"). Unknown keys raise `ValueError`. The other environments accept and ignore `options`.

3.5 Rendering

The render mode is chosen at construction with the `render_mode` argument:

None	(default) No rendering. <code>render()</code> returns <code>None</code> and warns once.
"rgb_array"	<code>render()</code> returns the current frame as a (H, W, 3) <code>uint8</code> array. Frames are drawn off-screen and need no display.
"human"	A window is opened on the first frame and updated automatically at the end of every <code>reset()</code> and <code>step()</code> , following the Gymnasium convention; do not call <code>render()</code> yourself in this mode. The frame rate is capped at <code>metadata["render_fps"]</code> .
"ansi"	(<code>LineMsg</code> and <code>WirelessComm</code>) <code>render()</code> returns a plain-text summary of the agents' actions and states.

Table 3.5: Render modes and frame sizes (width $\times$ height in pixels).

Environment	Modes	Frame size	Drawing backend, <code>render_fps</code>
Kuramoto	human, <code>rgb_array</code>	1000×560	matplotlib, 30
LineMsg	human, <code>rgb_array</code> , <code>ansi</code>	1000×560	matplotlib, 10
WirelessComm	human, <code>rgb_array</code> , <code>ansi</code>	1000×560	matplotlib, 10
Pistonball	human, <code>rgb_array</code>	$(160 + 40n) \times 560$	pygame, 20
Consensus / Formation	human, <code>rgb_array</code>	1000×560	matplotlib, 20
AJLATT	human, <code>rgb_array</code>	1100×620	matplotlib, 8

Renderer design. The matplotlib renderers are subclasses of `MatplotlibRenderer` from `env_lib.utils`. They create their axes and artists once and only update the artists' data on later frames. In "rgb_array" mode the static part of the figure (axes, ticks, labels, legends) is rasterised once and cached, and each frame redraws only the registered dynamic artists (blitting), which is typically three to six times faster than a full redraw. The renderers never modify `matplotlib.rcParams`. Pistonball draws with pygame surfaces.

Human mode and headless machines. Human mode needs an interactive matplotlib backend (for example TkAgg or QtAgg) or, for Pistonball, a display for pygame. With a non-interactive backend the environment issues a `RuntimeWarning` and shows no window. On servers and in continuous integration set

```
export MPLBACKEND=Agg SDL_VIDEODRIVER=dummy
```

and use `render_mode="rgb_array"`.

Themes. Two colour themes ship with the package: "dark" (the default) and "light", which is intended for papers and slides and was used for all figures in this manual. Select a theme with `env_lib.utils.set_theme`. The theme is read when a renderer is created, that is at the first `render()` call of an environment, so set it before rendering starts. Custom palettes are `env_lib.utils.Theme` instances registered with `register_theme`.

```
import env_lib
from env_lib.utils import set_theme

set_theme("light")          # renderers created from now on use the light theme
env = env_lib.make("Consensus-v0", render_mode="rgb_array")
obs, info = env.reset(seed=0)
for _ in range(100):
    action = env.action_space.sample()
    obs, reward, terminated, info = env.step(action)
    if terminated or truncated:
        break
frame = env.render()        # numpy array of shape (560, 1000, 3), dtype uint8
env.close()
```

3.6 Recording episodes

`env_lib.utils` provides two functions to turn rollouts into animations:

```
record_episode(env, policy=None, path=None, *, max_steps=None, seed=None,
              fps=None, render_every=1) -> list[numpy.ndarray]
save_animation(frames, path, fps=30.0, *, loop=0) -> pathlib.Path
```

`record_episode` resets the environment with `seed`, renders the initial state and then steps the environment with `policy` (a callable mapping an observation to an action; uniformly random actions from `env.action_space` if omitted), rendering one frame every `render_every` steps. The rollout stops when any entry of `terminated` or `truncated` is true or after `max_steps` steps. The function returns the list of frames and, when `path` is given, writes them with `save_animation` at `fps` frames per second (default `env.metadata["render_fps"]`). The environment must use `render_mode="rgb_array"`.

`save_animation` writes a sequence of equally sized RGB frames and returns the path. The format follows the file suffix: `.gif` is written with Pillow (always available); `.mp4`, `.webm`, `.avi`, `.mov` and `.mkv` are written with `imageio` and `imageio-ffmpeg` (extra video); frames are cropped to even dimensions for video codecs. `loop` is the GIF loop count (0 loops forever). Missing parent directories are created.

```
import env_lib
from env_lib.utils import record_episode, save_animation

env = env_lib.make("Formation-v0", render_mode="rgb_array")
policy = lambda obs: env.unwrapped.laplacian_policy()  # baseline controller
frames = record_episode(env, policy, "renders/formation.gif",
                       max_steps=150, seed=0, render_every=2)
save_animation(frames[:2], "renders/formation_fast.gif", fps=20)
env.close()
```

3.7 Registered environments

Table 3.6 lists all ids returned by `env_lib.list_envs()` with the constructor arguments each registration sets; all other arguments keep their defaults. The version suffixes follow the historical ids of the project and denote preset configurations, not revisions: `WirelessComm-v1`, for example, is the 4×4 grid and not a newer version of `WirelessComm-v0`. Gymnasium may therefore emit an “out of date” `DeprecationWarning` when a `-v0` id is created; the warning is harmless.

Table 3.6: Registered Gymnasium ids and their configuration.

Id	Class and configuration
KuramotoOscillator-v0	KuramotoOscillatorEnv; defaults (10 oscillators, dynamic coupling)
KuramotoOscillator-v1	KuramotoOscillatorEnv; <code>n_oscillators=5</code>
KuramotoOscillator-Constant-v0	KuramotoOscillatorEnv; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code>
KuramotoOscillator-FreqSync-Constant-v0	KuramotoOscillatorEnv; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code> , <code>reward_type="frequency_synchronization"</code> , <code>target_frequency=2.0</code>
KuramotoOscillatorTorch-v0	KuramotoOscillatorEnvTorch; defaults (10 oscillators, one system)
KuramotoOscillatorTorch-v1	KuramotoOscillatorEnvTorch; <code>n_oscillators=8</code> , <code>n_agents=4</code> , <code>device="cpu"</code>
KuramotoOscillatorTorch-v2	KuramotoOscillatorEnvTorch; <code>n_oscillators=10</code> , <code>n_agents=1</code> , <code>device="auto"</code> , <code>integration_method="rk4"</code>
KuramotoOscillatorTorch-Constant-v0	KuramotoOscillatorEnvTorch; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code>
KuramotoOscillatorTorch-FreqSync-Constant-v0	KuramotoOscillatorEnvTorch; <code>n_oscillators=6</code> , <code>coupling_mode="constant"</code> , <code>reward_type="frequency_synchronization"</code> , <code>target_frequency=2.0</code>
LineMsg-v0	LineMsgEnv; defaults (10 agents)
WirelessComm-v0	WirelessCommEnv; defaults (6×6 grid)
WirelessComm-v1	WirelessCommEnv; <code>grid_x=4</code> , <code>grid_y=4</code>
Pistonball-v0	PistonballEnv; defaults (20 pistons); needs pymunk (and pygame to render)
Consensus-v0	ConsensusEnv; defaults (8 agents, rendezvous on a ring)
Formation-v0	ConsensusEnv; <code>task="formation"</code> (circle formation)
AJLATT-v0	AJLATTEnv; defaults (4 robots on obstacles04); passive environment checker disabled because rewards and <code>terminated</code> are per-robot arrays

Chapter 4

Kuramoto Oscillator Networks

The Kuramoto environments simulate a network of coupled phase oscillators that a controller has to synchronise. They are used to study learning-based control of network synchronisation: shaping the coupling of a network (coupling design), steering oscillators with bounded inputs, tracking a common frequency, and comparing learned controllers with classical results such as the synchronisation transition of the Kuramoto model. Two interchangeable backends implement the same model:

- `KuramotoOscillatorEnv` (module `env_lib.kos_env.kuramoto_env`): NumPy, one network per environment;
- `KuramotoOscillatorEnvTorch` (module `env_lib.kos_env.kuramoto_env_torch`): PyTorch, `n_agents` independent networks simulated in parallel on the CPU or a GPU; requires the `torch extra`.

With identical initial states and actions both backends produce the same trajectories up to `float32` rounding.

4.1 Model

The state consists of the phases $\theta_1, \dots, \theta_N$ of N oscillators with natural frequencies ω_i . The controlled dynamics are

$$\dot{\theta}_i = \omega_i + a_i + c \sum_{j=1}^N K_{ij} \sin(\theta_j - \theta_i), \quad i = 1, \dots, N, \quad (4.1)$$

where a_i is the control input of oscillator i , K the coupling matrix and $c = 1$, or $c = 1/N$ with `normalize_coupling=True` (the classical K/N scaling). Positive K_{ij} are attractive and negative K_{ij} repulsive couplings; the diagonal of K has no effect.

Integration. One environment step advances the system by $\Delta t = dt$ with the explicit Euler method, $\theta \leftarrow \theta + \Delta t f(\theta)$, or with the classical fourth-order Runge–Kutta method (`integration_method="rk4"`),

$$\begin{aligned} \theta &\leftarrow \theta + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4), \\ k_1 &= f(\theta), \quad k_2 = f\left(\theta + \frac{\Delta t}{2} k_1\right), \quad k_3 = f\left(\theta + \frac{\Delta t}{2} k_2\right), \quad k_4 = f(\theta + \Delta t k_3), \end{aligned}$$

where f is the right-hand side of (4.1) with the action held constant during the step. If `noise_std` = $\sigma > 0$, independent Gaussian noise $\mathcal{N}(0, \sigma^2)$ is added to every phase after the integration step (the noise is not scaled by $\sqrt{\Delta t}$). Finally the phases are wrapped to $[-\pi, \pi)$.

Synchronisation measures. The complex order parameter

$$r e^{i\psi} = \frac{1}{N} \sum_{j=1}^N e^{i\theta_j} \quad (4.2)$$

has magnitude $r \in [0, 1]$: $r = 1$ for identical phases and $r \approx 0$ for incoherent phases; ψ is the mean phase. The phase coherence is

$$C = \exp(-\text{Var}(\theta)), \quad (4.3)$$

where Var is the population variance (normalised by N) of the phases wrapped to $[-\pi, \pi)$. For frequency tracking the environment uses the mean absolute frequency error

$$E_f = \frac{1}{N} \sum_{i=1}^N |\dot{\theta}_i - \Omega|, \quad (4.4)$$

with target frequency $\Omega = \text{target_frequency}$; $\dot{\theta}_i$ is evaluated at the start of the step, with the new action applied (for RK4 this is k_1).

4.2 Coupling modes and topologies

The network topology is an undirected graph with adjacency matrix A . Only connected pairs are coupled. The available topologies are listed in Table 4.1; `adj_matrix` overrides `topology`, which is then reported as "custom".

Table 4.1: Network topologies. M is the number of undirected edges.

topology	Edges	M
"fully_connected"	every pair of oscillators	$N(N-1)/2$
"ring"	i is connected to $i \pm 1 \bmod N$	N (for $N \geq 3$)
"star"	oscillator 0 is connected to all others	$N-1$
"random"	every ordered pair is drawn with probability 1/2 and the matrix is symmetrised with a logical OR (a pair is connected with probability 3/4); the draw uses a local generator seeded with <code>topology_seed</code>	random
"custom"	non-zero entries of <code>adj_matrix</code> (either direction)	varies

Dynamic coupling. With `coupling_mode="dynamic"` (default) the coupling strength of every edge is part of the action. The edges are listed in row-major order of the upper triangle of A ; the list is available as `env.coupling_indices` (pairs (i, j) with $i < j$) and its length as `env.n_couplings` = M . The strength k_e of edge $e = (i, j)$ sets $K_{ij} = K_{ji} = k_e$. A non-symmetric `adj_matrix` triggers a warning, because every connected pair is one undirected edge with a single strength.

Constant coupling. With `coupling_mode="constant"` the coupling matrix is fixed and only the control inputs are actions. K is the matrix passed as `constant_coupling_matrix`, with its diagonal set to zero, or $K = \text{coupling_strength} \cdot A$ when no matrix is given. A matrix passed in dynamic mode is ignored with a warning.

4.3 Spaces

Action. A `float32 Box` with the layout of Table 4.2. Actions are clipped to the bounds; an action of the wrong shape or with non-finite values raises `ValueError`.

Table 4.2: Action layout.

Indices	Content	Bounds
0 : N	control inputs $a_1, \dots, a_N$	<code>control_input_range</code> , default $[-1, 1]$
N : N+M	coupling strengths $k_1, \dots, k_M$ (dynamic mode only)	<code>coupling_range</code> , default $[0, 5]$

Observation. An unbounded `float32 Box` of size $3N + M$ in dynamic mode and $3N$ in constant mode (Table 4.3). The default `KuramotoOscillator-v0` ($N = 10$, fully connected, $M = 45$) has an action of size 55 and an observation of size 75.

Table 4.3: Observation layout (dynamic mode; in constant mode the coupling block is absent and the control inputs occupy $2N : 3N$).

Indices	Content
0 : N	phases θ_i , wrapped to $[-\pi, \pi)$
N : 2N	natural frequencies ω_i
2N : 2N+M	coupling strengths of the last action (sampled at reset)
2N+M : 3N+M	control inputs of the last action (zero after reset)

4.4 Reward and episode end

The reward is computed after the step from the new phases. The reward type is selected with `reward_type`:

$$\begin{aligned}
 \text{"order_parameter": } R &= r, \\
 \text{"phase_coherence": } R &= C, \\
 \text{"combined": } R &= r + C, \\
 \text{"frequency_synchronization": } R &= -E_f,
 \end{aligned}$$

with r , C and E_f from (4.2)–(4.4). When r exceeds `sync_threshold` (default 0.99) the episode terminates (`terminated=True`) and, for all reward types except `"frequency_synchronization"`, `sync_bonus` (default 10) is added to the reward. A threshold above 1 disables termination on synchronisation. `truncated` becomes `True` after `max_steps` steps.

4.5 Parameters

Both backends accept the parameters of Table 4.4; all have defaults. Invalid values raise `ValueError` or `TypeError`.

Table 4.4: Constructor parameters of `KuramotoOscillatorEnv` and `KuramotoOscillatorEnvTorch`.

Parameter	Default	Description
<code>n_oscillators</code>	10	Number of oscillators N (≥ 2).
<code>n_agents</code>	1	PyTorch: number of systems simulated in parallel. NumPy: only sets the length of <code>info["agent_rewards"]</code> .
<code>dt</code>	0.01	Integration time step (> 0).
<code>max_steps</code>	1000	Episode length; reaching it sets <code>truncated</code> .
<code>coupling_range</code>	(0.0, 5.0)	Bounds of the coupling actions and of the random initial coupling strengths.
<code>control_input_range</code>	(-1.0, 1.0)	Bounds of the control inputs a_i .
<code>natural_freq_range</code>	(0.5, 2.0)	Natural frequencies are drawn uniformly from this range at every reset.
<code>device</code>	"cpu"	PyTorch: "cpu", "cuda", "cuda:k", a <code>torch.device</code> , or "auto" (CUDA when available). NumPy: ignored.
<code>render_mode</code>	None	None, "rgb_array" or "human".
<code>integration_method</code>	"euler"	"euler" or "rk4".
<code>reward_type</code>	"order_parameter"	"order_parameter", "phase_coherence", "combined" or "frequency_synchronization".
<code>noise_std</code>	0.0	Standard deviation of the phase noise added after each step.
<code>topology</code>	"fully_connected"	"fully_connected", "ring", "star", "random" or "custom".
<code>adj_matrix</code>	None	Custom (N, N) adjacency matrix (array or tensor); overrides <code>topology</code> .
<code>target_frequency</code>	1.0	Target Ω of the frequency-synchronisation reward.
<code>coupling_mode</code>	"dynamic"	"dynamic" (couplings are actions) or "constant".
<code>constant_coupling_matrix</code>	None	Fixed (N, N) coupling matrix of constant mode.
<code>coupling_strength</code>	1.0	Uniform strength of constant mode without an explicit matrix ($K = \text{coupling_strength} \cdot A$).
<code>normalize_coupling</code>	False	Divide the coupling sum by N .
<code>topology_seed</code>	42	Seed of the local generator that builds the "random" topology.
<code>sync_threshold</code>	0.99	Order parameter above which the episode terminates.
<code>sync_bonus</code>	10.0	Reward bonus on synchronisation.
<code>render_agent</code>	0	PyTorch only: index of the system shown by <code>render()</code> .

Initial state and reset options. `reset()` draws phases uniformly from $[-\pi, \pi)$, natural frequencies uniformly from `natural_freq_range` and, in dynamic mode, coupling strengths

uniformly from `coupling_range`; the control inputs start at zero. Any part can be fixed with `reset(options=...)` using the keys `"phases"`, `"natural_frequencies"` (shape $(N,)$) and `"coupling_strengths"` (shape $(M,)$, dynamic mode only). The PyTorch backend also accepts arrays of shape $(n_agents, size)$ with one row per system.

4.6 Info dictionary

Table 4.5: Keys of the info dictionary. The PyTorch backend returns per-system NumPy arrays with a leading dimension `n_agents` for the quantities marked with an asterisk.

Key	Content
<code>order_parameter*</code>	order parameter r after the step
<code>phase_coherence*</code>	phase coherence C after the step
<code>phases*</code>	wrapped phases, shape $(N,)$
<code>natural_frequencies*</code>	natural frequencies ω_i
<code>coupling_matrix*</code>	coupling matrix K used in the step, shape (N, N)
<code>dphases_dt*</code>	phase velocities $\dot{\theta}$ at the start of the step (not in the reset info)
<code>step_count</code>	number of steps taken in the episode
<code>device</code>	device of the simulation ("cpu" for NumPy)
<code>n_agents</code>	value of <code>n_agents</code>
<code>agent_rewards</code>	float64 array of shape $(n_agents,)$; NumPy: copies of the reward; PyTorch: the reward of every system, including its bonus (not in the reset info)

4.7 PyTorch backend

`KuramotoOscillatorEnvTorch` simulates `n_agents` independent networks with the same topology in one batch of tensors. All state tensors have a leading batch dimension.

- `step(action)` accepts an action of shape $(act_dim,)$, applied to every system, or (n_agents, act_dim) with one row per system, as a NumPy array or a tensor.
- The returned observation, scalar reward and `terminated` flag refer to system 0; `truncated` is shared. The per-system values are in `info` (Table 4.5).
- `get_batch_observations()` returns the observations of all systems as a tensor of shape (n_agents, obs_dim) on `env.device`.
- `get_batch_rewards(dphases_dt=None, include_bonus=False)` returns the per-system rewards of the current state as a tensor of shape $(n_agents,)$; the synchronisation bonus is only included with `include_bonus=True` (as in `info["agent_rewards"]`).
- Systems are not reset individually: a system that synchronises keeps evolving (and keeps earning its bonus) until the whole batch is reset.

Computations run in `float32`. For large batches or networks select a GPU with `device="cuda"` or `device="auto"`.

4.8 Rendering

Figure 4.1 shows the dashboard (1000×560 pixels). The phase portrait on the left places every oscillator on the unit circle at angle θ_i , coloured by its natural frequency (colour bar), with a glow and a fading trail of its recent phases. Couplings are drawn as chords whose width and opacity scale with $|K_{ij}|$; repulsive couplings use the theme’s negative colour. The arrow from the origin is the order parameter $re^{i\psi}$. On the right, the upper panel shows $r(t)$ and the phase coherence over the episode with the synchronisation threshold as a dashed line, and the lower panel a raster of $\sin \theta_i$ over the most recent steps. The header reports the step, time, r and the reward. The PyTorch backend shows system `render_agent`.

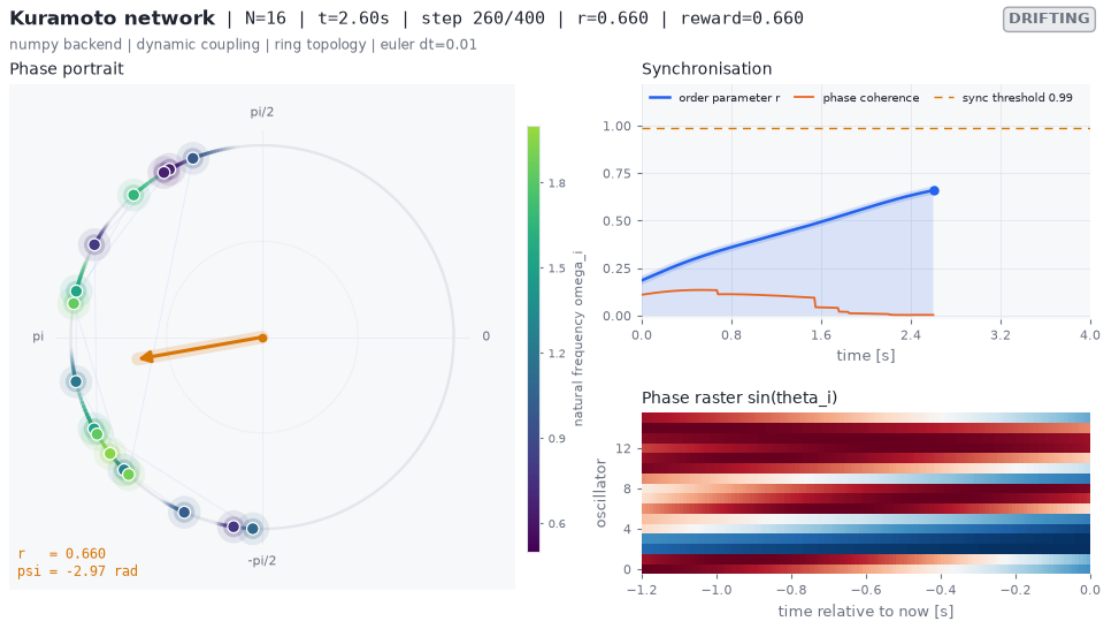


Figure 4.1: Kuramoto dashboard (16 oscillators on a ring, dynamic coupling, light theme). Left: phase portrait with coupling chords, trails and the order-parameter vector. Right: order parameter and phase coherence over time, and the phase raster $\sin \theta_i$.

4.9 Registered ids

Table 4.6 lists the registered Kuramoto ids. The `KuramotoOscillatorTorch-*` ids require the `torch` extra; keyword arguments of `env_lib.make` override their configuration.

4.10 Usage

The first example drives a network with constant coupling to synchronisation with a proportional phase-alignment input $a_i = r \sin(\psi - \theta_i)$ computed from the observation:

```
import numpy as np
import env_lib

env = env_lib.KuramotoOscillatorEnv(
    n_oscillators=12, coupling_mode="constant", coupling_strength=0.5
)
obs, info = env.reset(seed=0)
```

Table 4.6: Kuramoto ids (all other parameters keep their defaults).

Id	Configuration
KuramotoOscillator-v0	NumPy, $N = 10$, dynamic coupling
KuramotoOscillator-v1	NumPy, $N = 5$, dynamic coupling
KuramotoOscillator-Constant-v0	NumPy, $N = 6$, constant coupling
KuramotoOscillator-FreqSync-Constant-v0	NumPy, $N = 6$, constant coupling, frequency reward, $\Omega = 2$
KuramotoOscillatorTorch-v0	PyTorch, $N = 10$, one system
KuramotoOscillatorTorch-v1	PyTorch, $N = 8$, 4 systems, CPU
KuramotoOscillatorTorch-v2	PyTorch, $N = 10$, RK4, device="auto"
KuramotoOscillatorTorch-Constant-v0	PyTorch, $N = 6$, constant coupling
KuramotoOscillatorTorch-FreqSync-Constant-v0	PyTorch, $N = 6$, constant coupling, frequency reward, $\Omega = 2$

```

n = env.n_oscillators
while True:
    phases = obs[:n].astype(np.float64)
    z = np.exp(1j * phases).mean()                #  $r \exp(i \psi)$ 
    action = np.abs(z) * np.sin(np.angle(z) - phases) # phase alignment
    action = action.astype(np.float32)
    obs, reward, terminated, truncated, info = env.step(action)
    if terminated or truncated:
        break
print(info["step_count"], info["order_parameter"]) # 51, above 0.99
env.close()

```

The second example runs four systems in parallel on the PyTorch backend, one coupling level per system:

```

import numpy as np
import env_lib

env = env_lib.make("KuramotoOscillatorTorch-v1") # 8 oscillators, 4 systems
core = env.unwrapped
obs, info = env.reset(seed=0)
print(core.get_batch_observations().shape)        # torch.Size([4, 52])

n, act_dim = core.n_oscillators, core.action_space.shape[0]
actions = np.zeros((core.n_agents, act_dim), dtype=np.float32)
actions[:, n:] = np.linspace(1.0, 4.0, core.n_agents)[:n, None]
for _ in range(300):
    obs, reward, terminated, truncated, info = env.step(actions)
print(info["order_parameter"])                    # one value per system
print(info["agent_rewards"])                      # per system, with bonus
env.close()

```

The example script `examples/kuramoto_demo.py` ramps the coupling strengths and adds phase alignment, and records the transition from incoherence to synchronisation:

```

python examples/kuramoto_demo.py --save renders/kuramoto.gif
python examples/kuramoto_demo.py --backend torch --n-agents 4 --theme light
python examples/kuramoto_demo.py --mode constant --topology two_clusters \
    --render-mode human

```

4.11 Reproducibility and performance

Seeding. `reset(seed=s)` seeds `self.np_random`, from which the NumPy backend draws the initial state and the phase noise. The PyTorch backend seeds its own `torch.Generator` from `self.np_random` at every reset; neither backend touches the global random state. The random topology is built once from `topology_seed`. The two backends draw from different generators, so the same seed gives different initial states; to compare them, pass the initial state through `reset(options=...)`.

Performance. Table 4.7 lists step times measured with `benchmarks/benchmark_envs.py` on the development machine (random actions, no rendering). The coupling term is evaluated with two matrix–vector products instead of N^2 sine evaluations. Rendering in "rgb_array" mode takes about 25–35 ms per frame and dominates the cost of recorded rollouts; use `render_every` in `record_episode` to thin out long episodes. Use Euler integration for speed and RK4 when accuracy at larger `dt` matters, and a GPU for large batches.

Table 4.7: Step time in milliseconds on the development machine.

Case	Before 1.0	Version 1.0
NumPy, $N = 10$	0.17	0.04
NumPy, $N = 50$	2.36	0.07–0.11
NumPy, RK4, $N = 50$	7.1	0.1
PyTorch (CPU), $N = 50$, 8 systems	10.3	0.3–0.4

4.12 Changes in 1.0

The behaviour of the Kuramoto environments changed in several ways; see Appendix B for migration advice.

- The PyTorch backend had a sign error that made the coupling repulsive ($\sin(\theta_i - \theta_j)$ instead of $\sin(\theta_j - \theta_i)$). Both backends now implement (4.1) and produce the same trajectories; normalisation by N is opt-in for both.
- The coupling uses the full matrix; the former NumPy loop skipped entries $K_{ij} \leq 0$. Default settings are unaffected.
- Synchronisation sets `terminated` and the step limit sets `truncated` (previously both were reported as `terminated`); `sync_threshold` and `sync_bonus` are configurable.
- Constant mode without a matrix uses `coupling_strength · A`, so the `*-Constant-v0` ids work (they raised before).
- The environments no longer call `np.random.seed(42)` or `torch.manual_seed(42)`; the random topology uses a local generator (`topology_seed`) and is unchanged.
- The PyTorch backend accepts `device="auto"`, and its phase coherence and bonus rules match the NumPy backend.
- The dynamics are vectorised (20 times faster at $N = 50$, 65 times with RK4) and the renderer was rewritten; the former "rgb_array" path failed with matplotlib 3.10 and later.
- `env_lib.kos_env.register()` is deprecated; the ids are registered when `env_lib` is imported.

Chapter 5

Networked Communication Environments

This chapter documents two discrete multi-agent benchmarks in which agents on a network cooperate under local observations:

- **LineMsg** (`env_lib.linemsg_env.LineMsgEnv`): agents on a line relay a message from a source to a sink;
- **WirelessComm** (`env_lib.wireless_comm_env.WirelessCommEnv`): agents on a grid send deadline-constrained packets to shared access points and must avoid collisions.

Both environments have small discrete state and action spaces, local observation windows controlled by `n_obs_neighbors`, per-agent rewards and a fixed episode length. They are designed for research on decentralised and networked multi-agent reinforcement learning: exploiting the locality of interactions (policies and value functions that depend on a κ -hop neighbourhood), credit assignment, communication and coordination protocols, scalability in the number of agents, and robustness to failing agents. The WirelessComm environment is the Gymnasium joint-action port of an earlier PettingZoo implementation used in the DSDP project.

5.1 LineMsg

5.1.1 Model

N agents sit on a line. Agent $N - 1$ at the right end is the source, agent 0 at the left end is the sink. Each agent holds a binary state $s_i \in \{0, 1\}$ (1: it holds the message) and chooses a binary action $a_i \in \{0, 1\}$ at every step. All states are updated synchronously from the previous states:

$$\begin{aligned} s_0 &\leftarrow s_1, \\ s_i &\leftarrow a_i \wedge (s_{i+1} \vee [u_i < 0.8]), \quad 0 < i < N - 1, \quad u_i \sim \mathcal{U}(0, 1), \\ s_{N-1} &\leftarrow a_{N-1}. \end{aligned} \tag{5.1}$$

An active middle agent ($a_i = 1$) therefore receives the message from its right neighbour for sure and otherwise obtains one with probability 0.8; an inactive agent loses its message. The sink copies agent 1 and its own action has no effect; the source holds the message exactly when it acts. At reset all agents hold the message ($s_i = 1$).

Reward. Agent i receives

$$r_i = w_i s_i, \quad w_0 = 1.0, \quad w_i = 0.1 \quad (i > 0), \quad (5.2)$$

evaluated on the new states, and the team reward is $R = \sum_i r_i$. The maximum per step is $1 + 0.1(N - 1)$, which the “always relay” policy ($a_i = 1$ for all i) attains at every step.

Episode end. `terminated` is always `False`; `truncated` becomes `True` after `max_iter` steps.

5.1.2 Spaces

Action. The joint action can be given in two formats, both of which `step()` always accepts:

- an integer in `Discrete(2**N)` whose bit i is the action of agent i (the declared space for `action_space_type="discrete"`, the default; only possible for $N \leq 62$);
- an array of N values in $\{0, 1\}$ (integer or boolean), the declared space `MultiBinary(N)` for `action_space_type="multibinary"`, which has no limit on N .

Out-of-range integers, arrays of the wrong shape and values other than 0 and 1 raise `ValueError`.

Observation. `Box(0, 3, (N, 2k+1), float32)` with $k = \text{n_obs_neighbors}$. The state vector is padded with k boundary cells on each side, and row i is the window of the $2k + 1$ cells centred on agent i :

Table 5.1: LineMsg observation (row i).

Column	Content
0 : k	states of agents $i - k, \dots, i - 1$ (left neighbours, towards the sink)
k	own state s_i
k+1 : 2k+1	states of agents $i + 1, \dots, i + k$ (right neighbours, towards the source)
Values	0: no message, 1: message, 2: boundary cell beyond the end of the line

5.1.3 Parameters and info

Table 5.2: Constructor parameters of `LineMsgEnv`.

Parameter	Default	Description
<code>num_agents</code>	10	Number of agents N (≥ 2 : a sink and a source, plus any number of relays).
<code>n_obs_neighbors</code>	1	Observation radius k ; 0 is treated as 1 with a warning.
<code>max_iter</code>	50	Episode length; reaching it sets <code>truncated</code> .
<code>render_mode</code>	None	None, "human", "rgb_array" or "ansi".
<code>action_space_type</code>	"discrete"	Declared action space: "discrete" (<code>Discrete(2**N)</code> , $N \leq 62$) or "multibinary" (<code>MultiBinary(N)</code>).

The info dictionary of `reset()` and `step()` contains a single key, `"agent_rewards"`: the float64 per-agent rewards r_i (zeros at reset). The attribute `state` holds the padded state vector (agent i is `state[k + i]`), `actions` the last joint action with the same padding (2 before the first step), and `num_moves` the number of steps taken. `step()` before `reset()` raises `env_lib.ResetNeededError`.

5.1.4 Rendering

The dashboard (Figure 5.1) shows the line of agents at the top, coloured by state, with a halo under agents that act and arrows on the active links, which point in the direction of message flow towards the sink. Below it a space-time raster (one row per step, one column per agent) makes message fronts visible, and the right panel traces the team reward with its maximum. The header reports the step, the last reward, the episode return and the number of agents holding the message. `render_mode="ansi"` returns a text line with the action and state of every agent.



Figure 5.1: LineMsg dashboard (12 agents, step 30 of 50). Top: agent states and active links. Bottom left: space-time raster of the agent states. Bottom right: team reward per step.

5.1.5 Usage

```
import numpy as np
from env_lib.linemsg_env import LineMsgEnv

env = LineMsgEnv(num_agents=10, action_space_type="multibinary")
obs, info = env.reset(seed=0) # obs: (10, 3) float32
rng = np.random.default_rng(0)
episode_return = 0.0
while True:
    action = (rng.random(env.num_agents) < 0.9).astype(np.int64) # duty cycle
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(episode_return, info["agent_rewards"])

env.reset(seed=1)
env.step(0b111111110) # integer action: all agents but the sink act
```

The example script compares a random policy, duty-cycled relays and the always-relay policy and can record an episode:

```
python examples/linemsg_demo.py --policy relay --save renders/linemsg.gif
python examples/linemsg_demo.py --num-agents 30 --episodes 50
python examples/linemsg_demo.py --render-mode ansi
```

Over 20 episodes with $N = 10$ and `max_iter=50` the mean returns are 44.1 for uniformly random actions, 85.2 for duty-cycled relays that act with probability 0.9, and 95.0 for the always-relay policy (the maximum).

5.2 WirelessComm

5.2.1 Model

$G_x \times G_y$ agents sit on a grid (`grid_x` rows, `grid_y` columns); agent (i, j) has index $iG_y + j$. Access points sit at the $(G_x - 1)(G_y - 1)$ interior corners of the grid: access point (a, b) lies between agents (a, b) , $(a + 1, b)$, $(a, b + 1)$ and $(a + 1, b + 1)$. Every agent holds a packet queue with $D = \text{ddl}$ deadline slots; slot 0 holds the packet with the earliest deadline. Each agent chooses one of five actions per step (Table 5.3).

Table 5.3: WirelessComm actions of agent (i, j) . A direction without an access point (at the border of the grid) is a failed attempt.

Action	Meaning	Access point
0	stay idle	none
1	transmit up-left	$(i - 1, j - 1)$
2	transmit down-left	$(i, j - 1)$
3	transmit up-right	$(i - 1, j)$
4	transmit down-right	(i, j)

Transition. Let ℓ_m be the load of access point m , the number of agents that transmit to it in the step; every transmitting agent counts, whether or not it holds a packet. The transmission of agent k is *eligible* when the agent transmits, holds at least one packet, its access point exists and that access point has load exactly $\ell_m = 1$. Each eligible transmission succeeds with probability q ; one uniform draw is made per eligible agent, in agent order. A success removes the agent's earliest-deadline packet and earns reward 1. Afterwards every queue shifts one slot towards the deadline (packets in slot 0 expire and are lost), and a new packet arrives in the last slot with probability p . At reset every queue slot is occupied independently with probability $1/2$.

Reward and outcomes. The per-agent reward is 1 for a delivered packet and 0 otherwise; the team reward is the number of packets delivered in the step. The outcome of every agent is reported in `info["outcomes"]` (Table 5.5): idle (0), success (1), collision (2: the access point exists and has load ≥ 2) or lost (3: any other failed attempt, that is no packet, no access point or a channel loss).

Episode end. `terminated` is always `False`; `truncated` becomes `True` after `max_iter` steps.

5.2.2 Spaces

Action. `MultiDiscrete([5] * n)` with $n = G_x G_y$. An action of shape `(n,)` or `(grid_x, grid_y)` is accepted; other shapes and values outside $0, \dots, 4$ raise `ValueError`.

Observation. `Box(0, 2, (n, D * w^2), float32)` with $w = 2k+1$ and $k = \text{n_obs_neighbors}$. The queues form a (D, G_x, G_y) array that is padded with k cells of value 2 on every side of the grid. Row m of the observation is the (D, w, w) window centred on agent m , flattened in C order (slot, then row, then column). Values are 0 (empty slot), 1 (packet) and 2 (outside the grid). The agent's own slot d is column $dw^2 + kw + k$. The default configuration ($D = 2, k = 1$) has 18 columns.

5.2.3 Parameters and info

Table 5.4: Constructor parameters of `WirelessCommEnv`.

Parameter	Default	Description
<code>grid_x, grid_y</code>	6, 6	Number of agent rows and columns (each ≥ 2 , so that at least one access point exists).
<code>ddl</code>	2	Deadline horizon D : queue slots per agent (≥ 1).
<code>packet_arrival_probability</code>	0.8	Arrival probability p of a new packet per agent and step.
<code>success_transmission_probability</code>	0.8	Success probability q of an eligible transmission.
<code>n_obs_neighbors</code>	1	Observation radius k (Chebyshev distance, ≥ 0).
<code>max_iter</code>	50	Episode length; reaching it sets <code>truncated</code> .
<code>render_mode</code>	None	None, "human", "rgb_array" or "ansi".

Table 5.5: `WirelessComm` info dictionary (the same keys, all zero, at reset).

Key	Content
<code>agent_rewards</code>	<code>float64</code> array <code>(n,)</code> : 1 for a delivered packet, else 0
<code>outcomes</code>	<code>int8</code> array <code>(n,)</code> : 0 idle, 1 success, 2 collision, 3 lost (constants <code>OUTCOME_IDLE</code> , <code>OUTCOME_SUCCESS</code> , <code>OUTCOME_COLLISION</code> , <code>OUTCOME_LOST</code> and <code>OUTCOME_LABELS</code> in <code>env_lib.wireless_comm_env.wireless_comm_env</code>)
<code>ap_load</code>	<code>int64</code> array <code>(grid_x - 1, grid_y - 1)</code> : number of agents that transmitted to each access point

The environment also exposes `last_outcomes` and `last_ap_load` (the values of the last step), `access_point_mapping(i, j, action)`, which returns the access point (a, b) reached by agent (i, j) or `(None, None)`, and `check_transmission_fail(a, b, load)`. `step()` before `reset()` raises `env_lib.ResetNeededError`.

5.2.4 Rendering

The dashboard (Figure 5.2) draws the agent grid on the left. Every agent is a circle coloured by the outcome of its last action, with a small queue bar below it (one cell per deadline slot; the left cell expires first). Access points are squares at the interior corners, coloured by their load (idle, one transmitter, collision), and a line joins every transmitting agent to the access point it chose,

coloured by the outcome; the line of an attempt towards a missing access point ends outside the grid. The right panels show the packets delivered per step with a rolling mean and the rolling collision count, and the cumulative return. `render_mode="ansi"` returns the action and queue of every agent as text.

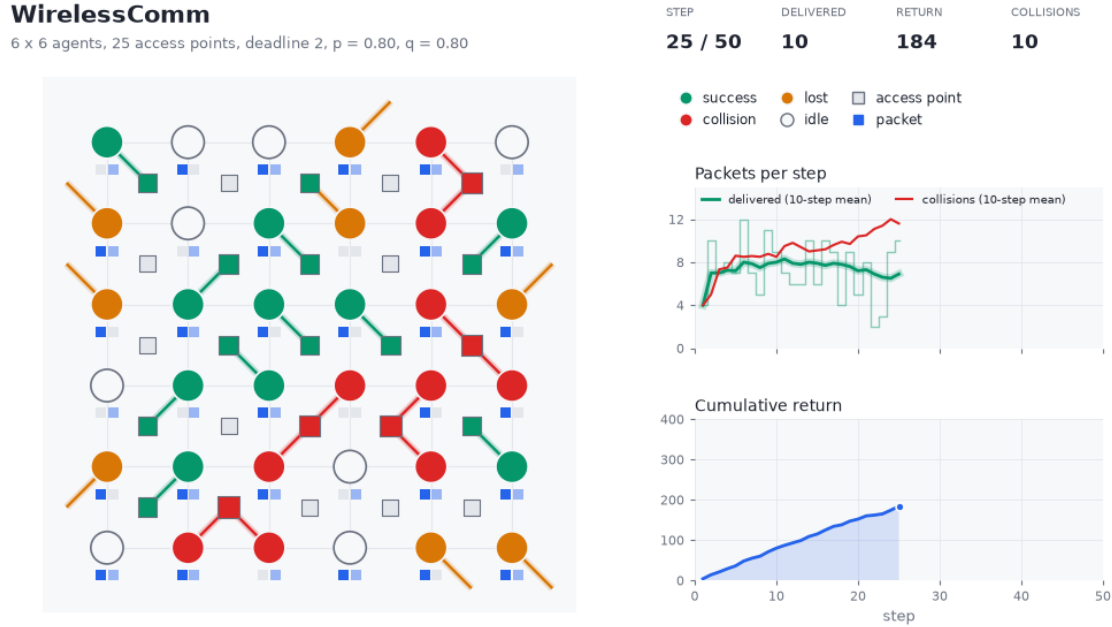


Figure 5.2: WirelessComm dashboard (6 × 6 agents, 25 access points, step 25 of 50). Left: agents coloured by outcome with their packet queues, access points coloured by load and transmission links. Right: packets delivered and collisions per step (10-step means) and the cumulative return.

5.2.5 Usage

The following policy is a collision-free rotating schedule: at step t every agent that holds a packet uses the same direction $(4, 1, 2, 3)[t \bmod 4]$ if an access point exists in that direction. A common direction maps agents to access points one-to-one, so no collisions occur.

```
import numpy as np
from env_lib.wireless_comm_env import WirelessCommEnv

env = WirelessCommEnv(grid_x=6, grid_y=6)
obs, info = env.reset(seed=0) # obs: (36, 18) float32
k, w = env.n_obs_nghbr, env.window
own = [d * w * w + k * w + k for d in range(env.ddl)] # own queue columns
directions = (4, 1, 2, 3)
cells = [(i, j) for i in range(env.grid_x) for j in range(env.grid_y)]
reachable = {d: np.array([env.access_point_mapping(i, j, d)[0] is not None
                        for i, j in cells])
             for d in directions}
episode_return = 0.0
for t in range(env.max_iter):
    d = directions[t % 4]
    has_packet = obs[:, own].max(axis=1) > 0.5
    action = np.where(has_packet & reachable[d], d, 0)
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
print(episode_return, np.bincount(info["outcomes"], minlength=4))
```

The example script compares uniformly random actions, p -persistent slotted ALOHA and the rotating schedule:

```
python examples/wireless_comm_demo.py --policy schedule \
    --save renders/wireless.gif
python examples/wireless_comm_demo.py --grid 8 8 --tau 0.5 --episodes 50
```

Over 20 episodes on the 6×6 grid the mean returns are 385 for random actions, 398 for ALOHA with transmit probability 0.6 and 910 for the rotating schedule, which causes no collisions.

5.3 Registered ids

Table 5.6: Registered ids of the networked environments.

Id	Class	Configuration
LineMsg-v0	LineMsgEnv	defaults (10 agents)
WirelessComm-v0	WirelessCommEnv	defaults (6×6 grid, 36 agents)
WirelessComm-v1	WirelessCommEnv	grid_x=4, grid_y=4 (16 agents)

5.4 Reproducibility and performance

Both environments draw all randomness (relay successes in LineMsg; initial queues, transmission successes and packet arrivals in WirelessComm) from `self.np_random`, seeded by `reset(seed=...)`. Seeded trajectories are bit-identical to those of the implementation before version 1.0, so earlier results can be reproduced exactly.

Measured on the development machine without rendering, a LineMsg step with 10 agents takes 0.013 ms (0.019 ms before 1.0), and a WirelessComm step takes 0.05 ms on the 6×6 grid (0.154 ms before) and 0.046 ms on a 12×12 grid (0.56 ms before). Rendering a dashboard frame in "rgb_array" mode takes about 15 ms for either environment; the history needed by the dashboards is only recorded in the "human" and "rgb_array" modes.

5.5 Changes in 1.0

The seeded dynamics of both environments are unchanged. Interface changes are summarised below; see Appendix B for migration advice.

LineMsg.

- Action decoding and observations are vectorised.
- The joint action may be an integer or an array of N bits. The new option `action_space_type="multibinary"` declares a `MultiBinary(N)` space and supports more than 62 agents.
- `step()` before `reset()` raises instead of resetting implicitly.
- The former text output is `render_mode="ansi"`; "human" and "rgb_array" show the new dashboard.

WirelessComm.

- `step()` is fully vectorised (up to 37 times faster on large grids).
- Fixed: the stored actions were never updated, so rendering showed no actions. Per-agent outcomes and access-point loads are reported in `info`.
- Arguments and actions are validated; "ansi" text mode and a new dashboard were added.
- `env_lib.wireless_comm_env.register()` is deprecated; the ids are registered when `env_lib` is imported.

Chapter 6

Pistonball

Pistonball is a cooperative physics game. A row of pistons forms the floor of a walled arena and a ball is dropped near the right wall; every piston is an agent that moves up or down, and together the pistons must roll the ball to the left wall. Each piston observes the ball only when it is close to it, and the reward of a piston depends on the ball's progress while it observes the ball. The environment is used to study cooperation under local observability, credit assignment with local rewards, scalability in the number of agents and the effect of the observation radius κ on learning. The physics is simulated with `pymunk`; `pygame` is only needed for rendering and keyboard control (extra `pistonball`).

The class is `env_lib.pistonball_env.PistonballEnv`; the package also exports `ManualPolicy`, `PistonballRenderer` and `PistonballLayout`.

6.1 Arena and physics

All coordinates are pygame screen pixels with y growing downwards. With n pistons the screen is $W = 160 + 40n$ pixels wide and 560 pixels high; the walls are 80 pixels from each screen edge, so the arena spans $x \in [80, W - 80]$ and $y \in [80, 480]$. The left wall is the goal.

- **Pistons.** Piston i is 40 pixels wide with its left edge at $x = 80 + 40i$. Its height y_i (the y coordinate of the piston head) moves by 4 pixels per unit action and is limited to $y_i \in [387, 451]$, a travel of 64 pixels around the middle position 419. At reset every piston starts at $451 - d_i$ with d_i drawn uniformly from $\{0, 4, \dots, 28\}$.
- **Ball.** A disc of radius 40 pixels with mass `ball_mass`, friction `ball_friction` and elasticity `ball_elasticity`. It starts at $x = W - 150 + \delta_x$, $y = 370 + \delta_y$, where $\delta_x \in \{-30, \dots, 30\}$ and $\delta_y \in \{-15, \dots, 15\}$ are uniform integers when `random_drop=True` and zero otherwise, with an angular velocity drawn uniformly from $[-6\pi, 6\pi]$ rad/s when `random_rotate=True`.
- **Simulation.** Gravity is 750 px/s² downwards, walls and pistons have friction 0.64, and one environment step is one physics step of $\Delta t = 1/20$ s. The pistons are kinematic bodies that are moved to their new height before the physics step; the ball speed is capped at 1000 px/s and its position is kept inside the screen.

6.2 Spaces

Action. One value per piston, stacked in an array of shape `(n,)`:

- continuous (`continuous=True`, default): `Box(-1, 1, (n,), float32)`; the value is clipped to $[-1, 1]$ and moves the piston by $4a_i$ pixels, where $+1$ moves it up;
- discrete (`continuous=False`): `MultiDiscrete([3] * n)` with $0 = \text{down}$, $1 = \text{stay}$ and $2 = \text{up}$ (4 pixels).

Actions of the wrong shape, continuous actions with NaN or infinite values, and discrete values outside $\{0, 1, 2\}$ raise `ValueError`.

Observation. `float32` array of shape $(n, 7)$ in an unbounded `Box`. Row i is the observation of piston i (Table 6.1). The piston under the ball has index

$$p(x) = \text{clip}(\lfloor (x - 85)/40 \rfloor, 0, n - 1), \quad (6.1)$$

and piston i observes the ball when $|i - p(x)| \leq \kappa$, where x is the ball centre and $\kappa = \text{kappa}$. Columns 2–6 are zero for pistons that do not observe the ball. The method `observable_mask()` returns the boolean mask of the observing pistons; its optional argument `ball_x` evaluates the mask for another ball position.

Table 6.1: Pistonball observation (row i). (x, y) is the ball centre, (v_x, v_y) its velocity in px/s and $\dot{\phi}$ its angular velocity.

Column	Value	Meaning
0	$(y_i - 419)/32$	piston height in $[-1, 1]$; $+1$ is the lowest position
1	$(5 + 40i)/(40n)$	piston position along the arena in $[0, 1]$
2	$(x - 80)/(40n)$	ball x position (observers only)
3	$(y - 80)/400$	ball y position (observers only)
4	$v_x/15$	ball x velocity (observers only)
5	$v_y/8$	ball y velocity (observers only)
6	$\dot{\phi}/8$	ball angular velocity (observers only)

6.3 Reward and episode end

Let $b_t = \lfloor x_t - 40 \rfloor$ be the x coordinate of the ball’s left edge after step t and $\Delta b = b_{t-1} - b_t$ its leftward displacement. The ball term rewards leftward progress at half rate and penalises rightward motion in full:

$$g_t = \begin{cases} 0.5 \Delta b, & \Delta b > 0, \\ \Delta b, & \text{otherwise.} \end{cases} \quad (6.2)$$

The local reward of piston i is

$$r_i = \tau + g_t \mathbf{1}[i \in O(b_{t-1}) \cup O(b_t)] + \rho_T \mathbf{1}[\text{goal}, i \in O(x_t)] + \rho_L \mathbf{1}[\text{goal}, i = 0] + \mu \frac{|\Delta y_i|}{4} \mathbf{1}[|\Delta y_i| > \epsilon_\mu], \quad (6.3)$$

where $O(z) = \{i : |i - p(z)| \leq \kappa\}$ is the set of pistons within κ hops of the piston under position z (the ball term uses the left-edge positions b , the goal bonus the ball centre x_t), $\tau = \text{time_penalty}$, $\rho_T = \text{termination_reward}$, $\rho_L = \text{leftmost_piston_reward}$, $\mu = \text{movement_penalty}$, $\epsilon_\mu = \text{movement_penalty_threshold}$ and Δy_i the movement of piston i in pixels. The team reward returned by `step()` is $R = \sum_i r_i$.

Goal and episode end. With `terminated_condition=True` (default) the goal is reached when the ball's left edge will reach the left wall within one step, $x_t - 40 + v_x \Delta t \leq 80.5$; the episode then terminates. `truncated` becomes `True` after `max_cycles` steps. For example, with `leftmost_piston_reward=1.0` and the default termination reward, piston 0 receives $0.5 + 1.0 = 1.5$ on reaching the goal, the other pistons that observe the ball receive 0.5, and the remaining pistons receive no bonus.

6.4 Parameters

Table 6.2: Constructor parameters of `PistonballEnv`.

Parameter	Default	Description
<code>n_pistons</code>	20	Number of pistons (agents), ≥ 1 .
<code>time_penalty</code>	-0.1	Reward τ added to every piston at every step.
<code>continuous</code>	<code>True</code>	Continuous actions in $[-1, 1]$; <code>False</code> for <code>MultiDiscrete([3] * n)</code> .
<code>random_drop</code>	<code>True</code>	Randomise the initial ball position (± 30 px horizontally, ± 15 px vertically).
<code>random_rotate</code>	<code>True</code>	Random initial angular velocity in $[-6\pi, 6\pi]$ rad/s.
<code>ball_mass</code>	0.75	Ball mass (> 0).
<code>ball_friction</code>	0.3	Ball friction (≥ 0).
<code>ball_elasticity</code>	1.5	Ball elasticity (≥ 0).
<code>max_cycles</code>	125	Episode length; reaching it sets <code>truncated</code> .
<code>render_mode</code>	<code>None</code>	<code>None</code> , <code>"human"</code> (pygame window) or <code>"rgb_array"</code> .
<code>movement_penalty</code>	0.0	Reward μ per 4 pixels of piston movement, typically negative; 0 disables it.
<code>movement_penalty_threshold</code>	0.01	Movements of at most this many pixels are not penalised.
<code>kappa</code>	1	Observation radius κ in pistons (≥ 0).
<code>terminated_condition</code>	<code>True</code>	End the episode when the ball reaches the left wall.
<code>leftmost_piston_reward</code>	0.0	Bonus ρ_L for piston 0 when the goal is reached.
<code>termination_reward</code>	0.5	Bonus ρ_T for every piston that observes the ball when the goal is reached.

Physics presets. The ball parameters change the character of the task considerably. A heavy, damped ball (`ball_mass=2.0`, `ball_friction=0.8`, `ball_elasticity=0.5`) rolls slowly and rewards coordinated ramps; a light, bouncy ball (`ball_mass=0.5`, `ball_friction=0.1`, `ball_elasticity=2.0`) is harder to control.

Info dictionary. `step()` returns `"local_rewards"` and `"agent_rewards"` (both the float64 local rewards r_i , shape `(n,)`) and `"total_reward"` (their sum, equal to the returned reward). The info dictionary of `reset()` is empty. `step()` before `reset()` raises `env_lib.ResetNeededError`.

6.5 Keyboard control

`ManualPolicy(env, agent_id=0, show_obs=False)` lets a person control one piston at a time in a window. Create the environment with `render_mode="human"` so that the pygame window

exists and receives the key presses, and call the policy once per step: **W** or **Up** raises the selected piston, **S** or **Down** lowers it (hold the key to keep moving), **A/Left** and **D/Right** select the previous or next piston, **Backspace** resets the environment, and **Esc** or closing the window sets `policy.running` to `False`. All other pistons stay in place.

```
from env_lib.pistonball_env import ManualPolicy, PistonballEnv

env = PistonballEnv(n_pistons=8, render_mode="human")
policy = ManualPolicy(env)
obs, info = env.reset(seed=0)
while policy.running:
    obs, reward, terminated, truncated, info = env.step(policy(obs))
    if terminated or truncated:
        obs, info = env.reset()
env.close()
```

6.6 Rendering

The renderer draws with pygame and needs no image assets (Figure 6.1). It shows a gradient background with shaded walls and a highlighted goal strip on the left wall, the pistons as rounded rectangles (pistons that observe the ball in the accent colour with a glow), the ball with shading, a rotation marker and a fading motion trail, a heads-up display with the step counter, episode return, ball x velocity and number of observing pistons, and a progress bar with the ball's distance to the goal. Frames have the screen size, $(160 + 40n) \times 560$ pixels. The `"rgb_array"` mode never initialises the display and works on headless machines; `"human"` mode opens a window capped at 20 frames per second. Colours follow the active theme (Section 3.5).



Figure 6.1: Pistonball with 12 pistons (light theme) at the moment the ball reaches the goal. The three pistons that observe the ball ($\kappa = 1$) are highlighted; the ramp formed by the pistons rolls the ball to the left wall.

6.7 Registered id

Pistonball-v0 creates `PistonballEnv` with its defaults (20 pistons, continuous actions, $\kappa = 1$). Creating it requires `pymunk`; rendering also requires `pygame`.

6.8 Usage

The following heuristic reads the ball position from the observing pistons and shapes the pistons into a ramp that falls towards the left: pistons right of the ball move up, pistons left of it move down.

```
import numpy as np
from env_lib.pistonball_env import PistonballEnv

env = PistonballEnv(n_pistons=12, kappa=1)
obs, info = env.reset(seed=0)           # obs: (12, 7) float32
episode_return = 0.0
while True:
    action = np.zeros(env.n_pistons, dtype=np.float32)
    observers = np.any(obs[:, 2:] != 0.0, axis=1)
    if observers.any():
        ball_x = obs[observers, 2].mean()      # (x - 80) / (40 n)
        offset = (obs[:, 1] - ball_x) * env.n_pistons # in piston widths
        action = np.clip(2.0 * offset, -1.0, 1.0).astype(np.float32)
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(env.frames, terminated, episode_return)  # goal after 99 steps
env.close()
```

The example script evaluates a similar heuristic against random actions, records episodes and offers keyboard control:

```
python examples/pistonball_demo.py --save renders/pistonball.gif
python examples/pistonball_demo.py --discrete --kappa 2 --movement-penalty -0.05
python examples/pistonball_demo.py --manual --n-pistons 8
```

Over 20 episodes with the default configuration the heuristic of the script reaches a mean return of 857.1 ± 254.3 and brings the ball to the goal in 95% of the episodes, whereas random actions obtain -290.5 and never reach the goal.

6.9 Reproducibility and performance

`reset(seed=...)` seeds `self.np_random`, from which the initial piston heights, the ball position and its angular velocity are drawn; the physics itself is deterministic. The random draws are made in the same order as before version 1.0, so seeded episodes start from the same states as in earlier versions.

Measured on the development machine with 20 pistons, a step without rendering takes 0.06 ms (1.11 ms before 1.0, when the screen was drawn at every step). Rendering an `"rgb_array"` frame takes about 3 ms.

6.10 Changes in 1.0

See Appendix B for migration advice.

- Nothing is drawn unless rendering is requested; pygame is initialised lazily. Together with vectorised observations and rewards this makes steps 16 to 30 times faster.
- In discrete mode the declared action space is `MultiDiscrete([3] * n)`, one entry per piston. The per-piston action arrays accepted by `step()` are unchanged; the former single-integer `Discrete(3**n)` declaration was removed.
- Actions are validated (shape, range, NaN), and `step()` before `reset()` raises.
- New vector renderer (shaded pistons, highlighted κ -hop observers, ball trail, heads-up display, progress bar); the PNG sprites were removed. The former drawing methods (`draw()`, `draw_background()`, `draw_pistons()`, `enable_render()`) are deprecated no-ops.
- Compatible with pymunk 6 and 7.

Chapter 7

Consensus and Formation Control

`env_lib.consensus_env.ConsensusEnv` is a networked multi-agent control benchmark, new in version 1.0. N agents move in a bounded plane and must either agree on a common position (*consensus* or *rendezvous*) or assume a geometric formation up to translation (*formation control*). Every agent only sees the agents it is connected to in a communication graph, which is either fixed or rebuilt every step from the agents' distances. The environment connects multi-agent reinforcement learning with the classical theory of distributed control: the Laplacian consensus protocol is included as a baseline controller, the algebraic connectivity of the graph is reported at every step, and the difficulty can be tuned through the topology, the dynamics (single or double integrators), noise and the formation shape. Typical research uses are learning distributed controllers from local observations, studying the effect of graph connectivity and time-varying graphs on learning, and comparing learned policies with model-based protocols.

7.1 Model

Agent i has position $x_i \in \mathbb{R}^2$, velocity $v_i \in \mathbb{R}^2$ and a formation offset d_i . The offsets are zero for consensus and the slots of the chosen formation shape, centred at the origin, for formation control. The task is solved when the shifted positions $y_i = x_i - d_i$ agree. The task error

$$e = \frac{1}{N} \sum_{i=1}^N \|y_i - \bar{y}\|^2, \quad \bar{y} = \frac{1}{N} \sum_{j=1}^N y_j, \quad (7.1)$$

measures the disagreement; the task is solved when $e < \text{tolerance}$.

Dynamics. The action u_i of agent i is clipped to $[-u_{\max}, u_{\max}]^2$ with $u_{\max} = \text{max_control}$. With time step $\Delta t = \text{dt}$, noise intensity $\sigma = \text{noise_std}$, $w_i \sim \mathcal{N}(0, I_2)$ and the arena $[-a, a]^2$ ($a = \text{arena_size}$):

- single integrator (`dynamics="single"`, the action is a velocity):

$$x_i \leftarrow \text{clip}_{[-a, a]}(x_i + \Delta t u_i + \sigma \sqrt{\Delta t} w_i); \quad (7.2)$$

the reported velocity is the realised one, $v_i = (x_i^+ - x_i)/\Delta t$;

- double integrator (`dynamics="double"`, the action is an acceleration, damping $c = \text{damping}$):

$$v_i \leftarrow v_i + \Delta t (u_i - c v_i) + \sigma \sqrt{\Delta t} w_i, \quad x_i \leftarrow \text{clip}_{[-a, a]}(x_i + \Delta t v_i) \quad (7.3)$$

(semi-implicit Euler); a velocity component is set to zero when the arena wall stops the agent.

Communication graph. The agents communicate over an undirected graph with adjacency matrix A and neighbour sets $\mathcal{N}_i$ (Table 7.1). Static graphs are built once at construction; the proximity graph is rebuilt after every step. The environment reports the algebraic connectivity $\lambda_2(L)$, the second smallest eigenvalue of the graph Laplacian $L = D - A$ (D the diagonal degree matrix). It is positive exactly when the graph is connected and bounds the convergence rate of linear consensus.

Table 7.1: Communication topologies.

topology	Graph
"ring"	i is connected to $i \pm 1 \bmod N$
"line"	i is connected to $i \pm 1$ (a path)
"star"	agent 0 is the hub connected to all others
"complete"	all pairs are connected
"erdos_renyi"	every pair is connected independently with probability <code>edge_probability</code> ; the graph is resampled (up to 1000 times) until it is connected, using a generator seeded with <code>graph_seed</code>
"proximity"	disk graph: i and j are connected when $\ x_i - x_j\ \leq \text{comm_radius}$; rebuilt after every step and may disconnect during an episode

Formation shapes. `make_formation(shape, n_agents, radius)` returns the offsets d_i , centred at the origin; all shapes have a characteristic extent of $2R$ with $R = \text{formation_radius}$:

- "circle": regular polygon of radius R , agent 0 at the top;
- "line": evenly spaced on a horizontal segment of length $2R$;
- "grid": row-major grid with $\lceil \sqrt{N} \rceil$ columns whose longer side spans $2R$;
- "wedge": V shape pointing up, agent 0 at the apex and the others alternating between the two arms (half opening angle 35 degrees, arm length $2R$).

A formation that does not fit into the arena raises `ValueError`.

Initial state. Positions are drawn uniformly from $[-0.8a, 0.8a]^2$ and velocities are zero. For the proximity topology the draw is conditioned on a connected initial graph (rejection sampling in vectorised batches); if `comm_radius` is so small that no connected placement is found, the agents are placed sequentially, each within 0.95comm_radius of a previously placed agent, and a warning is issued once. `reset(options={"positions": ..., "velocities": ...})` starts from a given state (arrays of shape $(N, 2)$; positions are clipped to the arena).

7.2 Spaces

Action. `Box(-max_control, max_control, (N, 2), float32)`: one velocity command (single integrator) or acceleration (double integrator) per agent. Any array with $2N$ entries is reshaped to $(N, 2)$; values are clipped, and NaN or infinite values raise `ValueError`.

Observation. An unbounded `Box` of shape $(N, 6 + 5K)$, `float32`, where $K = \text{max_neighbors}$ is the number of neighbour slots. Row i has the layout of Table 7.2. The slots hold the neighbours $j \in \mathcal{N}_i$ sorted by physical distance $\|x_j - x_i\|$ (ties by index); unused slots are zero with mask 0. If an agent has more neighbours than slots, the farthest ones are left out of the observation; they

still count in the reward and in the baseline controller. The property `env.observation_layout` returns the column slices ("position", "velocity", "offset", "neighbors"), and `obs[:, layout["neighbors"]].reshape(N, K, 5)` gives the slots, whose columns are described by `ConsensusEnv.neighbor_slot_layout`.

Table 7.2: Consensus observation (row i ; slot $s = 0, \dots, K - 1$).

Columns	Layout key	Content
0:2	position	x_i
2:4	velocity	v_i
4:6	offset	d_i (zeros for consensus)
6+5s : 6+5s+2	neighbors	slot s : $(x_j - d_j) - (x_i - d_i) = y_j - y_i$
6+5s+2 : 6+5s+4	neighbors	slot s : $v_j - v_i$
6+5s+4	neighbors	slot s : valid mask (1 or 0)

K defaults to the maximum degree of a static graph (2 for the default ring, so the default observation has shape (8, 16)) and to $\min(6, N - 1)$ for the proximity graph.

7.3 Reward and episode end

Agent i receives the negative mean squared disagreement with its neighbours and a quadratic control cost,

$$r_i = -\frac{1}{\max(|\mathcal{N}_i|, 1)} \sum_{j \in \mathcal{N}_i} \|y_i - y_j\|^2 - c_u \|u_i\|^2, \quad (7.4)$$

with $c_u = \text{control_cost}$ and the clipped action u_i , evaluated after the step on the new graph (isolated agents only pay the control cost). The team reward is $R = \frac{1}{N} \sum_i r_i$, plus `success_bonus` on the step on which the task error first drops below `tolerance`.

`terminated` is `True` when $e < \text{tolerance}$; `truncated` is `True` once `max_steps` steps were taken. Both can be `True` on the last step.

7.4 Parameters

All constructor arguments are keyword-only.

Table 7.3: Constructor parameters of `ConsensusEnv`.

Parameter	Default	Description
<code>n_agents</code>	8	Number of agents N (≥ 2).
<code>task</code>	"consensus"	"consensus" (rendezvous) or "formation".
<code>topology</code>	"ring"	"ring", "line", "star", "complete", "erdos_renyi" or "proximity".
<code>dynamics</code>	"single"	"single" (velocity input) or "double" (acceleration input with damping).
<code>dt</code>	0.1	Integration time step (> 0).
<code>max_steps</code>	200	Episode length before truncation.
<code>arena_size</code>	10.0	Positions are clipped to $[-a, a]^2$ (> 0).
<code>max_control</code>	1.0	Bound of every action component (> 0).
<code>control_cost</code>	0.01	Weight c_u of the control penalty (≥ 0).

Parameter	Default	Description
<code>noise_std</code>	0.0	Noise intensity σ ; each step adds noise with standard deviation $\sigma\sqrt{\Delta t}$ to the positions (single) or velocities (double).
<code>tolerance</code>	0.05	Success threshold on the task error (> 0 , squared distance units).
<code>success_bonus</code>	10.0	Team-reward bonus paid once, on the step the task is solved.
<code>formation_shape</code>	"circle"	"circle", "line", "grid" or "wedge" (formation task only).
<code>formation_radius</code>	3.0	Scale R of the formation (> 0).
<code>comm_radius</code>	4.0	Communication range of the proximity topology (inclusive, > 0).
<code>edge_probability</code>	0.3	Edge probability of the Erdős–Rényi topology, in $(0, 1]$.
<code>graph_seed</code>	None	Seed of the Erdős–Rényi graph, sampled once at construction; None uses fresh entropy, so two instances may differ in graph and observation size.
<code>max_neighbors</code>	None	Number of neighbour slots K (≥ 1); defaults to the maximum degree of a static graph and to $\min(6, N - 1)$ for the proximity graph.
<code>damping</code>	0.5	Linear velocity damping c of the double integrator (≥ 0).
<code>render_mode</code>	None	None, "human" or "rgb_array".

7.5 Info dictionary and properties

Table 7.4: Consensus info dictionary (returned by `reset()` and `step()`).

Key	Content
<code>agent_rewards</code>	float64 array (N,) of the rewards r_i (zeros at reset)
<code>error</code>	task error e (7.1)
<code>algebraic_connectivity</code>	$\lambda_2(L)$ of the current graph
<code>adjacency</code>	boolean adjacency matrix (N, N) of the current graph
<code>success</code>	True once the task has been solved in the episode (sticky)
<code>step</code>	number of steps taken in the episode

The state is available through read-only properties (Table 7.5); `obs_dim` and `max_neighbors` are plain attributes.

The module `env_lib.consensus_env` also exports the helpers used by the environment, which are useful for analysis and for reference controllers:

- `make_topology(topology, n_agents, *, edge_probability=0.3, rng=None)`: adjacency matrix of a static graph;
- `proximity_adjacency(positions, comm_radius)`: disk-graph adjacency;
- `make_formation(shape, n_agents, radius=1.0)`: formation offsets;
- `algebraic_connectivity(adjacency)` and `is_connected(adjacency)`;
- the tuples `TASKS`, `TOPOLOGIES`, `DYNAMICS` and `FORMATION_SHAPES` of valid option values.

Table 7.5: Read-only properties of `ConsensusEnv` (arrays are copies).

Property	Content
<code>positions, velocities</code>	x_i and v_i , shape $(N, 2)$
<code>formation_offsets</code>	offsets d_i , shape $(N, 2)$ (zeros for consensus)
<code>adjacency, laplacian</code>	adjacency matrix A and Laplacian $L = D - A$ of the current graph
<code>algebraic_connectivity</code>	$\lambda_2(L)$ of the current graph
<code>task_error</code>	task error e
<code>step_count</code>	number of steps taken in the episode
<code>observation_layout</code>	column slices of an observation row

7.6 Baseline controller

`env.laplacian_policy(gain=1.0, velocity_gain=None)` returns the classical distributed consensus and formation feedback

$$u_i = -k \sum_{j \in \mathcal{N}_i} ((x_i - d_i) - (x_j - d_j)), \quad \text{that is} \quad u = -k L (x - d), \quad (7.5)$$

with gain k , clipped to the action bounds. For the double integrator the velocity feedback $-k_v v_i$ is added; the default $k_v = \max(0, \sqrt{k} - c)$ makes the total damping $c + k_v$ equal to $\sqrt{k}$. The feedback uses the full neighbour sets, even when the observation is truncated by `max_neighbors`. The discrete-time single-integrator loop is stable when $k \Delta t \lambda_{\max}(L) < 2$; the method warns once otherwise.

Table 7.6 reports the success rate of the controller with gain 1 for all topologies and task/dynamics combinations (default parameters otherwise; 100 episodes with reset seeds 0–99 and `graph_seed` equal to the reset seed). The controller solves every episode on connected static graphs. On the line, the slowest graph, some episodes run out of time, and on the proximity graph agents can lose contact.

Table 7.6: Success rate of `laplacian_policy(gain=1.0)` over 100 episodes, with the mean episode length in steps (truncated episodes count 200).

Topology	Consensus		Formation (circle)	
	single	double	single	double
ring	1.00 (84)	1.00 (81)	1.00 (97)	1.00 (86)
line	0.82 (160)	0.94 (154)	0.53 (183)	0.72 (174)
star	1.00 (73)	1.00 (80)	1.00 (85)	1.00 (85)
complete	1.00 (65)	1.00 (81)	1.00 (78)	1.00 (87)
Erdős–Rényi	1.00 (90)	1.00 (90)	1.00 (104)	1.00 (97)
proximity	0.83 (72)	0.70 (109)	0.55 (135)	0.12 (190)

7.7 Rendering

The dashboard (1000×560 pixels, Figures 7.1 and 7.2) shows the arena on the left: agents as coloured markers with halos, communication links whose opacity decreases with their length, fading trails of the recent positions, the formation slots around the current centroid (hollow markers, formation task only) and the centroid (cross). On the right, the task error is plotted on

a logarithmic scale with the tolerance as a dashed line over a shaded success band, and below it the algebraic connectivity λ_2 . The header reports the task, the team size, the step, the error, λ_2 and a status badge.

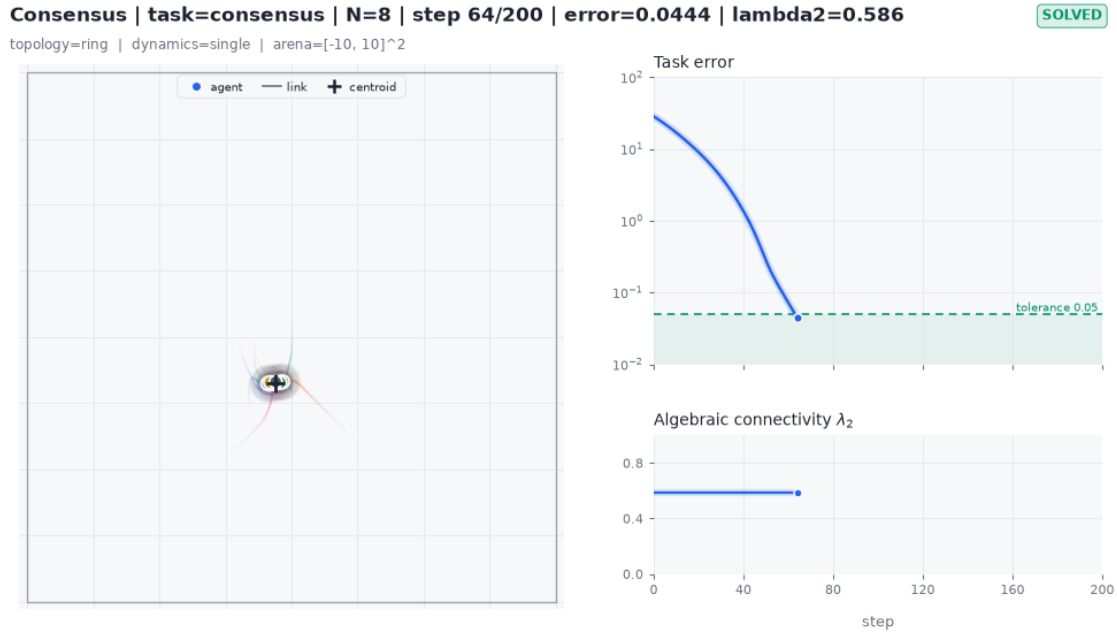


Figure 7.1: Consensus (rendezvous) of 8 single integrators on a ring, solved at step 64 by the Laplacian controller. The agents have gathered at the centroid; the task error has fallen below the tolerance and λ_2 of the static ring is constant.

7.8 Registered ids

Table 7.7: Registered ids of the consensus environment.

Id	Class	Configuration
Consensus-v0	ConsensusEnv	defaults (rendezvous, 8 agents on a ring)
Formation-v0	ConsensusEnv	task="formation" (circle of radius 3)

Table 7.7 lists the two ids. Keyword arguments of `env_lib.make` override the registered configuration, for example `env_lib.make("Formation-v0", formation_shape="wedge", topology="proximity")`.

7.9 Usage

```
import env_lib

env = env_lib.make("Formation-v0", formation_shape="wedge",
                  topology="proximity", dynamics="double")
core = env.unwrapped
obs, info = env.reset(seed=0)  # obs: (8, 36), K = 6 slots
layout = core.observation_layout
n, k = core.n_agents, core.max_neighbors
slots = obs[:, layout["neighbors"]].reshape(n, k, 5)  # neighbour slots
```

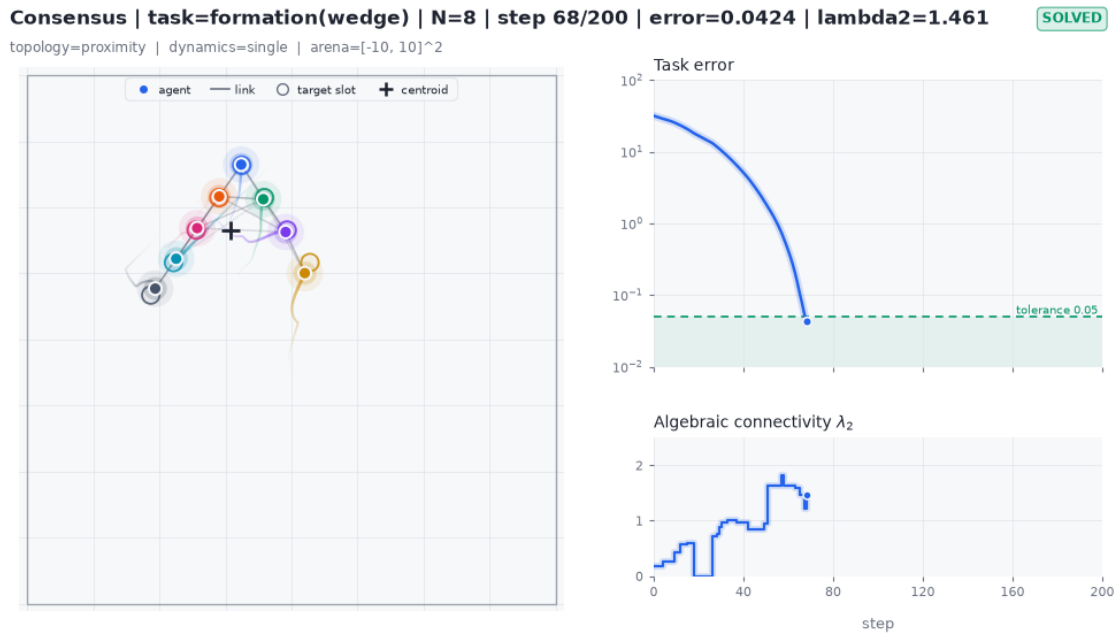


Figure 7.2: Wedge formation of 8 agents on a proximity graph, solved at step 68. Hollow markers are the formation slots around the centroid; λ_2 changes as links appear and disappear and drops to zero while the graph is disconnected.

```
episode_return = 0.0
while True:
    action = core.laplacian_policy(gain=1.0) #  $u = -gain L (x - d) - k_v v$ 
    obs, reward, terminated, truncated, info = env.step(action)
    episode_return += reward
    if terminated or truncated:
        break
print(info["step"], info["error"], info["success"], episode_return)
env.close()
```

The example script compares random actions with the Laplacian controller and records the controller:

```
python examples/consensus_demo.py
python examples/consensus_demo.py --task formation --save renders/formation.gif
python examples/consensus_demo.py --task formation --shape wedge \
    --topology proximity --dynamics double --save renders/wedge.gif
```

7.10 Reproducibility and performance

The initial positions and the process noise are drawn from `self.np_random`, seeded by `reset(seed=...)`. The Erdős–Rényi graph is sampled once at construction from `graph_seed`; pass a seed to obtain the same graph (and observation size) in every instance, for example in vectorised training environments.

Measured on the development machine without rendering, a step takes about 0.1 ms with the default 8 agents and 0.18 ms for a grid formation of 32 agents on a proximity graph. Rendering an "rgb_array" frame takes about 16–21 ms.

Chapter 8

AJLATT: Active Joint Localisation and Target Tracking

In AJLATT a team of mobile robots with range-bearing sensors has to localise itself and track a moving target on an occupancy grid map. Every robot runs an estimator for its own pose and for the target; estimates are exchanged within a limited communication range and fused with covariance intersection. The robots choose their velocities, and the reward drives the team to keep the target and its own poses well estimated while avoiding obstacles, the map border and each other. The environment is used to study active perception and information-driven control, cooperative localisation and distributed target tracking, and multi-agent reinforcement learning with continuous actions, partial observability and safety constraints.

The main entry points are exported by `env_lib.ajlatt_env`:

AJLATTEnv	the Gymnasium environment with joint multi-agent spaces (also available as <code>env_lib.AJLATTEnv</code>);
AJLATTConfig	a validated dataclass holding every parameter;
TeamRewardWrapper	a scalar-reward view for single-agent algorithms;
make	the backwards-compatible factory accepting the original keyword names;
maps	occupancy maps, the map builder and map plotting (Section 8.9).

8.1 Scenario and motion model

The team consists of `num_robots` robots (default 4) and `num_targets` targets (default 1). Target 0 moves according to a target policy (Section 8.6); further targets are static landmarks whose positions are estimated in 2-D. One step lasts $\Delta t = \text{sampling_period}$ (0.5 s), so the default episode of 120 steps covers 60 s. Robots and the target follow the unicycle model

$$\begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}_{k+1} = \begin{bmatrix} p_x \\ p_y \\ \theta \end{bmatrix}_k + \begin{bmatrix} v \Delta t \cos \theta_k \\ v \Delta t \sin \theta_k \\ \omega \Delta t \end{bmatrix}, \quad (8.1)$$

with the heading wrapped to $[-\pi, \pi)$. The true robot poses are propagated with noisy velocities $v \sim \mathcal{N}(v_{\text{cmd}}, \sigma_v^2)$, $\omega \sim \mathcal{N}(\omega_{\text{cmd}}, \sigma_\omega^2)$, where $(\sigma_v, \sigma_\omega) = (\text{sigma_vR}, \text{sigma_wR})$; with `process_noise_fixed=False` the noise scales with the speed, $\sigma_v = \sigma/\sqrt{2}$ and $\sigma_\omega = 2\sqrt{2}\sigma$ with $\sigma = \text{sigmaR} \cdot v_{\text{cmd}}$. The target moves exactly with the command of its policy, while the

robots predict their target beliefs with a noisy copy of that command (`sigma_vT`, `sigma_wT`, or `sigmapT.target_linear_velocity` in the speed-dependent case).

8.2 Sensing, communication and estimation

Beliefs. Robot i keeps a Gaussian belief $(\hat{x}_i, P_i)$ of its own pose and a belief $(\hat{x}_{i,t}^T, P_{i,t}^T)$ of every target t (a pose for target 0, a position for landmarks). At reset the means are drawn around the initial poses with the initial covariances: $P_i = \text{robot_init_cov}[i] I$ with heading variance 10^{-3} , and $P_{i,t}^T = \text{target_init_cov}[t] I$ with heading variance 0.1 for target 0.

Prediction. Every belief is predicted with an extended Kalman filter step for (8.1):

$$\hat{x} \leftarrow f(\hat{x}, u), \quad P \leftarrow \Phi P \Phi^\top + G Q G^\top, \quad Q = \text{diag}(\sigma_v^2, \sigma_\omega^2), \quad (8.2)$$

where Φ is the Jacobian of the motion with respect to the pose and $G = \Delta t \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix}$ its Jacobian with respect to the velocity noise.

Measurements. Robot ℓ measures the range and bearing $z = (\rho, \beta)$ of another robot or a target j when `sensor_r_min` $< \rho < \text{sensor_r_max}$, $|\beta| \leq \text{fov}/2$, the straight line between the two (true) positions does not cross an occupied cell of the grid (Bresenham ray casting), and robot ℓ 's estimated position lies inside the map. The measurement noise has standard deviations $\sigma_\rho = \text{sigma_p} \cdot \rho$ (with `range_noise_proportional=True`) or `sigma_r`, and $\sigma_\beta = \text{sigma_th}$. All three default to zero: by default the measurements are exact, and the uncertainty of the estimates stems from the motion noise and from the uncertainty of the measured party's own estimate.

Communication. Robots closer than `commu_r_max` (true distance) are neighbours; the neighbour set $\mathcal{N}_i$ of robot i includes i itself.

Update. With `use_update=True` (default) the robots are processed one after another in index order, and each update is applied in place, so later robots use the updated beliefs of earlier ones:

1. *Self-localisation.* Every measurement z of an entity j with belief $(\hat{x}_j, P_j)$ contributes the information pair

$$S = H_i^\top W H_i, \quad y = H_i^\top W (z - \hat{z} + H_i \hat{x}_i), \quad W = (R + H_j P_j H_j^\top)^{-1},$$

where $\hat{z}$ is the predicted measurement and H_i, H_j are the Jacobians of the range-bearing model with respect to the observer and the observed pose. These pairs and the prior $(P_i^{-1}, P_i^{-1} \hat{x}_i)$ are fused by covariance intersection.

2. *Target tracking.* For each target, the measurements of all neighbours $r \in \mathcal{N}_i$ that detect it contribute analogous information pairs (with the roles of observer and target exchanged); these are fused by covariance intersection in information form. The neighbours' prior beliefs of the target are fused in the same way, and the two results are combined by a final covariance intersection. If no neighbour detects the target, robot i 's belief is the fusion of the neighbours' priors.

With `use_update=False` the robots only dead-reckon.

Covariance intersection. Given information pairs (S_k, y_k) , $k = 1, \dots, K$, covariance intersection chooses weights c on the probability simplex that minimise the trace of the fused covariance,

$$c^* = \arg \min_{c \geq 0, \sum_k c_k = 1} \text{tr} \left(\left(\sum_k c_k S_k \right)^{-1} \right), \quad P = \left(\sum_k c_k^* S_k \right)^{-1}, \quad \hat{x} = P \sum_k c_k^* y_k. \quad (8.3)$$

The fused estimate is consistent without knowledge of the correlations between the sources. The solver is chosen with `ci_solver`: "newton" (default) is an active-set projected Newton method with the analytic gradient $g_k = -\text{tr}(PS_kP)$ and Hessian $H_{kl} = 2\text{tr}(PS_kPS_lP)$; its objective is never worse than that of "slsqp", the original solver (SciPy SLSQP with finite-difference gradients), which reproduces trajectories of the implementation before version 1.0 to about 10^{-6} .

8.3 Spaces

Action. Box of shape `(num_robots, 2)`, one command (v, ω) per robot with $0 \leq v \leq v_{\max}$ and $|\omega| \leq \omega_{\max}$, where $v_{\max} = \text{max_linear_velocity}$ (default 2 m/s) and $\omega_{\max} = \text{max_angular_velocity}$ (default $\pi/4$ rad/s). The per-robot space is `env.single_action_space`. Actions are not clipped unless `clip_actions=True` (the original environment did not clip). NumPy arrays and tensors are accepted; rows beyond `num_robots` are ignored with a warning, and fewer rows raise `ValueError`.

Observation. Box of shape `(num_robots, obs_dim)` with `obs_dim = 7 + 4(n_R - 1) + 5` for n_R robots (24 for the default team); the per-robot space is `env.single_observation_space`. Row i is computed from robot i 's beliefs; positions and headings of other entities are expressed in robot i 's estimated body frame. `env.observation_layout()` returns the slices of Table 8.1.

Table 8.1: AJLATT observation of robot i ($n = 6 + 4(n_R - 1)$).

Key of <code>observation_layout()</code>	Columns	Content
<code>target_relative_position</code>	0:2	believed target position in the robot frame
<code>target_relative_heading</code>	2	believed target heading minus own heading
<code>target_velocity</code>	3:5	commanded target velocity (v, ω) of the last step (zero after reset)
<code>target_cov_trace</code>	5	$\text{tr } P_{i,0}^T$
<code>neighbours</code>	6:n	for every other robot, in index order: relative x , relative y , relative heading (estimates), trace of its covariance
<code>closest_obstacle</code>	n:n+2	range and bearing of the closest obstacle cell within the field of view and sensor range; $(r_{\max}, \pi)$ if none; zeros while the robot is within <code>obstacle_sensing_margin</code> of the map border
<code>self_pose</code>	n+2:n+5	own pose estimate (x, y, θ) in the map frame
<code>self_cov_trace</code>	n+5	$\text{tr } P_i$

8.4 Reward and episode end

`step()` returns the per-robot rewards as an array of shape `(num_robots,)`:

$$r_i = -w_T \text{tr } P_{i,0}^T - w_R \text{tr } P_i - \beta_B \mathbf{1}_{\text{border}}(i) - \beta_O \mathbf{1}_{\text{obstacle}}(i) - \beta_M \mathbf{1}_{\text{mutual}}(i), \quad (8.4)$$

with $w_T = \text{target_cov_weight}$ (10), $w_R = \text{robot_cov_weight}$ (5) and the penalties

- $\beta_B = \text{boundary_penalty}$ (20) when the estimated position lies within 0.1 m of the map border or outside the map;
- otherwise $\beta_O = \text{obstacle_penalty}$ (1.3) when an obstacle cell is closer to the estimated pose than `obstacle_collision_distance` (0.2 m, searched in all directions up to `sensor_r_max`); this also sets the robot's collision flag;
- $\beta_M = \text{mutual_collision_penalty}$ (0.5) when another robot's estimated position is closer than `mutual_collision_distance` (0.4 m).

The team reward is $\sum_i r_i$ (`info["team_reward"]`).

Episode end. `terminated` is a boolean array of shape `(num_robots,)` holding the obstacle collision flags (all `False` with `terminate_on_collision=False`). If an estimator diverges (non-finite covariance) all flags are set, a `RuntimeWarning` is issued and `info["numerical_error"]` is `True`. `truncated` is a `bool`, set after `max_episode_steps` steps. The environment keeps simulating after a collision flag; whether a flag ends the episode is left to the caller (`record_episode` and `TeamRewardWrapper` use `any(terminated)`).

8.5 Configuration

`AJLATTEnv(config=None, render_mode=None, **overrides)` takes an `AJLATTConfig` and/or individual fields as keyword arguments, which are applied on top of `config`. Unknown field names raise `TypeError`.

```
from env_lib import AJLATTConfig, AJLATTEnv

env = AJLATTEnv(map_name="obstacles05", num_robots=3)    # fields as keywords
config = AJLATTConfig(map_name="obstacles04", sensor_r_max=5.0, fov=120.0)
env = AJLATTEnv(config, render_mode="rgb_array")
env = AJLATTEnv(config.replace(ci_solver="slsqp"))      # validated copy
```

`AJLATTConfig` validates its values on construction (for example $0 \leq \text{sensor_r_min} < \text{sensor_r_max}$, $0 < \text{fov} \leq 360$, non-negative noise levels, enough initial poses and covariances for the team) and offers `replace(**changes)`, `to_dict()` and `from_dict(data)`, the property `observation_dim`, and `from_kwargs(strict=True, **kwargs)`, which also accepts the legacy names of Table 8.3 (as do `AJLATTEnv`, `env_lib.ajlatt_env.make` and the callable module `env_lib.ajlatt_env`). All fields are listed in Table 8.2.

Table 8.2: Fields of `AJLATTConfig` (keyword arguments of `AJLATTEnv`).

Field	Default	Description
<i>Scenario</i>		
<code>map_name</code>	"obstacles04"	Bundled map name or path to a user map (Section 8.9).
<code>num_robots</code>	4	Team size n_R (≥ 1).
<code>num_targets</code>	1	Number of targets; target 0 moves, the others are static landmarks.
<code>max_episode_steps</code>	120	Episode length; reaching it sets <code>truncated</code> .
<code>sampling_period</code>	0.5	Duration Δt of one step in seconds.
<code>margin2wall</code>	1.0	Safety margin of the map queries <code>in_bound</code> and <code>is_collision</code> .

Field	Default	Description
robot_init_pose	six poses	(x, y, θ) per robot; the first four defaults are (7.5, 12.5, 0), (7.5, 10.5, 0), (10, 12.5, 0), (10, 10.5, 0).
target_init_pose	13 poses	(x, y, θ) per target; target 0 starts at (10.5, 11.5, 0).
robot_init_cov	(0.1,) * 6	Initial position variance per robot.
target_init_cov	(1.0,) + (10.0,) * 10	Initial position variance per target.
<i>Sensing and communication</i>		
sensor_r_min	0.0	Minimum sensing range (m).
sensor_r_max	3.0	Maximum sensing range (m); also the range of the obstacle reading.
fov	180.0	Field of view in degrees, centred on the heading.
sigma_p	0.0	Range noise as a fraction of the range (proportional mode).
sigma_r	0.0	Absolute range noise (m) when <code>range_noise_proportional=False</code> .
sigma_th	0.0	Bearing noise (rad).
range_noise_proportional	True	Use <code>sigma_p</code> instead of <code>sigma_r</code> .
use_update	True	Run measurement updates (False : dead reckoning only).
commu_r_max	6.0	Communication range (m).
ci_solver	"newton"	Covariance-intersection solver, "newton" or "slsqp".
<i>Motion noise</i>		
process_noise_fixed	True	Fixed velocity noise; False scales it with the speed.
sigmapR	0.1	Robot noise factor in the speed-dependent mode.
sigma_vR	0.1	Robot linear-velocity noise (m/s).
sigma_wR	$0.5\pi/180$	Robot angular-velocity noise (rad/s, 0.5 degrees/s).
sigmapT	0.1	Target noise factor in the speed-dependent mode.
sigma_vT	0.2	Target linear-velocity noise used by the estimators (m/s).
sigma_wT	$0.5\pi/180$	Target angular-velocity noise used by the estimators (rad/s).
<i>Target motion</i>		
target_policy	"auto"	Target policy (Table 8.4).
target_linear_velocity	0.3	Speed of the "sine" policy; also scales the speed-dependent target noise.
target_omega_bound	$\pi/4$	Turn-rate bound of the "random" policy.
k_theta	0.5	Heading gain of the "sine" and "waypoints" policies.
k_p	1.0	Kept for compatibility; currently unused.
<i>Actions</i>		
max_linear_velocity	2.0	Upper bound of v (m/s); the lower bound is 0.
max_angular_velocity	$\pi/4$	Bound of $ \omega $ (rad/s).
clip_actions	False	Clip actions to the action space.

Field	Default	Description
<i>Rewards and termination</i>		
target_cov_weight	10.0	Weight w_T of the target covariance trace.
robot_cov_weight	5.0	Weight w_R of the self-localisation covariance trace.
boundary_penalty	20.0	Penalty β_B near or beyond the map border (0.1 m margin).
obstacle_penalty	1.3	Penalty β_O for an obstacle closer than <code>obstacle_collision_distance</code> .
obstacle_collision_distance	0.2	Obstacle collision distance (m).
mutual_collision_penalty	0.5	Penalty β_M for robots closer than <code>mutual_collision_distance</code> .
mutual_collision_distance	0.4	Robot-robot distance threshold (m).
terminate_on_collision	True	Report obstacle collisions in <code>terminated</code> .
obstacle_sensing_margin	1.0	Robots closer than this to the map border report a zero obstacle reading (original behaviour).
<i>Miscellaneous</i>		
seed	None	Seed of the first <code>reset()</code> when none is passed.

The default poses and covariances cover up to six robots and eleven targets; larger teams need explicit `robot_init_pose` and `robot_init_cov` (and likewise for targets).

Table 8.3: Legacy parameter names; each emits a `DeprecationWarning`.

Legacy name	Replacement
num_Robot, num_Target	num_robots, num_targets
T_steps	max_episode_steps
maxmum_run	caps max_episode_steps at maxmum_run + 1
SIGPERCENT	range_noise_proportional
useupdate	use_update
fix_seed	seed (fix_seed=True sets seed=1)
render=True	render_mode="human"
obsolete keys (figID, ssh_debug, episode_num, ...)	ignored with a warning

Command-line integration. `AJLATTConfig.add_arguments(parser, prefix="")` registers every scalar field as an option `--<prefix><name>` on an `argparse` parser, and `AJLATTConfig.from_namespace(namespace, prefix="")` builds a configuration from the parsed arguments. Training scripts keep their own options next to the environment's:

```
import argparse
from env_lib import AJLATTConfig, AJLATTEnv

parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4) # training options
AJLATTConfig.add_arguments(parser, prefix="env.") # --env.map_name, ...
args = parser.parse_args(["--env.map_name", "obstacles05",
                          "--env.num_robots", "3"])
env = AJLATTEnv(AJLATTConfig.from_namespace(args, prefix="env."))
```

8.6 Target motion

The target policy maps the true target pose to a unicycle command. Policies are per-environment objects that are reset with the environment's generator at every `reset()`.

Table 8.4: Target policies (`target_policy`).

Policy	Behaviour
"auto"	"sine" on the map empty, "waypoints" on obstacles04 and obstacles05, "circle" otherwise
"sine"	follows the slope field $dy/dx = 5 \cos x$ at speed <code>target_linear_velocity</code> with heading gain <code>k_theta</code> until $x \geq 32$ m, then circles with $v = 0.25$ m/s on a 4 m radius
"waypoints"	proportional heading control (gain <code>k_theta</code>) towards piecewise waypoints; built-in routes for obstacles04 (0.1 m/s) and obstacles05 (0.25 m/s); other routes with <code>env.lib.ajlatt.env.controllers.WaypointPolicy</code>
"circle"	constant $(v, \omega) = (0.25 \text{ m/s}, 0.15 \text{ rad/s})$
"random"	$v = 0.25$ m/s and a turn rate drawn uniformly from $[-\text{target_omega_bound}, \text{target_omega_bound}]$ at every step
"static"	the target does not move

8.7 Info dictionary and episode statistics

Table 8.5: AJLATT info dictionary ($n_R = \text{num_robots}$).

Key	Content
<code>agent_rewards</code>	float64 array ($n_R, $), equal to the returned rewards (zeros at reset)
<code>team_reward</code>	sum of the per-robot rewards
<code>collisions</code>	boolean array ($n_R, $): obstacle collisions in this step
<code>episode_collisions</code>	integer array ($n_R, $): number of border, obstacle and robot-robot penalty events in the episode
<code>target_observed</code>	boolean array ($n_R, $): robot measured target 0 in this step
<code>communication</code>	boolean matrix (n_R, n_R): communication links, including self-loops
<code>robot_cov_trace</code>	array ($n_R, $): $\text{tr } P_i$
<code>target_cov_trace</code>	array ($n_R, $): $\text{tr } P_{i,0}^T$
<code>step</code>	number of steps taken
<code>numerical_error</code>	True if an estimator diverged (<code>step()</code> only)

`env.episode_statistics()` returns per-step traces of the current episode as arrays of shape $(\text{steps} + 1, \text{num_robots})$, starting with the state after reset: `robot_cov_trace`, `target_cov_trace`, `robot_error` and `target_error` (Euclidean position errors of the robots' own and target beliefs) and `reward`. The traces are cleared at every reset.

8.8 Team reward wrapper and legacy factory

`TeamRewardWrapper(env)` returns the team reward $\sum_i r_i$ as a float and `any(terminated)` as a bool, for algorithms that expect scalar values. The per-robot values remain available in the info dictionary under `"agent_rewards"` and `"agent_terminated"`.

```

from env_lib import AJLATTEnv
from env_lib.ajlatt_env import TeamRewardWrapper

env = TeamRewardWrapper(AJLATTEnv(map_name="obstacles04"))
obs, info = env.reset(seed=1)
obs, reward, terminated, truncated, info = env.step(env.action_space.sample())

```

The original factory remains available for existing code: `env_lib.ajlatt_env.make(**kwargs)`, or the callable module `env_lib.ajlatt_env(num_Robot=4, T_steps=100)`. It translates the legacy names of Table 8.3, ignores unknown keys with a warning and maps `render=True` to `render_mode="human"`. New code should create `AJLATTEnv` directly.

8.9 Maps

The subpackage `env_lib.ajlatt_env.maps` contains the occupancy maps and the tools to create and inspect them.

8.9.1 Bundled maps

`available_maps()` lists the bundled maps (Table 8.6); `ajlatt-plot-map --list` prints the same list. All maps start at the origin (`mapmin = (0,0)`). Figure 8.1 shows `obstacles05`.

Table 8.6: Bundled maps. Size in cells ($n_x \times n_y$) and metres.

Map	Cells	Cell size	Extent	Notes
obstacles	51×51	0.5 m	25.5 m	
obstacles02, obstacles07, obstacles07-02, obstacles08	181×181	0.2 m	36.2 m	
obstacles03	102×102	0.2 m	20.4 m	
obstacles04	181×181	0.2 m	36.2 m	default; waypoint route
obstacles05	181×181	0.4 m	72.4 m	waypoint route
obstacles06, obstacles06-02, obstacles06-03, obstacles08-02	181×181	0.1 m	18.1 m	
example_complete_map, sample_map_config	181×181	0.4 m	72.4 m	map builder examples
empty	82×62	0.5 m	41×31 m	no obstacles; sine target policy
empty05	181×181	0.4 m	72.4 m	no obstacles
emptySmall	135×135	0.2 m	27 m	no obstacles
emptySmall05	65×65	0.4 m	26 m	no obstacles
dynamic_map, dynamic_map_2	181×181	0.4 m	72.4 m	obstacles resampled at every reset

8.9.2 Map files

A static map consists of two files with the same name:

- `<name>.yaml`, the header, with the keys `mapdim` ($[n_x, n_y]$, cells along x and y), `mapres` (cell size $[r_x, r_y]$ in metres), `mapmin` and `mapmax` (corners of the map in metres) and optionally

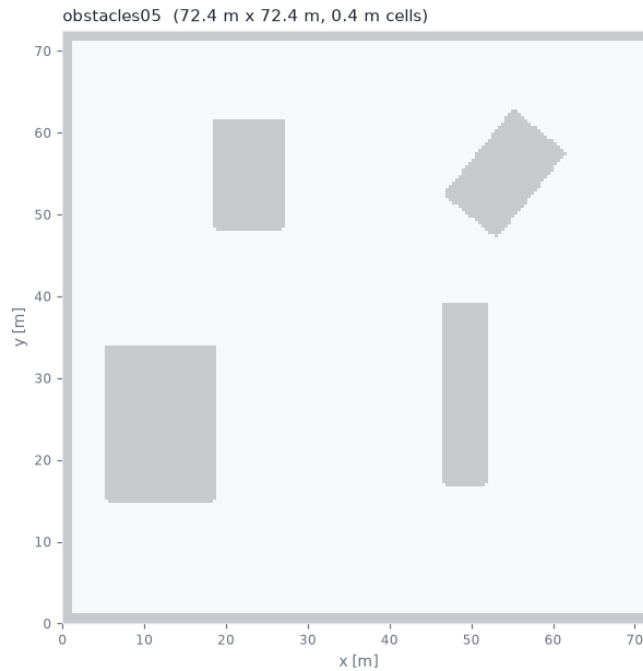


Figure 8.1: The bundled map `obstacles05` (72.4×72.4 m, 0.4 m cells) plotted with `ajlatt-plot-map obstacles05`; occupied cells are grey, including the one-cell wall around the map.

`origin`. Further keys written by older tools or the map builder (`datatype`, `mappath`, `origincells`, `storage`, the embedded specification) are ignored by the loader.

- `<name>.cfg`, the occupancy grid as whitespace-separated numbers, one text row per grid row: n_y rows of n_x values, where row 0 is the bottom row ($y = \text{mapmin}$) and 1 marks an occupied cell.

Cell (c_x, c_y) has its centre at $(c + 0.5)r + \text{mapmin}$. A grid with fewer cells than its header declares is padded with occupied cells and a warning. An empty (zero-byte) `.cfg` file denotes an obstacle-free map in which only the boundary blocks rays; a map whose name contains `empty` may omit the file altogether, while a missing grid for any other static map raises `FileNotFoundError`. A dynamic map has only a header, which defines `submaporigin` (four (row, column) cell anchors, flattened to eight numbers) and `lib_path` (a directory of `.npz` obstacle masks relative to the header).

To use your own map, pass its path to `map_name`, with or without the `.yaml` suffix, for example `AJLATTEnv(map_name="maps/my_map")`; the grid file must sit next to the header. Check that the default initial poses (Table 8.2) are free cells of the map, or pass `robot_init_pose` and `target_init_pose`.

8.9.3 Map API

- `load_grid_map(name_or_path, margin2wall=0.5)` returns a `GridMap`, or a `DynamicMap` for a header with `submaporigin` and no grid. `load_map(name)` returns (`occupancy`, `mapmin`, `mapmax`) for plotting, and `resolve_map_path(name)` the file path without suffix.
- `GridMap` holds the grid as `map` (shape (n_y, n_x) , `None` for empty maps) together with `mapdim`, `mapres`, `mapmin` and `mapmax`; its queries are listed in Table 8.7. `GridMap.from_array(occupancy, mapmin=(0, 0), mapres=0.4)` builds a map from an array of shape (n_y, n_x) .

- `DynamicMap.generate_map(chosen_idx=None, rot_angs=None, rng=None)` places four distinct obstacles from its library at the four anchors, each rotated by a random multiple of 18 degrees, and closes the border. The environment calls it with its own generator at every reset.

Table 8.7: Queries of `GridMap` (positions and poses in metres and radians).

Method	Result
<code>is_blocked(start, end)</code>	True if the straight line crosses an occupied cell (exact Bresenham ray casting)
<code>is_blocked_batch(starts, ends)</code>	the same for arrays of shape $(k, 2)$ or $(k, 3)$
<code>get_closest_obstacle(pose, ang_res=0.05, fov=pi, r_max=3.0)</code>	range and bearing of the closest occupied cell in the field of view, or None
<code>is_collision(pos, margin=None)</code>	True if <code>pos</code> is out of bounds or within <code>margin</code> (default <code>margin2wall</code>) of an occupied cell
<code>in_bound(pos)</code>	True if <code>pos</code> is at least <code>margin2wall</code> inside the map
<code>se2_to_cell(pos), cell_to_se2(cell)</code>	conversion between positions and cell indices

8.9.4 Building maps

The map builder (`env_lib.ajlatt_env.maps.builder`, command `ajlatt-build-map`) rasterises a YAML specification that describes the map in metres:

```
map_info:
  name: my_map
  width: 36.0           # metres along x
  height: 36.0          # metres along y
  resolution: 0.2        # cell size in metres
  boundary:
    enabled: true
    thickness: 0.2
    style: rectangle     # rectangle | circle | polygon
    rectangle: {margin: 0.0}
  obstacles:
    - {type: rectangle, center: [10, 10], width: 4, height: 2, angle: 0.3}
    - {type: circle, center: [25, 25], radius: 3, filled: false}
    - {type: polygon, points: [[5, 20], [9, 20], [7, 26]]}
    - {type: line, start: [0, 18], end: [12, 18], width: 0.4}
    - {type: random, num_obstacles: 5, min_size: 0.5, max_size: 1.5, seed: 3}
```

The grid has $n_x = \text{round}(\text{width}/\text{resolution})$ columns and $n_y = \text{round}(\text{height}/\text{resolution})$ rows; a cell is occupied when its centre lies inside a shape. The boundary (enabled by default) is a rectangular wall with an optional `margin`, the outside of a circle (`circle: {center, radius}`) or the outside of a polygon (`polygon: {points}`), `thickness` metres thick. The obstacle types are listed in Table 8.8; closed shapes accept `filled: false` for a one-cell outline.

The builder writes `<name>.yaml` (the grid header followed by the specification, so a map can always be rebuilt from its own header) and `<name>.cfg`, in the orientation the environment reads. The name is `map_info.name` unless `-n` is given:

```
ajlatt-build-map --create-sample my_map      # writes my_map.spec.yaml
ajlatt-build-map my_map.spec.yaml --png      # my_map.yaml, .cfg, .png
ajlatt-build-map spec.yaml -o maps/ -n arena # other directory and name
```

Table 8.8: Obstacle types of the map builder (lengths in metres, angles in radians).

Type	Keys
rectangle	center, width, height, angle (default 0), filled
circle	center, radius, filled
triangle	vertices (three points), filled
hexagon	center, radius, angle, filled
polygon	points (at least three), filled
line	start, end, width (default: one cell)
maze	cell_size (5.0), wall_thickness (0.5) and gaps, a list of {type: horizontal vertical, position: <cell index>} walls to leave open
random	num_obstacles (10), min_size (1.0), max_size (3.0), seed: axis-aligned squares

`--create-sample` sets `map_info.name` to the requested file name, so the second command writes `my_map.yaml`, `my_map.cfg` and `my_map.png`. The same functionality is available from Python: `build_map(spec, output_dir=None, *, name=None, png=False)` accepts a path or a dictionary and returns the paths of the header and grid, `rasterize(spec)` returns the occupancy array and the header, and `save_map` writes them.

```
from env_lib import AJLATTEnv
from env_lib.ajlatt_env.maps.builder import build_map

spec = {
    "map_info": {"name": "corridor", "width": 20.0, "height": 10.0,
                 "resolution": 0.2},
    "obstacles": [
        {"type": "line", "start": [10, 0], "end": [10, 6], "width": 0.4},
        {"type": "circle", "center": [15, 5], "radius": 1.5},
    ],
}
header, grid = build_map(spec, "maps", png=True)  # maps/corridor.yaml, ...
env = AJLATTEnv(map_name="maps/corridor",
                 robot_init_pose=[(2, 2, 0), (2, 4, 0), (4, 2, 0), (4, 4, 0)],
                 target_init_pose=[(5, 5, 0)], target_policy="static")
```

8.9.5 Plotting maps

`ajlatt-plot-map` draws a bundled or user map; without `--save` it writes `<map>.png` in the working directory. It uses the light theme unless `--theme dark` is given. Dynamic maps are drawn with a fixed obstacle sample.

```
ajlatt-plot-map --list                    # bundled map names
ajlatt-plot-map obstacles04 --save o4.png
ajlatt-plot-map maps/corridor --theme dark
```

From Python, `plot_map(map_or_name, ax=None, *, theme=None, title=None, save=None, dpi=150)` in `env_lib.ajlatt_env.maps.plotting` draws a map into existing axes (or a new off-screen figure) and returns the axes.

8.10 Rendering

The dashboard (1100 × 620 pixels, Figure 8.2) shows the map on the left with the robots' true poses as oriented arrowheads, their pose beliefs as covariance ellipses, their sensing sectors, fading trajectory trails, communication links, the active target measurements, the true target (star) and every robot's belief about the target (crosses). The two panels on the right plot the trace of each robot's target covariance and of its self-localisation covariance over the episode. The header shows the map, the step, the simulated time, the team reward and how many robots currently see the target. Human mode requires an interactive matplotlib backend; TkAgg needs the tk bindings of the Python installation (`python3-tk` on Debian and Ubuntu, `tk` in conda).

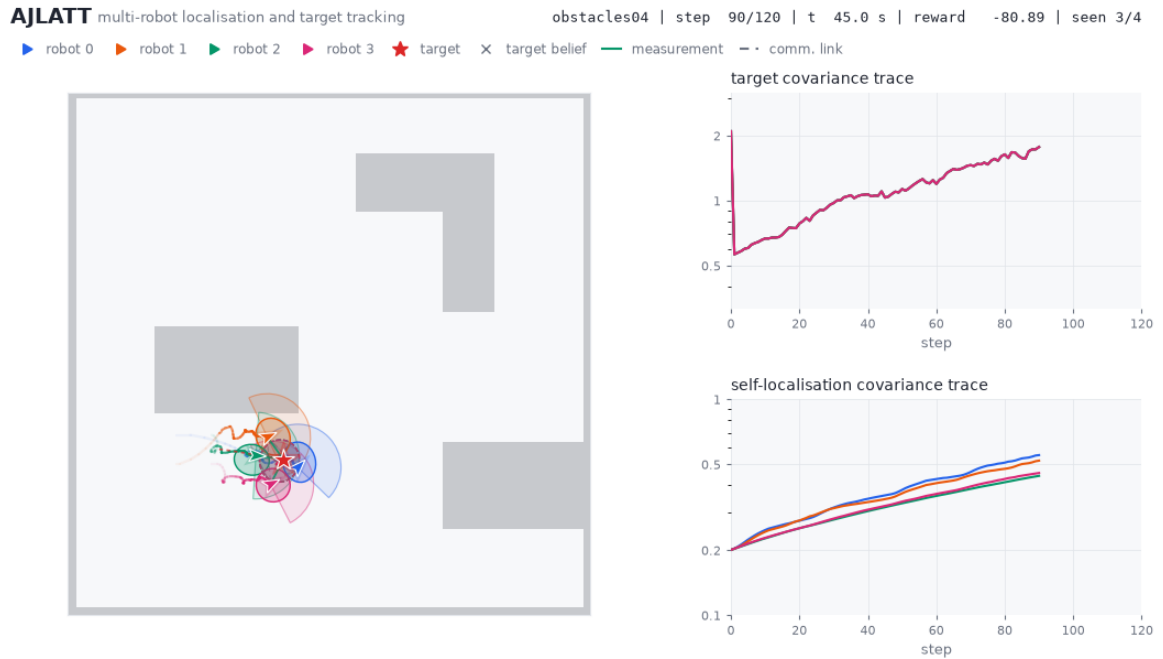


Figure 8.2: AJLATT dashboard on `obstacles04` (four robots running the encircling heuristic of the example script, step 90 of 120). The robots surround the target; the right panels show the trace of the target covariance and of the self-localisation covariances of the four robots.

8.11 Registered id

AJLATT-v0 creates `AJLATTEnv` with the default configuration (four robots on `obstacles04`). Gymnasium's passive environment checker is disabled for this id because rewards and `terminated` are per-robot arrays. Keyword arguments of `env_lib.make` are configuration fields, for example `env_lib.make("AJLATT-v0", map_name="obstacles05")`.

8.12 Usage

The following controller sends robot i to the i -th of n_R evenly spaced slots on a circle of radius 1.8m around its belief of the target, using only the observation:

```
import numpy as np
from env_lib import AJLATTEnv
```

```

def encircle(obs, layout, radius=1.8, gain=1.2):
    """Send robot i to the i-th slot of a circle around its target belief."""
    n = len(obs)
    pose = obs[:, layout["self_pose"]].astype(np.float64)  # x, y, theta
    rel = obs[:, layout["target_relative_position"]].astype(np.float64)
    c, s = np.cos(pose[:, 2]), np.sin(pose[:, 2])
    target = pose[:, :2] + np.column_stack([c * rel[:, 0] - s * rel[:, 1],
                                           s * rel[:, 0] + c * rel[:, 1]])

    angle = 2 * np.pi * np.arange(n) / n
    slot = target + radius * np.column_stack([np.cos(angle), np.sin(angle)])
    delta = slot - pose[:, :2]
    heading = np.arctan2(delta[:, 1], delta[:, 0])
    error = np.angle(np.exp(1j * (heading - pose[:, 2])))  # wrapped
    speed = np.minimum(2.0, 0.5 * np.linalg.norm(delta, axis=1))
    v = speed * np.maximum(0.0, np.cos(error))
    w = np.clip(gain * error, -np.pi / 4, np.pi / 4)
    return np.column_stack([v, w])

env = AJLATTEnv(map_name="obstacles04", num_robots=4)
layout = env.observation_layout()
obs, info = env.reset(seed=0)  # obs: (4, 24) float32
episode_return = np.zeros(env.num_robots)
while True:
    obs, reward, terminated, truncated, info = env.step(encircle(obs, layout))
    episode_return += reward  # per-robot rewards
    if truncated:  # collision flags ignored
        break
stats = env.episode_statistics()  # arrays of shape (121, 4)
print(episode_return.sum(), info["episode_collisions"])
print(stats["target_cov_trace"][-1])
env.close()

```

The example script compares a similar heuristic with random actions and can record an episode:

```

python examples/ajlatt_demo.py --episodes 3
python examples/ajlatt_demo.py --save renders/ajlatt.gif
python examples/ajlatt_demo.py --map obstacles05 --render-mode human

```

With the default configuration the heuristic of the script obtains a team return of about -7250 per episode without collision events and keeps the trace of the target covariance near 2, whereas random actions obtain returns of $-25\,000$ or lower with dozens of collision events and target covariance traces of 7 to 12 (the random actions are not seeded, so these values vary from run to run).

8.13 Reproducibility and performance

All randomness (initial beliefs, motion and measurement noise, the random target policy and dynamic maps) is drawn from `self.np_random`. If `reset()` is called without a seed for the first time and `config.seed` is set, that seed is used. With `ci_solver="slsqp"` trajectories match the implementation before version 1.0 to about 10^{-6} ; the default Newton solver finds weights that are at least as good and therefore produces slightly different, not worse, estimates.

Measured on the development machine without rendering, a step takes 5–9 ms on `obstacles04`

(90 ms before 1.0) and 6–7 ms on `obstacles05` (67 ms before). The gains come from exact batched Bresenham ray casting (closest-obstacle queries are 7 to 19 times faster) and from the Newton covariance-intersection solver. Rendering a frame takes about 25 ms (about 110 ms before). The cost of a step grows with the number of robots, because every robot fuses the measurements and priors of its neighbours.

8.14 Changes in 1.0

AJLATT was rewritten as a Gymnasium environment. See Appendix B for migration advice.

- The environment class `AJLATTEnv` with its configuration dataclass `AJLATTConfig` replaces the former factory, which parsed the command line internally and therefore failed in scripts with their own command-line arguments. The legacy factory still works and translates the old names with a `DeprecationWarning`. Training options that used to live in the parameter module (PPO, LLM and debug settings) were removed; use `AJLATTConfig.add_arguments`.
- `reset()` returns `(observation, info)`. Rendering is off by default; pass `render_mode`.
- The spaces are joint multi-agent spaces with one row per robot, and rewards and `terminated` are per-robot arrays; the per-robot spaces are available as `single_action_space` and `single_observation_space`.
- The target-velocity entries of the observation contain the commanded target velocity instead of a configured constant.
- Fixed a memory leak: covariance traces accumulated across episodes (per-episode traces are now returned by `episode_statistics()`). Also fixed: target policies were shared between environments and never reset, unknown maps triggered an interactive prompt, and dynamic maps did not work; they now resample at every reset and no longer need scikit-image. Three bundled maps that were one row short of their header were padded with a wall row.
- About ten times faster (Section 8.13) and a new renderer.
- Map tools were consolidated into `env_lib.ajlatt_env.maps`; the builder now writes grids in the orientation the environment reads (the old builder produced transposed maps) and embeds its specification in the header.
- The id `AJLATT-v1`, an unusable alias of `AJLATT-v0`, was removed.

Part II

Toolkit

Chapter 9

Toolkit Overview and Parameter Management

The `toolkit` package collects research utilities that are independent of the environments. This chapter describes its structure and then documents `toolkit.parakit`, the parameter-management module. The plotting and neural-network subpackages have their own chapters.

9.1 Structure of the toolkit

Table 9.1: Subpackages of `toolkit`.

Subpackage	Contents	Dependencies	Chapter
<code>toolkit.plotkit</code>	Learning curves, bars, histograms, heatmaps, style presets, palettes, figure export, gallery CLI	matplotlib (core)	10
<code>toolkit.neural_toolkit</code>	Policy, value and Q-networks, encoders, decoders, factories, <code>NetworkUtils</code> , tabular tools	PyTorch (<code>torch</code> extra)	11
<code>toolkit.parakit</code>	Save, load, validate and apply <code>argparse</code> defaults; Tk editor window	standard library; Tk for the window	9

The subpackages are imported lazily: `import toolkit` only defines `toolkit.__version__`, and the first access to `toolkit.plotkit`, `toolkit.neural_toolkit` or `toolkit.parakit` imports that subpackage. A script that only plots therefore never imports PyTorch, and `parakit`'s headless functions work on systems without Tk. The usual import style names the subpackage explicitly:

```
from toolkit.plotkit import plot_learning_curves, save_figure
from toolkit.parakit import apply_parameters, load_parameters
from toolkit.neural_toolkit import MLPPolicyNetwork, NetworkUtils # needs torch
```

All three subpackages follow the conventions of the environments: they never print, never configure logging (diagnostics go to `logging` loggers named after the module), never touch the global random state, and report problems with exceptions or `warnings` whose messages say how to fix them.

9.2 parakit: managing argparse parameters

Research scripts are usually configured through an `argparse.ArgumentParser`. `parakit` treats the parser as the single source of truth for the parameters, their types and their allowed values, and adds four operations on top of it: reading the current defaults, saving them to a JSON file, loading a file, and installing validated values as the new defaults. Because the values become *defaults*, flags given on the command line still override them. An optional Tk window (Section 9.2.6) lets users edit the values interactively.

9.2.1 Quick start

The typical workflow saves the configuration of a run next to its results and later restores it:

```
import argparse
from toolkit.parakit import (
    apply_parameters, get_parameters, load_parameters, save_parameters,
)

parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4, help="learning rate")
parser.add_argument("--batch_size", type=int, default=64)
parser.add_argument("--optimizer", choices=["adam", "sgd"], default="adam")
parser.add_argument("--hidden", type=int, nargs="+", default=[256, 256])
parser.add_argument("--layer_norm", action="store_true")
parser.add_argument("--checkpoint", default=None)

# Save the current defaults into a directory (timestamped file name).
path = save_parameters(get_parameters(parser), "runs/params")

# Later: load the newest file of the directory and install it as defaults.
apply_parameters(parser, load_parameters("runs/params"))

# Values given as text are converted according to the parser.
apply_parameters(parser, {"lr": "1e-3", "hidden": "64 64", "layer_norm": "yes"})
args = parser.parse_args([]) # lr=0.001, hidden=[64, 64], layer_norm=True
args = parser.parse_args(["--lr", "0.01"]) # command-line flags still win
```

9.2.2 Headless API

Table 9.2 lists the functions; none of them needs Tk.

Tunable options. `tunable_actions` skips help, version and sub-parser actions and options whose destination or default is `argparse.SUPPRESS`. When several options share a destination, such as a `--flag/--no-flag` pair, the first one is used. Positional arguments are included and keyed by their name.

Errors. `apply_parameters` either applies all values or none. With `strict=True` (the default) a key that is not a parser destination raises a `ValueError` that lists the unknown keys and the valid names; with `strict=False` unknown keys are ignored. Otherwise every value is converted, and all conversion and validation failures are reported together in one `ValueError`, one line per parameter. The parser is left unchanged whenever an exception is raised.

Table 9.2: Headless functions of `toolkit.parakit`.

Function	Description
<code>get_parameters(parser)</code>	Dictionary <code>{dest: default}</code> of all tunable options (lists are copied).
<code>save_parameters(values, path_or_dir)</code>	Write <code>values</code> as JSON and return the <code>Path</code> of the file. A path with a suffix is used as the file name; a directory (existing, or a path without suffix) receives a new timestamped file. Parent directories are created and the write is atomic.
<code>load_parameters(path)</code>	Read a JSON object from a file, or from the newest <code>parameters_*.json</code> file of a directory. Raises <code>FileNotFoundError</code> or <code>ValueError</code> .
<code>apply_parameters(parser, values, strict=True, validation_callbacks=None)</code>	Convert and validate every value with <code>parse_value</code> , then call <code>parser.set_defaults</code> . Returns the parser.
<code>parse_value(action, text, validator=None)</code>	Convert one value (text or an already typed JSON value) for an <code>argparse</code> action; see Section 9.2.4.
<code>parse_bool(value)</code>	Convert <code>true/false</code> , <code>yes/no</code> , <code>on/off</code> , <code>1/0</code> (any case) or a boolean to <code>bool</code> .
<code>format_value(action, value)</code>	Text that <code>parse_value</code> converts back (lists, also nested, as JSON; <code>None</code> as <code>"None"</code>).
<code>describe_type(action)</code>	Short type description such as <code>"float"</code> , <code>"list[int]"</code> , <code>"list[list[int]]"</code> or <code>"str or None"</code> .
<code>tunable_actions(parser)</code>	Dictionary <code>{dest: action}</code> of the options <code>parakit</code> manages.

9.2.3 File format

A parameter file is a flat JSON object that maps each destination name (`action.dest`) to its value, written in UTF-8 with an indentation of four spaces:

```
{
    "lr": 0.001,
    "batch_size": 64,
    "optimizer": "adam",
    "hidden": [
        64,
        64
    ],
    "layer_norm": true,
    "checkpoint": null
}
```

- *File names.* Saving into a directory creates `parameters_YYYYmmdd_HHMMSS.json`; if that name exists (two saves in the same second), `_1`, `_2`, ... is appended. Loading from a directory reads the most recent file, ranked by the timestamp and counter in its name (so `_10` is newer than `_9`); other `parameters_*.json` names rank below the timestamped ones.
- *Values.* JSON numbers, strings, booleans, `null` and lists are stored as such. NumPy arrays and scalars are converted with `tolist()`, sets are stored as sorted lists and any other object as its `str()`.
- *Loading.* Values read from a file pass through the same conversion as text, so a file may contain `"64"` or `64` for an integer option, and `"yes"` or `true` for a flag. A file may contain a subset of the parameters; the other defaults are kept.

- *Atomic writes.* Files are written to a temporary file in the target directory and renamed, so an interrupted save never leaves a truncated file behind. The file receives the usual permissions of new files (0o666 minus the umask).

9.2.4 Parsing rules

`parse_value(action, text)` derives the conversion from the parser's own declarations, so the same rules apply to values typed in the editor window, loaded from JSON or passed to `apply_parameters`. Table 9.3 summarises them.

Table 9.3: Conversion rules of `parse_value`.

Declaration	Accepted input and result
<code>type=int</code> , <code>type=float</code>	Text is converted after removing surrounding blanks. Already typed values from JSON are checked: an <code>int</code> option accepts 3 and 3.0 but not 3.5; a <code>float</code> option accepts integers.
Other <code>type=</code> callables	Text is passed to the callable. A value that is not text (for example from JSON) is passed as it is and, if that fails, as text (booleans as <code>"true"/"false"</code>), so converters written for command-line strings, such as <code>lambda s: int(s, 0)</code> , survive a save and load.
No <code>type</code>	The type is inferred from the default (<code>bool</code> , <code>int</code> , <code>float</code> , otherwise <code>str</code>), or from the <code>choices</code> when all have the same type.
Booleans: <code>store_true</code> , <code>store_false</code> , <code>BooleanOptionalAction</code> , <code>type=bool</code> or a boolean default	<code>true/false</code> , <code>yes/no</code> , <code>on/off</code> , 1/0 in any case, or a JSON boolean. The string <code>"False"</code> gives <code>False</code> .
Lists: <code>nargs="+"</code> , <code>"*"</code> , an integer, <code>append</code> and <code>extend</code> actions	A JSON list (<code>[1.0, 2.0]</code>), a Python literal list or tuple, or tokens separated by commas (if the text contains a comma) or by whitespace. Each element is converted; <code>"+"</code> requires at least one element and an integer <code>nargs</code> exactly that many, while <code>append</code> and <code>extend</code> accept any number. <code>append</code> with <code>nargs</code> holds a list of lists such as <code>[[1, 2], [3, 4]]</code> ; for an integer <code>nargs</code> a flat list is split into groups of that size.
Default <code>None</code> or <code>nargs="?"</code>	<code>None</code> , <code>null</code> (any case) and the empty string give <code>None</code> . For other options <code>None</code> is rejected, except that a string option accepts the text <code>"None"</code> literally.
<code>choices</code>	Enforced on the converted value (on every element of a list).
<code>count</code> action	Converted as an integer.

Declare types explicitly. Type inference from the default is a convenience of `parakit`, not of `argparse`. An option declared as `add_argument("--gamma", default=0.99)` is converted to `float` by `parakit`, but `parser.parse_args(["--gamma", "0.9"])` returns the *string* `"0.9"`. Give every option a `type=` so that both paths agree.

Validation callbacks. A validator receives the *converted* value and signals the result in one of the following ways: return `True/False`; return a tuple (`is_valid`, `message`) or the legacy form (`is_valid`, `converted_value`, `message`); return `None` (valid); or raise `ValueError` or `TypeError`, whose text becomes the error message. Validators are not called when the converted value is `None`, and a value returned by a validator is ignored: the result always follows the parser declaration.

```
def positive_even(value):
    return value > 0 and value % 2 == 0, "batch size must be a positive even number"

validators = {"batch_size": positive_even,
              "lr": lambda v: (0 < v <= 1, "lr must lie in (0, 1]")}
apply_parameters(parser, {"batch_size": "128", "lr": 0.01},
                  validation_callbacks=validators)

try:
    apply_parameters(parser, {"batch_size": 7, "optimizer": "rmsprop"},
                    validation_callbacks=validators)
except ValueError as exc:
    print(exc)
# Invalid parameter values:
# batch_size: batch size must be a positive even number
# optimizer: 'rmsprop' is not a valid choice; value must be one of: 'adam', 'sgd'
```

9.2.5 Recommended workflow in training scripts

The following pattern gives a script three configuration layers with a clear precedence: parser defaults, then an optional parameter file, then explicit command-line flags.

```
import argparse
import sys
from toolkit.parakit import apply_parameters, load_parameters, save_parameters

def build_parser():
    parser = argparse.ArgumentParser()
    parser.add_argument("--config", default=None, help="JSON file or directory")
    parser.add_argument("--lr", type=float, default=3e-4)
    parser.add_argument("--seed", type=int, default=0)
    return parser

def main(argv=None):
    parser = build_parser()
    pre, _ = parser.parse_known_args(argv)           # read --config first
    if pre.config is not None:
        apply_parameters(parser, load_parameters(pre.config), strict=False)
    args = parser.parse_args(argv)                   # flags override the file
    run_dir = f"runs/seed{args.seed}"
    save_parameters(vars(args), f"{run_dir}/config.json")
    return args

main(["--lr", "0.001"])
```

AJLATTConfig.add_arguments(parser) registers every AJLATT option on a parser (Chapter 8), so environment parameters can be managed in the same way.

9.2.6 The ParameterTuner window

ParameterTuner opens a Tk window with one input field per tunable option: a check box for booleans, a drop-down list for options with choices, and a text field otherwise. Each field shows the option's help text and its type as reported by describe_type. Values are validated with parse_value (and the optional validation callbacks), written to timestamped JSON files, and

installed as the parser's defaults when the window closes.

Table 9.4: Constructor arguments of `ParameterTuner`.

Argument	Default	Meaning
<code>parser</code>	<code>required</code>	The <code>argparse.ArgumentParser</code> to tune.
<code>save_delay</code>	<code>30000</code>	Auto-save interval in milliseconds.
<code>save_path</code>	<code>None</code>	Output directory for the JSON files; <code>None</code> uses the current working directory. Created on the first save.
<code>auto_save_enabled</code>	<code>True</code>	Start with auto-save switched on.
<code>validation_callbacks</code>	<code>None</code>	Per-destination validators (Section 9.2.4); callbacks for unknown names are ignored with a warning.
<code>inactivity_timeout</code>	<code>10</code>	Seconds without keyboard or mouse activity after which the values are saved and the window closes; <code>None</code> disables the automatic close.

Window behaviour. The window offers the following controls:

- *Save Parameters* validates all fields and writes a new file. Invalid values are listed in an error dialog and nothing is saved.
- *OK* saves and closes the window; if the values are invalid the window stays open.
- *Auto-save* (check box) saves every `save_delay` milliseconds while enabled; problems are shown in the status line instead of dialogs.
- After `inactivity_timeout` seconds without activity the window saves (if the values are valid) and closes. The default of ten seconds suits scripts that open the window at start-up and should continue unattended; pass a larger value or `None` for interactive editing.
- *Browse* selects another output directory and *Reset to Default* returns to the initial one. Closing the window with the window manager does not save.

`tune()` blocks until the window closes and returns the parser. The values of the last *successful* save (manual, OK, auto-save or inactivity save) become the parser's defaults; if nothing was saved, the parser is unchanged. The path of the last file is available as `tuner.last_saved_path`. The class method `ParameterTuner.tune_parameters(parser, ...)` creates a tuner, calls `tune()` and returns the parser; `ParameterAdjuster` is a backward-compatible alias of the class. The font of the window is the class attribute `font_family` (default "Times New Roman").

```
from toolkit.parakit import ParameterTuner

tuner = ParameterTuner(parser, save_path="runs/params", inactivity_timeout=None,
                       validation_callbacks=validators)
parser = tuner.tune()           # blocks until the window is closed
args = parser.parse_args()
print("saved to", tuner.last_saved_path)
```

Systems without Tk or without a display. `tune()` raises `ImportError` with installation instructions when Tk is missing (the text of the message is the constant `TK_MISSING_MESSAGE` of `toolkit.parakit`) and `RuntimeError` when no display is available, for example on a cluster node or over SSH without X forwarding. Scripts that should run everywhere catch both exceptions and fall back to the headless API:

```
try:
    ParameterTuner(parser, save_path="runs/params").tune()
except (ImportError, RuntimeError) as exc:
    print(f"Editor not available ({exc}); using the current defaults.")
```

The script `examples/parakit_demo.py` demonstrates the complete workflow, including the window (`--gui`), loading a file or directory (`--load`) and command-line overrides (`--set KEY=VALUE`).

Chapter 10

plotkit: Publication-Quality Plots

`toolkit.plotkit` produces the figures that research papers on reinforcement learning need most often: learning curves aggregated over seeds with uncertainty bands, grouped bar charts with error bars, histograms and annotated heatmaps. It depends only on `matplotlib`, accepts data from `NumPy`, `pandas`, `PyTorch`, `TensorFlow` and `JAX`, and never changes the global `matplotlib` configuration unless explicitly asked to.

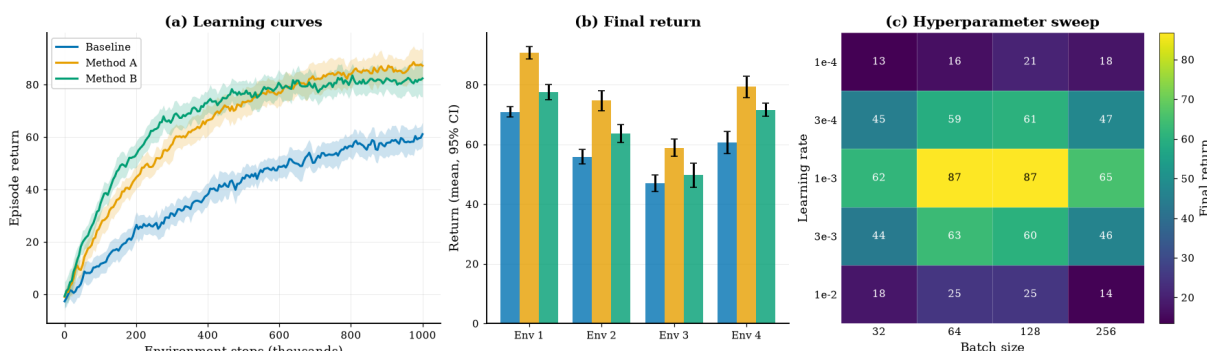


Figure 10.1: A three-panel summary figure made with `plotkit` (`examples/plotkit_gallery.py`, synthetic data): (a) learning curves of three methods over eight seeds with 95 % confidence bands and EMA smoothing, (b) grouped bars with 95 % confidence intervals, (c) an annotated heatmap of a hyperparameter sweep. Each method keeps its Okabe-Ito colour in every panel.

10.1 Overview

All plot functions share the same behaviour:

- they draw into the `ax` argument when it is given and otherwise create a new figure of size `figsize`;
- they apply the `style` preset *locally*, inside a `matplotlib.rc_context`, so the caller's `rcParams` are unchanged afterwards;
- they return the `matplotlib.axes.Axes` they drew on, which can be customised further with ordinary `matplotlib` calls.

A non-default style also restyles the axes it draws on (spines, tick labels, axis-label fonts), so that axes created outside the style context match the preset. Pass `style="default"` to leave an existing axes exactly as it is.

Table 10.1: Public API of toolkit.plotkit.

Name	Purpose
<code>plot_shadow_curve</code>	Mean curves with a shaded band (std, SEM, 95 % CI, min-max or explicit) and optional smoothing.
<code>plot_learning_curves</code>	{method: runs} learning curves aggregated over seeds; runs may have different lengths.
<code>plot_line</code> , <code>plot_scatter</code>	Line and scatter plots of one or several series.
<code>plot_bar</code>	Bar charts; several series are grouped side by side, with error bars and value labels.
<code>plot_histogram</code>	Overlaid histograms on common bins.
<code>plot_heatmap</code> , <code>plot_gray_scale</code>	Annotated heatmaps with colorbar (seaborn-compatible arguments, matplotlib only).
<code>save_figure</code>	Save a figure to several formats at once.
<code>style_context</code> , <code>set_research_style</code> , <code>reset_style</code> , <code>STYLE_PRESETS</code>	Style presets applied locally or globally.
<code>get_palette</code> , <code>RESEARCH_COLORS</code> , <code>OKABE_ITO_COLORS</code> (and the <code>*_COLOR_LIST</code> variants)	Colour palettes, including a colour-blind-safe one.

```
import numpy as np
from toolkit.plotkit import plot_shadow_curve, save_figure

rng = np.random.default_rng(0)
returns = rng.normal(size=(5, 100)).cumsum(axis=1)      # 5 seeds x 100 steps
ax = plot_shadow_curve(returns, band="ci95", smoothing=10, labels="Agent",
                      xlabel="Episode", ylabel="Return")
save_figure(ax, "renders/first_curve", formats=("pdf", "png"))
```

10.2 Input rules

Every function converts its data inputs with the same rules:

- A 1-D array-like of numbers, including a flat Python list of scalars, is *one* series.
- A list or tuple of array-likes is *several* series, one per element; the elements may have different lengths.
- A 2-D array is read by the function: `plot_shadow_curve` and `plot_learning_curves` read it as *samples of one curve* (by default rows are samples such as seeds and columns are steps); `plot_line`, `plot_scatter`, `plot_bar` and `plot_histogram` read it as (`n_series`, `n_points`). For compatibility with matplotlib's `ax.plot(x, Y)`, `plot_line` reads a 2-D array whose columns, but not rows, have `len(x)` points as one series per column.
- A pandas `DataFrame` contributes one series per column, except in `plot_shadow_curve` and `plot_learning_curves`, where it is a sample matrix like a 2-D array (rows are samples).
- Dates and times (`datetime`, pandas `Timestamp`, NumPy `datetime64`) are valid x values and are drawn on a date axis; durations (`timedelta`, `timedelta64`) are converted to

seconds. Dates used as bar categories are labelled in ISO format.

- pandas **Series**, PyTorch tensors (on any device, with or without `requires_grad`), TensorFlow eager tensors and JAX arrays are converted to NumPy automatically. Masked arrays become NaN.
- A single `x` is shared by all series; a list of arrays (or a 2-D array) gives one `x` per series. When `x` is omitted, each series is plotted against `0, ..., n - 1`.

Length mismatches raise a `ValueError` that states both shapes and, when a 2-D input looks transposed, suggests transposing it. To pass a nested list as a single 2-D sample matrix to `plot_shadow_curve`, convert it with `np.asarray` first; otherwise it is read as several curves.

10.3 Common parameters

The parameters in Table 10.2 have the same meaning in all plot functions (the heatmap functions have no `labels`, `colors`, `legend` or `grid`).

Table 10.2: Parameters shared by the plot functions.

Parameter	Default	Meaning
<code>labels</code>	<code>None</code>	Legend labels, one per series; default "Curve 1", "Series 1", A wrong count gives a warning.
<code>colors</code>	<code>None</code>	<code>None</code> (the style's colour cycle), one colour, a list of colours (cycled), a palette name ("okabe_ito", "research", ...) or a matplotlib colormap name.
<code>ax</code>	<code>None</code>	Axes to draw on; a new figure is created when <code>None</code> .
<code>figsize</code>	(10, 6)	Size in inches of a new figure ((8, 6) for heatmaps).
<code>title, xlabel, ylabel</code>	<code>None</code>	Axes title and axis labels.
<code>legend</code>	<code>True</code>	Show a legend when there are several series or labels were given; a string sets the location, e.g. "lower right".
<code>grid</code>	<code>True</code>	Draw a major grid (only along the value axis for bars and histograms).
<code>style</code>	"research"	Style preset applied locally (Section 10.7), a matplotlib style name, a dict of <code>rcParams</code> , or <code>None</code> .
<code>**kwargs</code>		Passed to the underlying matplotlib call (<code>plot</code> , <code>scatter</code> , <code>bar</code> , <code>hist</code> , <code>imshow</code>); <code>color</code> and <code>label</code> are accepted as aliases of <code>colors</code> and <code>labels</code> .

10.4 Curves with uncertainty bands

10.4.1 `plot_shadow_curve`

```
plot_shadow_curve(y, x=None, y_std=None, labels=None, colors=None, alpha=0.2,
                  ax=None, axis=0, figsize=(10, 6), title=None, xlabel=None,
                  ylabel=None, legend=True, grid=True, style="research",
                  legend_labels=None, x_tick_labels=None, y_tick_labels=None,
                  *, band="std", smoothing=None, **kwargs) -> Axes
```

Each element of `y` gives one curve. A 2-D element holds samples of the curve (for example one row per seed) and is drawn as its mean with a band; a 1-D element is drawn as it is, with a band only when `y_std` is given.

Table 10.3: Parameters specific to `plot_shadow_curve`.

Parameter	Default	Meaning
<code>y</code>	required	One or several curves (see the input rules).
<code>x</code>	None	Shared x values or one array per curve.
<code>y_std</code>	None	Explicit half-width of the band: a scalar, one array for all curves, or one entry per curve (entries may be None). Overrides <code>band</code> .
<code>alpha</code>	0.2	Band opacity; 0 hides the band.
<code>axis</code>	0	Sample axis of 2-D inputs: 0 means rows are samples and columns are steps; use 1 for the transposed layout.
<code>band</code>	"std"	"std", "sem", "ci95", "minmax" or None (see below). Keyword only.
<code>smoothing</code>	None	int: trailing moving-average window; float in (0,1): EMA weight. Keyword only.
<code>legend_labels</code>	None	Alias of <code>labels</code> (takes precedence).
<code>x_tick_labels</code> , <code>y_tick_labels</code>	None	Custom tick labels: a mapping {position: label}, or a sequence (one label per x value of the first curve, otherwise spread evenly over the data range).

Bands. Let $y_{i,t}$ be sample i (for example seed i) at step t , and let $\mathcal{I}_t$ be the set of samples that are not NaN at step t , with $n_t = |\mathcal{I}_t|$. The drawn curve is the mean

$$\bar{y}_t = \frac{1}{n_t} \sum_{i \in \mathcal{I}_t} y_{i,t}, \quad (10.1)$$

and the band is $[\bar{y}_t - h_t, \bar{y}_t + h_t]$ with

$$\text{"std"} : \quad h_t = \sigma_t = \sqrt{\frac{1}{n_t} \sum_{i \in \mathcal{I}_t} (y_{i,t} - \bar{y}_t)^2}, \quad (10.2)$$

$$\text{"sem"} : \quad h_t = \frac{s_t}{\sqrt{n_t}}, \quad s_t = \sqrt{\frac{1}{n_t - 1} \sum_{i \in \mathcal{I}_t} (y_{i,t} - \bar{y}_t)^2}, \quad (10.3)$$

$$\text{"ci95"} : \quad h_t = z_{0.975} \frac{s_t}{\sqrt{n_t}}, \quad z_{0.975} \approx 1.96, \quad (10.4)$$

while "minmax" draws $[\min_i y_{i,t}, \max_i y_{i,t}]$ over $\mathcal{I}_t$. The "std" band uses the population standard deviation; the SEM-based bands use the sample standard deviation and are undefined (not drawn) where fewer than two samples exist. The "ci95" band is the normal approximation of a 95 % confidence interval of the mean; with few seeds (fewer than about ten) it is narrower than the exact Student- t interval, a fact worth stating in a figure caption. Because the statistics ignore NaN, runs of different length can be padded with NaN.

Smoothing. Smoothing is applied along the step axis *after* the statistics, with the same filter for the mean and both band edges. An integer $w \geq 1$ selects a trailing moving average with the same output length; the window shrinks at the start of the series:

$$\tilde{y}_t = \frac{1}{|W_t|} \sum_{s \in W_t} y_s, \quad W_t = \{s : \max(0, t - w + 1) \leq s \leq t, y_s \text{ finite}\}. \quad (10.5)$$

A float $\beta \in (0, 1)$ selects a debiased exponential moving average with smoothing weight β , the convention of TensorBoard's smoothing slider (larger values are smoother):

$$\tilde{y}_t = \frac{\sum_{s \leq t, y_s \text{ finite}} \beta^{t-s} y_s}{\sum_{s \leq t, y_s \text{ finite}} \beta^{t-s}}. \quad (10.6)$$

The normalisation removes the bias towards zero of the plain recursion $m_t = \beta m_{t-1} + (1 - \beta)y_t$, $m_{-1} = 0$; for finite data the result equals `pandas.Series(y).ewm(alpha=1 - beta).mean()`. Non-finite values are skipped. Note that smoothing the band edges does not turn them into a confidence band of the smoothed curve; it is a visual aid.

10.4.2 plot_learning_curves

```
plot_learning_curves(runs, x=None, *, band="ci95", smoothing=None, colors=None,
                    alpha=0.2, ax=None, figsize=(10, 6), title=None,
                    xlabel="Step", ylabel="Return", legend=True, grid=True,
                    style="research", **kwargs) -> Axes
```

`runs` maps a method name to its results: a 2-D array (`n_seeds`, `n_steps`), a 1-D array (a single seed), or a list of 1-D arrays of possibly different lengths, which are padded with NaN. `x` is either shared or a mapping `{method: x}`. The function is a thin layer over `plot_shadow_curve` with a 95 % confidence band by default, and always shows the legend when `legend` is true.

10.5 Lines, scatter plots, bars and histograms

`plot_line(x, y, ...)` and `plot_scatter(x, y, ...)` draw one or several series with the input rules of Section 10.2. `plot_scatter` uses `s=50` and `alpha=0.7` unless they are given; passing per-point values `c=...` (for a colormap) disables the series colour.

10.5.1 plot_bar

```
plot_bar(x, height, labels=None, colors=None, ax=None, figsize=(10, 6),
        title=None, xlabel=None, ylabel=None, legend=True, grid=True,
        style="research", *, width=0.8, yerr=None, value_labels=False,
        horizontal=False, capsize=3.0, **kwargs) -> Axes
```

10.5.2 plot_histogram

`plot_histogram(data, bins="auto", ...)` overlays one histogram per series; its keyword-only arguments are `density=False`, `alpha=0.45` and `histtype="stepfilled"`. The bin edges are computed once from the pooled data (`numpy.histogram_bin_edges`) so that all series share the same bins; NaN and infinite values are dropped, and `range=(lo, hi)` restricts the shared bins as in matplotlib. `density=True` normalises each histogram to unit area, and the default y-label is "Density" or "Count" accordingly. For the filled types the fill uses `alpha` while the outline stays opaque.

10.6 Heatmaps

Table 10.4: Parameters specific to `plot_bar`.

Parameter	Default	Meaning
<code>x</code>	required	Category labels (strings or numbers), or <code>None</code> for the indices. Bars are placed at <code>0, ..., n_categories - 1</code> .
<code>height</code>	required	One series (1-D), a list of series, or a 2-D array (<code>n_series</code> , <code>n_categories</code>). Several series are grouped side by side.
<code>width</code>	0.8	Total width of a group in category units; each bar is <code>width / n_series</code> wide.
<code>yerr</code>	<code>None</code>	Error bars: a scalar, one array per series (or one shared array), or the lower and upper errors of a single series as a <code>(2, n_categories)</code> array or a pair of lists.
<code>value_labels</code>	<code>False</code>	Annotate the bars: <code>True</code> (format <code>".3g"</code>), a format spec such as <code>".1f"</code> or <code>"d"</code> , a template such as <code>"{: .1%}"</code> , a printf-style format such as <code>"%.2f"</code> , or a callable.
<code>horizontal</code>	<code>False</code>	Horizontal bars, first category at the top.
<code>capsize</code>	3.0	Error-bar cap size in points.
<code>colors</code>	<code>None</code>	As usual; for a single series a list with one colour per category colours each bar.

```
plot_heatmap(data, xlabels=None, ylabels=None, cmap="viridis", annot=False,
             ax=None, figsize=(8, 6), title=None, xlabel=None, ylabel=None,
             cbar=True, cbar_label=None, style="research", *, fmt=None,
             vmin=None, vmax=None, center=None, robust=False, mask=None,
             nan_color=None, aspect="auto", square=False, linewidths=None,
             linecolor="white", annot_kws=None, cbar_kws=None, **kwargs) -> Axes
```

`plot_heatmap` draws a 2-D array with row 0 at the top (a 1-D array is drawn as one row). It is implemented with matplotlib's `imshow` and accepts the most common seaborn arguments, so existing seaborn calls usually work unchanged.

`plot_gray_scale(data, ...)` is `plot_heatmap` with `cmap="gray"` (black is low, white is high), convenient for images, occupancy grids and attention maps; it accepts all keyword arguments of `plot_heatmap`.

10.7 Styles

A style preset is a set of `rcParams` overrides applied on top of the active configuration. Table 10.6 summarises the presets; `STYLE_PRESETS[name]` is a read-only view of the exact values.

"research" is the paper-ready default of every function. "presentation" enlarges everything for slides and posters, "minimal" produces small, quiet figures, and "default" (or `None`) leaves the active configuration untouched. The `style` argument also accepts any name from `matplotlib.style.available` and a dictionary of `rcParams`.

Three functions apply styles outside the plot functions:

```
import matplotlib.pyplot as plt
from toolkit.plotkit import reset_style, set_research_style, style_context

# 1. Locally, for any matplotlib code (rcParams are restored on exit).
with style_context("presentation"):
    fig, ax = plt.subplots()
```

Table 10.5: Parameters specific to `plot_heatmap`.

Parameter	Default	Meaning
<code>xlabels, ylabels</code>	<code>None</code>	One tick label per column / row. <code>None</code> labels cells with their index (up to 30 cells, otherwise automatic integer ticks); <code>False</code> hides the ticks. Long x labels are rotated. seaborn's <code>xticklabels</code> / <code>yticklabels</code> are aliases.
<code>cmap</code>	<code>"viridis"</code>	Colormap name or object; use a diverging map (<code>"RdBu_r"</code>) with <code>center</code> .
<code>annot</code>	<code>False</code>	<code>True</code> writes each value into its cell; an array of the same shape writes its entries instead. Text is black or white, whichever contrasts better with the cell colour.
<code>fmt</code>	<code>None</code>	Annotation format: a format spec (<code>".2g"</code> , <code>"d"</code>), a template (<code>"{: .1%}"</code>), a printf-style format (<code>"%.2f"</code>) or a callable. Default <code>".0f"</code> when all numeric annotations are whole numbers, else <code>".2g"</code> .
<code>cbar, cbar_label</code>	<code>True, None</code>	Colorbar and its label (also <code>cbar_kws={"label": ...}</code>). The colorbar is available as <code>ax.images[-1].colorbar</code> .
<code>vmin, vmax</code>	<code>None</code>	Colour limits (default: data range).
<code>center</code>	<code>None</code>	Value mapped to the middle of the colormap; missing limits are chosen symmetrically around it.
<code>robust</code>	<code>False</code>	Use the 2nd and 98th percentiles for missing limits.
<code>mask</code>	<code>None</code>	Boolean array; cells where it is <code>True</code> are hidden.
<code>nan_color</code>	<code>None</code>	Colour of NaN and masked cells (default transparent).
<code>aspect, square</code>	<code>"auto"</code> , <code>False</code>	Cell aspect ratio; <code>square=True</code> equals <code>aspect="equal"</code> .
<code>linewidths, linecolor</code>	<code>None</code> , <code>"white"</code>	Lines between cells; by default 0.5 for matrices of up to 50 cells per side and none for larger ones.
<code>annot_kws, cbar_kws</code>	<code>None</code>	Text properties of the annotations (a <code>color</code> entry disables the automatic contrast colour) and arguments of <code>Figure.colorbar</code> .

Table 10.6: Style presets. Font sizes are in points.

	<code>"research"</code>	<code>"presentation"</code>	<code>"minimal"</code>	<code>"default"</code>
Font family	serif	sans-serif	sans-serif	unchanged
Base / label size	12 / 12	16 / 18	10 / 10	unchanged
Title size, weight	14, bold	20, bold	11, normal	unchanged
Tick / legend size	10 / 10	14 / 14	9 / 9	unchanged
Line width	2.0	3.0	1.5	unchanged
Axes line width	1.2	1.5	0.8 (dark grey)	unchanged
Top and right spines	hidden	hidden	hidden	unchanged
Legend frame	framed	framed, no edge	none	unchanged
Grid opacity	0.3	0.3	0.2	unchanged
Colour cycle	<code>research</code>	<code>research</code>	<code>research</code>	unchanged

```

ax.plot([0, 1, 2], [0, 1, 4])

# 2. Globally, as an explicit opt-in; also sets savefig.dpi=300 and
#     savefig.bbox="tight". reset_style() restores the previous rcParams.
set_research_style()
fig, ax = plt.subplots()
ax.plot([0, 1, 2], [0, 1, 4])
reset_style()

```

Library code should prefer the `style` argument or `style_context`; `set_research_style` is meant for the top level of a plotting script.

10.8 Colour palettes

`get_palette(name="research", n=None)` returns a list of hex colours:

- "research" (alias "tab10"): matplotlib's ten default colours, also available as the dictionary `RESEARCH_COLORS` ("blue", "orange", ...) and the list `RESEARCH_COLOR_LIST`. It is the colour cycle of all presets.
- "okabe_ito" (alias "colorblind"): the eight colours of Okabe and Ito (2008), designed to remain distinguishable for the common forms of colour-vision deficiency; available as `OKABE_ITO_COLORS` and `OKABE_ITO_COLOR_LIST`.
- Any matplotlib colormap name. Qualitative colormaps ("tab20", "Set2", ...) return their colours; continuous colormaps ("viridis", ...) are sampled at `n` evenly spaced points in [0.1, 0.9] (eight by default).

Palettes are cycled when `n` exceeds their length. A palette name can be passed directly as `colors` to every plot function.

Table 10.7: The Okabe-Ito palette in plotting order.

Name	Hex	Name	Hex
blue	#0072B2	reddish_purple	#CC79A7
orange	#E69F00	sky_blue	#56B4E9
bluish_green	#009E73	yellow	#F0E442
vermillion	#D55E00	black	#000000

Colour-blind guidance. About one in twelve men has a red-green colour-vision deficiency. Several pairs of the default `tab10` colours, notably its red and green, are hard to tell apart with protanopia or deuteranopia, so for papers we recommend `colors="okabe_ito"`. In addition:

- use the first six Okabe-Ito colours for curves; yellow has low contrast on white and black carries no hue, so they come last;
- keep the colour of a method identical in every panel and figure, by passing the same list of colours (Section 10.10);
- do not rely on colour alone: vary line styles or markers, or label the curves directly, so that the figure also works in greyscale print;

- for heatmaps use perceptually uniform colormaps ("viridis", "cividis") for magnitudes and a diverging map with `center` for signed quantities; avoid "jet" and red-green diverging maps.

10.9 Saving figures

```
save_figure(fig_or_ax, path, formats=None, dpi=300, transparent=False,
            **savefig_kwargs) -> list[Path]
```

`save_figure` accepts a `Figure`, a `SubFigure`, an `Axes` or an array of axes (the whole parent figure is saved), creates missing parent directories and returns the written paths. With `formats` given, the extension of `path` is replaced by (or, if it has no recognised extension, extended with) each format: with the path "results/curve" and `formats=("pdf", "png")`, the files `results/curve.pdf` and `results/curve.png` are written. Without `formats` the extension of `path` is used, or `rcParams["savefig.format"]` if it has none. `dpi` applies to raster formats and to rasterised parts of vector files. The defaults `bbox_inches="tight"` and `pad_inches=0.05` can be overridden through `**savefig_kwargs`.

For LaTeX documents, save vector PDF at the final size of the figure (set `figsize` to the column or text width, for example 3.5×2.5 inches for a two-column paper, and include it without scaling), so that the font sizes of the style preset are the sizes that appear in print. Some publishers require TrueType instead of Type 3 fonts in PDF files; set `pdf.fonttype` while saving:

```
import matplotlib as mpl

with mpl.rc_context({"pdf.fonttype": 42, "ps.fonttype": 42}):
    save_figure(ax, "renders/curve_truetype.pdf")
```

10.10 Recipes

10.10.1 Reinforcement-learning training curves

Episode returns logged for several seeds form a 2-D array (`n_seeds`, `n_episodes`). Smoothing with an EMA weight between 0.6 and 0.9 makes the trend visible without hiding the variability between seeds:

```
import numpy as np
from toolkit.plotkit import plot_shadow_curve

rng = np.random.default_rng(1)
episodes = np.arange(500)
returns = 100 * (1 - np.exp(-episodes / 150)) + rng.normal(0, 15, (10, 500))

ax = plot_shadow_curve(returns, x=episodes, band="ci95", smoothing=0.9,
                      labels="PP0", colors="okabe_ito",
                      xlabel="Episode", ylabel="Episode return",
                      title="Training progress (10 seeds, 95% CI)")
```

10.10.2 Comparing methods, including unfinished runs

`plot_learning_curves` takes one entry per method. Runs of different length, for example episodes that ended early or seeds that are still training, are passed as lists and padded with NaN. The following recipe combines an environment of `env_lib` with `plotkit`: it records the order parameter r of the Kuramoto model for three coupling strengths and eight seeds. Runs with strong coupling synchronise and terminate early, so the curves end at different steps, and the band of the strongest coupling is computed from the seeds that are still running.

```
import numpy as np
import env_lib
from toolkit.plotkit import plot_learning_curves

def order_parameter_trace(coupling, seed, steps=300):
    env = env_lib.KuramotoOscillatorEnv(n_oscillators=10, coupling_mode="constant",
                                         coupling_strength=coupling, max_steps=steps)
    obs, info = env.reset(seed=seed)
    trace = [info["order_parameter"]]
    action = np.zeros(env.action_space.shape, dtype=np.float32) # no control input
    for _ in range(steps):
        obs, reward, terminated, truncated, info = env.step(action)
        trace.append(info["order_parameter"])
        if terminated or truncated:
            break
    env.close()
    return np.asarray(trace)

runs = {f"K = {k}": [order_parameter_trace(k, seed) for seed in range(8)]
        for k in (0.1, 0.2, 0.5)}
ax = plot_learning_curves(runs, band="ci95", colors="okabe_ito",
                          xlabel="Step", ylabel="Order parameter r",
                          legend="lower right")
```

10.10.3 Several panels with consistent colours

Create the axes yourself, pass them as `ax`, and give every panel the same list of colours. Creating the figure inside `style_context` also applies the preset to anything drawn directly with matplotlib, such as a figure title.

```
import matplotlib.pyplot as plt
import numpy as np
from toolkit.plotkit import (get_palette, plot_bar, plot_learning_curves,
                             save_figure, style_context)

rng = np.random.default_rng(0)
methods = ["Baseline", "Method A", "Method B"]
colors = get_palette("okabe_ito", len(methods))
runs = {m: rng.normal(size=(8, 200)).cumsum(axis=1) + 0.1 * i * np.arange(200)
        for i, m in enumerate(methods)}
finals = np.array([r[:, -1].mean() for r in runs.values()]) # one per method

with style_context("research"):
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.8), layout="constrained")
    plot_learning_curves(runs, colors=colors, smoothing=0.8, ax=axes[0],
                        title="(a) Learning curves")
    plot_bar(methods, finals, colors=colors, ax=axes[1], value_labels=".1f",
```

```

        title="(b) Final return")                # one colour per bar
save_figure(fig, "renders/comparison", formats=("pdf", "png"))

```

10.10.4 Distinguishing series without colour

Keyword arguments apply to all series of one call. To give each method its own line style, call the function once per method on the same axes and pass the colours explicitly:

```

fig, ax = plt.subplots(figsize=(6, 4))
for method, color, linestyle in zip(methods, colors, ["-", "--", ":"]):
    plot_shadow_curve(runs[method], labels=method, colors=color, ax=ax,
                     band="sem", linestyle=linestyle)

```

10.10.5 Bar chart of final performance

```

from toolkit.plotkit import plot_bar

methods = ["DQN", "PPO", "SAC", "Ours"]
accuracy = np.array([71.2, 78.5, 80.1, 84.3])
std_err = np.array([1.8, 1.2, 1.5, 1.1])
ax = plot_bar(methods, accuracy, yerr=std_err, value_labels=".1f",
              colors="okabe_ito", ylabel="Success rate (%)")

# Several environments: height has shape (n_methods, n_environments).
scores = rng.uniform(40, 90, size=(3, 4))
ax = plot_bar(["Env 1", "Env 2", "Env 3", "Env 4"], scores,
              labels=["A", "B", "C"], yerr=rng.uniform(1, 4, size=(3, 4)))

```

10.10.6 Confusion and correlation matrices

```

from toolkit.plotkit import plot_heatmap

classes = ["left", "stay", "right"]
cm = np.array([[48, 5, 2], [4, 40, 6], [1, 7, 52]])
ax = plot_heatmap(cm, xlabels=classes, ylabels=classes, annot=True,
                  cmap="Blues", square=True, xlabel="Predicted",
                  ylabel="True", cbar_label="Count")

# Correlations: diverging colormap centred at zero, upper triangle hidden.
corr = np.corrcoef(rng.normal(size=(6, 200)))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
ax = plot_heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0,
                  vmin=-1, vmax=1, mask=mask, square=True, cbar_label="r")

```

10.10.7 Distributions of returns

```

from toolkit.plotkit import plot_histogram

ax = plot_histogram([rng.normal(0, 1, 1000), rng.normal(1.5, 0.7, 1000)],
                    labels=["Policy A", "Policy B"], density=True,

```

```
colors="okabe_ito", xlabel="Return")
```

10.10.8 Plotting tensors and data frames

Tensors can be passed without conversion; the functions detach them and copy them to the CPU. A pandas DataFrame gives one series per column, which suits logs read with `pandas.read_csv`.

```
import torch
from toolkit.plotkit import plot_line

# three series of 100 points, shape (n_series, n_points)
losses = torch.rand(3, 100, requires_grad=True).cumsum(dim=1)
ax = plot_line(None, losses, labels=["actor", "critic", "entropy"])
```

10.11 Command-line gallery

The module is executable and shows every plot type with synthetic data. The same program is installed as the console script `plotkit-gallery`.

```
python -m toolkit.plotkit --demo all          # all eight demos in one window
# writes renders/plotkit/plotkit_heatmap.png and .pdf
plotkit-gallery --demo heatmap --style presentation \
    --save renders/plotkit --format png pdf
```

Table 10.8: Options of `python -m toolkit.plotkit` and `plotkit-gallery`.

Option	Default	Meaning
<code>--demo</code>	<code>all</code>	shadow, learning, heatmap, grayscale, line, bar, scatter, histogram, or all for a 2×4 grid of all demos.
<code>--style</code>	<code>research</code>	Style preset.
<code>--save DIR</code>	<code>none</code>	Save to <code>DIR/plotkit_<demo>.<format></code> instead of showing a window.
<code>--format</code>	<code>png</code>	One or more output formats.
<code>--dpi</code>	<code>150</code>	Resolution of raster output.
<code>--seed</code>	<code>0</code>	Seed of the synthetic data.

Without `--save` the figure is shown with `plt.show()`; when the active matplotlib backend is non-interactive (for example with `MPLBACKEND=Agg`), the figure is saved to `renders/plotkit/` instead. The script `examples/plotkit_gallery.py` builds the three-panel figure of Figure 10.1 and is a good starting point for a paper figure; run it with `--help` for its options.

Chapter 11

neural_toolkit: Networks and Tabular Tools

`toolkit.neural_toolkit` provides configurable PyTorch modules for reinforcement learning: policy, value and Q-networks with MLP, convolutional, recurrent and transformer trunks, encoders and decoders (including a variational autoencoder), factories that build them by name, a collection of utilities for initialisation, optimisation and checkpoints, and NumPy-based tools for tabular methods. It does not implement learning algorithms; the modules are ordinary `torch.nn.Module` objects meant to be combined with your own training code.

The subpackage requires PyTorch (`pip install "[torch]"`). Importing it without PyTorch raises an `ImportError` that explains how to install it; the rest of `toolkit` does not need PyTorch.

11.1 Conventions

- *Batch first.* Inputs have a leading batch dimension B: feature vectors (B, D), images (B, C, H, W) and sequences (B, T, D).
- *One MLP convention.* Every fully connected stack in the toolkit consists of hidden blocks `Linear -> [LayerNorm] -> activation -> [Dropout]` followed by a linear output layer without activation.
- *Raw outputs.* Policies return unnormalised outputs of size `output_dim`: logits for a categorical policy, or distribution parameters (for example means) for a continuous one. Turning them into a distribution is left to the algorithm.
- *Devices.* Every constructor accepts `device` (default "cpu") and moves the module there at the end of construction; `device=None` leaves it where PyTorch created it. Inputs must be on the same device.
- *Validation.* Invalid sizes, dropout rates, activation names or cell types raise `ValueError` or `TypeError` at construction, with a message that lists the valid options.

11.2 Building blocks

Activations. `get_activation(name)` returns a new activation module for a case-insensitive name, or instantiates an `nn.Module` subclass passed instead of a name. The valid names are available as `ACTIVATION_NAMES`:

Name	Module	Name	Module
relu	nn.ReLU	gelu	nn.GELU
leaky_relu	nn.LeakyReLU	silu, swish	nn.SiLU
elu	nn.ELU	mish	nn.Mish
tanh	nn.Tanh	softplus	nn.Softplus
sigmoid	nn.Sigmoid	identity	nn.Identity

MLPs. `build_mlp` returns an `nn.Sequential`. Without `output_dim` only the hidden stack is built; its output width is `hidden_dims[-1]`.

```
build_mlp(input_dim, hidden_dims, output_dim=None, activation="relu", dropout=0.0,
          layer_norm=False, output_activation=None, bias=True) -> nn.Sequential
```

```
import torch
from toolkit.neural_toolkit import build_mlp, get_activation

mlp = build_mlp(4, [32, 32], output_dim=2, activation="tanh", layer_norm=True)
print(mlp(torch.zeros(8, 4)).shape)      # torch.Size([8, 2])
print(get_activation("swish"))           # SiLU()
```

Positional encoding. `PositionalEncoding(d_model, dropout=0.1, max_len=5000)` adds the sinusoidal table of Vaswani et al. (2017) to a batch-first sequence (B, T, d_model); position t receives

$$\text{PE}_{t,2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{PE}_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right), \quad (11.1)$$

for every batch element, followed by dropout. Sequences longer than `max_len` raise a `ValueError`. The table is a non-persistent buffer and is not stored in `state_dict`.

11.3 Network families

Table 11.1 lists all networks with their input and output shapes. Four classes have two names: the historical spellings `CNPolicyNetwork`, `RNPolicyNetwork`, `CNQNetwork` and `RNQNetwork` and the aliases with `CNN` and `RNN` refer to the same classes.

11.3.1 Constructor arguments

All classes share the arguments `activation="relu"`, `dropout=0.0`, `layer_norm=False` and `device="cpu"`; the transformer classes default to `dropout=0.1` and `layer_norm=True`. The remaining arguments depend on the trunk (Table 11.2).

`layer_norm=True` inserts `LayerNorm` after every hidden linear layer; in convolutional trunks it additionally inserts `BatchNorm2d` after every convolution.

11.3.2 Architectures

Convolutional trunk. `Conv2d -> [BatchNorm2d] -> activation -> [Dropout2d]` blocks, adaptive average pooling to `pooled_size × pooled_size` (1 for policies, value and Q-networks), flattening and an MLP with `fc_dims`, followed by the output layer. Attribute names: `conv_layers`, `adaptive_pool`, `fc_layers`.

Table 11.1: Networks, encoders and decoders with their shapes. B: batch, T: sequence length, A: action_dim, O: output_dim, L: latent_dim.

Class (alias)	Input	Output
<i>Policy networks</i> (input_dim, output_dim)		
MLPPolicyNetwork	(B, D)	(B, O)
CNPolicyNetwork (CNNPolicyNetwork)	(B, C, H, W)	(B, O)
RNPolicyNetwork (RNNPolicyNetwork)	(B, T, D) or (B, D)	((B, O), hidden)
TransformerPolicyNetwork	(B, T, D), mask (B, T)	(B, O)
<i>Value networks</i> (input_dim, output_dim=1)		
MLPValueNetwork	(B, D)	(B, O)
CNNValueNetwork	(B, C, H, W)	(B, O)
RNNValueNetwork	(B, T, D) or (B, D)	((B, O), hidden)
TransformerValueNetwork	(B, T, D), mask (B, T)	(B, O)
<i>Q-networks</i> (state_dim, action_dim); optional integer action (B,)		
MLPQNetwork	(B, D)	(B, A), or (B,) with action
MLPQNetwork, continuous critic	(B, D), (B, A)	(B, 1); see below
DuelingQNetwork	(B, D)	(B, A), or (B,) with action
CNQNetwork (CNNQNetwork)	(B, C, H, W)	(B, A), or (B,) with action
RNQNetwork (RNNQNetwork)	(B, T, D) or (B, D)	(q, hidden), q as above
TransformerQNetwork	(B, T, D), mask (B, T)	(B, A), or (B,) with action
<i>Encoders</i> (input_dim or input_channels, latent_dim)		
MLPEncoder	(B, D)	(B, L)
CNNEncoder	(B, C, H, W)	(B, L)
RNNEncoder	(B, T, D) or (B, D)	((B, L), hidden)
TransformerEncoder	(B, T, D), mask (B, T)	(B, L)
VariationalEncoder	(B, D)	(z, mu, logvar), each (B, L)
<i>Decoders</i> (latent_dim, output_dim or output_channels)		
MLPDecoder, VariationalDecoder	(B, L)	(B, O)
CNNDecoder	(B, L)	(B, C, S, S), $S = \text{initial_size} \cdot \prod \text{strides}$
RNNDecoder, TransformerDecoder	(B, L), seq_len	(B, T, O), T = seq_len or max_seq_len

Table 11.2: Trunk-specific constructor arguments and their defaults.

Trunk	Arguments
MLP	<code>hidden_dims=(256, 256)</code> . For <code>DuelingQNetwork</code> the shared trunk uses <code>hidden_dims[:-1]</code> and each stream one hidden layer of width <code>hidden_dims[-1]</code> .
Convolutional	<code>input_channels</code> , <code>conv_dims=(32, 64, 128)</code> , <code>fc_dims=(256, 256)</code> , <code>kernel_sizes</code> (int or one per layer, default 3), <code>strides</code> (default 1). Padding is <code>kernel_size // 2</code> . The feature map is globally average-pooled, so any image size works. <code>CNNEncoder</code> : <code>conv_dims=(32, 64, 128, 256)</code> , <code>fc_dims=(512, 256)</code> , <code>strides</code> 1 for the first layer and 2 afterwards, <code>pooled_size=4</code> (the map is pooled to 4×4).
Transposed conv.	<code>CNNDecoder</code> : <code>output_channels</code> , <code>fc_dims=(512, 256)</code> , <code>conv_dims=(256, 128, 64, 32)</code> , <code>kernel_sizes</code> (default 3), <code>strides</code> (default 2), <code>initial_size=4</code> , <code>output_activation="sigmoid"</code> ; the default output is 64×64 (property <code>output_size</code>).
Recurrent	<code>hidden_dim=256</code> , <code>num_layers=2</code> , <code>rnn_type="lstm"</code> (or "gru"), <code>bidirectional=False</code> (True for <code>RNNEncoder</code> ; not available for <code>RNNDecoder</code>), <code>fc_dims=(256,)</code> ; <code>RNNDecoder</code> : <code>max_seq_len=100</code> .
Transformer	<code>d_model=256</code> (divisible by <code>nhead</code>), <code>nhead=8</code> , <code>num_layers=6</code> , <code>dim_feedforward=1024</code> , <code>dropout=0.1</code> , <code>fc_dims=(256,)</code> , <code>max_len=5000</code> ; <code>TransformerDecoder</code> : <code>max_seq_len=100</code> , <code>max_len=None</code> (then <code>max(max_seq_len, 5000)</code>).

Recurrent trunk. A batch-first LSTM or GRU followed by an MLP head that reads the final hidden state of the last layer; for a bidirectional RNN the forward state after the last step and the backward state after the first step are concatenated, so both directions have seen the whole sequence. `forward(x, hidden=None)` accepts a sequence or a single step (B, D) and returns the new hidden state (`h_n` for a GRU, (`h_n`, `c_n`) for an LSTM), which can be passed to the next call:

```
from toolkit.neural_toolkit import RNNPolicyNetwork

policy = RNNPolicyNetwork(input_dim=5, output_dim=3, hidden_dim=32, rnn_type="gru")
hidden = None
for t in range(4):
    logits, hidden = policy(torch.randn(8, 5), hidden)
print(logits.shape, hidden.shape)  # (8, 3) and (num_layers, 8, 32)
```

Transformer trunk. A linear projection to `d_model`, the positional encoding, a batch-first `nn.TransformerEncoder`, mean pooling over the valid positions and an MLP head. The optional mask is a boolean (B, T) tensor in which True marks padding (PyTorch's `src_key_padding_mask` convention); padded positions are excluded from attention and pooling. Any sequence length up to `max_len` is accepted.

```
from toolkit.neural_toolkit import TransformerPolicyNetwork

policy = TransformerPolicyNetwork(input_dim=6, output_dim=4, d_model=64, nhead=4,
                                  num_layers=2, dim_feedforward=128)
x = torch.randn(2, 10, 6)  # two sequences of length 10
mask = torch.zeros(2, 10, dtype=torch.bool)
mask[1, 7:] = True  # second sequence has 7 steps
print(policy(x, mask).shape)  # torch.Size([2, 4])
```

Q-networks. Discrete Q-networks return one value per action, (B, A). Passing integer action indices of shape (B,) or (B, 1) as second argument returns the selected values $Q(s, a)$ with shape (B,), which is what temporal-difference losses need. `MLPQNetwork` with `continuous_action=True` is a critic for continuous actions (DDPG, TD3, SAC): it concatenates state and action and returns $Q(s, a)$ with shape (B, 1). `DuelingQNetwork` (Wang et al., 2016) combines a value stream $V(s)$ and an advantage stream $A(s, a)$ as

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'). \quad (11.2)$$

```
from toolkit.neural_toolkit import DuelingQNetwork, MLPQNetwork

q = DuelingQNetwork(state_dim=6, action_dim=3, hidden_dims=(64, 64))
states, actions = torch.randn(5, 6), torch.tensor([0, 1, 2, 1, 0])
print(q(states).shape, q(states, actions).shape)  # (5, 3) and (5,)

critic = MLPQNetwork(state_dim=6, action_dim=2, continuous_action=True)
print(critic(states, torch.randn(5, 2)).shape)  # (5, 1)
```

Variational autoencoder. `VariationalEncoder` returns (z, mu, logvar) with $z = \mu + \exp(\frac{1}{2} \log \sigma^2) \varepsilon$ and $\varepsilon \sim \mathcal{N}(0, I)$ drawn from PyTorch's default generator (the reparameterisation trick). The static method `kl_divergence(mu, logvar)` returns the per-sample KL term of the evidence lower bound,

$$\text{KL}(\mathcal{N}(\mu, \text{diag } \sigma^2) \parallel \mathcal{N}(0, I)) = -\frac{1}{2} \sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2 \right), \quad (11.3)$$

with shape (B,). `VariationalDecoder` is an MLP from z to the reconstruction.

```
import torch.nn.functional as F
from toolkit.neural_toolkit import VariationalDecoder, VariationalEncoder

encoder = VariationalEncoder(input_dim=64, latent_dim=8, hidden_dims=(128,))
decoder = VariationalDecoder(latent_dim=8, output_dim=64, hidden_dims=(128,))
x = torch.randn(32, 64)
z, mu, logvar = encoder(x)
reconstruction = F.mse_loss(decoder(z), x, reduction="none").sum(dim=1)
loss = (reconstruction + encoder.kl_divergence(mu, logvar)).mean()
```

Sequence decoders. `RNNDecoder` feeds the projected latent vector to a unidirectional LSTM or GRU at every time step and maps each output with a shared MLP head. `TransformerDecoder` is non-autoregressive: the queries are positional encodings of the output positions, the memory is the projected latent vector, and all positions are decoded in parallel. Both generate `max_seq_len` steps unless `forward(z, seq_len=...)` asks for another length.

11.4 Factories

Five factories build networks from a type name, which is convenient when the architecture is a command-line option. Names are case-insensitive; an unknown name raises a `ValueError` that lists the valid names. The remaining keyword arguments are passed to the constructor of the selected class.

Factory	Static method	Valid type names
PolicyFactory	<code>create_policy</code>	mlp, cnn, rnn, transformer
ValueFactory	<code>create_value_network</code>	mlp, cnn, rnn, transformer
QNetworkFactory	<code>create_q_network</code>	mlp, dueling, cnn, rnn, transformer
EncoderFactory	<code>create_encoder</code>	mlp, cnn, rnn, transformer, vae
DecoderFactory	<code>create_decoder</code>	mlp, cnn, rnn, transformer, vae

Each method takes the type name as its first argument followed by the keyword arguments of the constructor.

```
from toolkit.neural_toolkit import PolicyFactory, QNetworkFactory

policy = PolicyFactory.create_policy("mlp", input_dim=8, output_dim=4,
                                     hidden_dims=(64, 64))
q_net = QNetworkFactory.create_q_network("dueling", state_dim=8, action_dim=4)
```

11.5 NetworkUtils

NetworkUtils groups helper functions as static methods (Table 11.3).

11.6 Tabular tools

The tabular tools work on finite state and action spaces with NumPy arrays and do not need a GPU. Every table owns a `numpy.random.Generator` (argument `rng`: a generator, an integer seed, or `None` for fresh entropy) that is used for all sampling, so results are reproducible without touching the global random state. States and actions are integer indices; out-of-range indices raise `IndexError`.

The three table classes share one constructor:

```
QTable(state_space_size, action_space_size=None, initial_value=0.0,
        dtype=np.float32, rng=None)           # also ValueTable, PolicyTable
```

QTable

Action values $Q[s, a]$ (`action_space_size` is required). Entries are read and written with `get_value`, `set_value`, `update_value` and `get_q_values`. `get_max_q_value` and `get_max_action` return the greedy value and action (the lowest index among ties). `get_policy` samples an epsilon-greedy action, and the methods `get_epsilon_greedy_probs`, `get_softmax_probs` and `get_softmax_policy` give the corresponding probabilities or a softmax sample.

ValueTable

State values $V[s]$, with `get_values` for the whole array.

PolicyTable

A stochastic policy $\pi[s, a]$ that starts uniform. `set_value` and `update_value` change one probability and rescale the others so that the row still sums to one; `set_policy_probs`, `set_deterministic_policy`, `get_policy_probs` and `get_policy` (sampling) complete the interface.

Table 11.3: Static methods of `NetworkUtils`.

Method	Description
<i>Initialisation and parameters</i>	
<code>initialize_weights(module, method="xavier_uniform", gain=1.0)</code>	Initialise every linear and (transposed) convolution layer recursively; biases are set to zero. Methods: <code>xavier_uniform</code> , <code>xavier_normal</code> , <code>kaiming_uniform</code> , <code>kaiming_normal</code> , <code>orthogonal</code> , <code>uniform</code> ($U(-0.1, 0.1)$), <code>normal</code> ($\mathcal{N}(0, 0.1^2)$).
<code>count_parameters(model)</code>	Number of trainable parameters.
<code>count_parameters_by_layer(model)</code>	Trainable parameters owned directly by each sub-module.
<code>freeze_layers(model, names)</code> , <code>unfreeze_layers(model, names)</code>	Disable or enable gradients of parameters whose name contains any of <code>names</code> .
<i>Gradients</i>	
<code>get_grad_norm(model, norm_type=2.0)</code>	Total gradient norm.
<code>clip_grad_norm(model, max_norm, norm_type=2.0)</code>	Clip in place; returns the norm before clipping.
<i>Optimisation</i>	
<code>create_optimizer(model, optimizer_type="adam", lr=1e-3, weight_decay=0.0, skip_list=None, **kw)</code>	<code>adam</code> , <code>adamw</code> , <code>sgd</code> , <code>rmsprop</code> , <code>adagrad</code> , <code>adamax</code> . With <code>weight_decay > 0</code> the decay is applied to weights only (see <code>apply_weight_decay</code>).
<code>apply_weight_decay(model, weight_decay, skip_list=None)</code>	Parameter groups without decay for one-dimensional parameters (biases, normalisation scales) and for names in <code>skip_list</code> .
<code>create_scheduler(optimizer, scheduler_type="step", **kw)</code>	<code>step</code> , <code>multistep</code> , <code>exponential</code> , <code>cosine</code> , <code>cosine_warm_restart</code> , <code>plateau</code> , <code>linear</code> , <code>polynomial</code> ; <code>**kw</code> go to the PyTorch scheduler (for example <code>step_size</code> or <code>T_max</code>).
<i>Checkpoints</i>	
<code>save_checkpoint(model, optimizer, epoch, loss, filepath, **extra)</code>	Save model and optimizer state (<code>optimizer</code> may be <code>None</code>), epoch, loss and any extra entries; atomic write; returns the path.
<code>load_checkpoint(model, optimizer, filepath, map_location=None, weights_only=True, strict=True)</code>	Restore a checkpoint and return (<code>epoch</code> , <code>loss</code>). Tensors are loaded onto the device of the model by default, so GPU checkpoints load on CPU-only machines.
<i>Layer construction</i>	
<code>create_mlp_layers</code> , <code>create_conv_layers</code> , <code>create_transposed_conv_layers</code>	MLP and (transposed) convolution stacks following the conventions of Section 11.1.
<code>create_attention_layer(input_dim, num_heads=8, dropout=0.1)</code>	Batch-first <code>nn.MultiheadAttention</code> .
<code>create_positional_encoding(d_model, max_len)</code>	Frozen sinusoidal table of shape $(1, \text{max_len}, \text{d_model})$; <code>max_len=5000</code> .
<code>get_activation(activation)</code>	Same as the module-level function.
<i>Shape arithmetic</i>	
<code>compute_output_size(n, k, s=1, p=0, d=1)</code>	$\lfloor (n + 2p - d(k - 1) - 1)/s \rfloor + 1$
<code>compute_transposed_output_size(n, k, s=1, p=0, op=0, d=1)</code>	$(n - 1)s - 2p + d(k - 1) + op + 1$
<code>compute_conv_output_size((H, W), layers)</code>	Spatial size after a list of layer dictionaries.

DiscreteEnvironment

An abstract base class for small finite environments, constructed with (`state_space_size`, `action_space_size`, `rng=None`). Subclasses implement `reset()`, which returns the initial state, and `step(action)`, which returns (`state`, `reward`, `done`, `info`).

`update_value(state, value, action, learning_rate)` moves an entry towards a target: $x \leftarrow x + \alpha(\text{value} - x)$. The static methods of `DiscreteTools` implement the classic updates on top of it, with discount γ (`gamma=0.99`) and step size α (`alpha=0.1`). On a terminal transition (`done=True`) the bootstrap term is dropped.

$$\text{Q-learning: } Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)], \quad (11.4)$$

$$\text{SARSA: } Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)], \quad (11.5)$$

$$\text{Expected SARSA: } Q(s, a) \leftarrow Q(s, a) + \alpha\left[r + \gamma \sum_{a'} \pi(a' | s') Q(s', a') - Q(s, a)\right]. \quad (11.6)$$

For expected SARSA, π is a given `PolicyTable` or, if none is given, the epsilon-greedy policy of the Q-table with the given `epsilon`. `value_iteration_update(value_table, state, q_table)` sets $V(s) = \max_a Q(s, a)$ (the Q-table must already hold one-step look-ahead values), and `policy_iteration_update(policy_table, state, q_table)` makes $\pi(\cdot | s)$ greedy with respect to Q .

The exploration rules are static methods of `DiscreteTools` whose first two arguments are the Q-table and the state; they sample with the Q-table's generator.

epsilon_greedy_policy(q, s, epsilon)

A uniformly random action with probability ε , otherwise the greedy one. The corresponding probabilities (with ties sharing the greedy mass) are $\pi(a | s) = \varepsilon/|\mathcal{A}| + (1 - \varepsilon) \mathbf{1}[a \in G_s]/|G_s|$, where G_s is the set of greedy actions.

softmax_policy(q, s, temperature)

Also available as `boltzmann_policy`. Samples from $\pi(a | s) \propto \exp(Q(s, a)/\tau)$, computed in a numerically stable way.

ucb_policy(q, s, visit_counts, exploration_constant=1.0)

UCB1: $\arg \max_a Q(s, a) + c\sqrt{\ln N(s)/N(s, a)}$ with $N(s) = \sum_a N(s, a)$; untried actions are chosen first, at random among themselves. `visit_counts` has the shape of the Q-table.

thompson_sampling_policy(q, s, visit_counts, ...)

A heuristic Beta-Bernoulli Thompson sampler with $p = \text{sigmoid}(Q(s, a))$ and posterior $\text{Beta}(\alpha_0 + np, \beta_0 + n(1 - p))$, $n = N(s, a)$, where α_0 and β_0 are `prior_alpha` and `prior_beta` (default 1). It is exact only for Bernoulli rewards with Q in logit space.

The following example learns to walk right on a six-state chain. The reward is sparse, so the Q-table starts with optimistic values of 1, which makes the greedy policy try every action before settling:

```
import numpy as np
from toolkit.neural_toolkit import DiscreteEnvironment, DiscreteTools

class Chain(DiscreteEnvironment):
    """Chain 0..n-1: action 1 moves right, 0 moves left; reward 1 at the end."""

    def reset(self):
        self.current_state = 0
        return self.current_state
```

```

def step(self, action):
    move = 1 if action == 1 else -1
    last = self.state_space_size - 1
    self.current_state = int(np.clip(self.current_state + move, 0, last))
    done = self.current_state == last
    return self.current_state, float(done), done, {}

env = Chain(state_space_size=6, action_space_size=2, rng=0)
q = DiscreteTools.create_q_table(6, 2, initial_value=1.0, rng=0)  # optimistic
for episode in range(200):
    state = env.reset()
    for t in range(50):
        action = DiscreteTools.epsilon_greedy_policy(q, state, epsilon=0.1)
        next_state, reward, done, _ = env.step(action)
        DiscreteTools.q_learning_update(q, state, action, reward, next_state,
                                         gamma=0.9, alpha=0.5, done=done)
        state = next_state
    if done:
        break
print([q.get_max_action(s) for s in range(5)])  # [1, 1, 1, 1, 1]

```

11.7 Example: a DQN agent for LineMsg

The following script trains a small deep Q-network on the LineMsg environment of Chapter 5 with four agents, treating the 2^4 joint actions as one discrete action. It shows the pieces that a training loop typically needs: action-value selection with `q_net(s, a)`, a target network, an optimizer created by name, gradient clipping and checkpointing. On a laptop CPU it runs in a few seconds and the greedy policy reaches the return of the always-relay strategy (about 65 per episode, compared with about 30 for random actions).

```

from collections import deque

import numpy as np
import torch
import torch.nn.functional as F

import env_lib
from toolkit.neural_toolkit import MLPQNetwork, NetworkUtils

torch.manual_seed(0)
rng = np.random.default_rng(0)
env = env_lib.LineMsgEnv(num_agents=4, max_iter=50)
obs_dim = int(np.prod(env.observation_space.shape))  # 4 agents x 3 cells
n_actions = int(env.action_space.n)  # 2**4 joint actions

q_net = MLPQNetwork(obs_dim, n_actions, hidden_dims=(64, 64))
target_net = MLPQNetwork(obs_dim, n_actions, hidden_dims=(64, 64))
target_net.load_state_dict(q_net.state_dict())
optimizer = NetworkUtils.create_optimizer(q_net, "adam", lr=1e-3)
buffer = deque(maxlen=10_000)

obs, info = env.reset(seed=0)
for step in range(4000):
    epsilon = max(0.05, 1.0 - step / 2000)
    if rng.random() < epsilon:

```

```

        action = int(rng.integers(n_actions))
    else:
        with torch.no_grad():
            action = int(q_net(torch.as_tensor(obs.reshape(1, -1))).argmax())
    next_obs, reward, terminated, truncated, info = env.step(action)
    buffer.append((obs.ravel(), action, reward, next_obs.ravel(), terminated))
    obs = next_obs
    if terminated or truncated:
        obs, info = env.reset()

    if len(buffer) >= 500:
        batch = [buffer[i] for i in rng.integers(len(buffer), size=64)]
        s, a, r, s2, done = (torch.as_tensor(np.array(x)) for x in zip(*batch))
        with torch.no_grad():
            next_q = target_net(s2).max(dim=1).values
            target = r.float() + 0.99 * (1.0 - done.float()) * next_q
        loss = F.smooth_l1_loss(q_net(s, a), target)  # q_net(s, a): Q(s, a), (B,)
        optimizer.zero_grad()
        loss.backward()
        NetworkUtils.clip_grad_norm(q_net, max_norm=10.0)
        optimizer.step()
    if step % 500 == 0:
        target_net.load_state_dict(q_net.state_dict())

NetworkUtils.save_checkpoint(q_net, optimizer, epoch=step, loss=loss.item(),
                             filepath="checkpoints/dqn_linemsg.pt")

```

Evaluating the greedy policy and restoring the checkpoint later:

```

def evaluate(policy, episodes=10):
    returns = []
    for episode in range(episodes):
        obs, info = env.reset(seed=100 + episode)
        total, done = 0.0, False
        while not done:
            obs, reward, terminated, truncated, info = env.step(policy(obs))
            total += reward
            done = terminated or truncated
        returns.append(total)
    return float(np.mean(returns))

restored = MLPQNetwork(obs_dim, n_actions, hidden_dims=(64, 64))
epoch, last_loss = NetworkUtils.load_checkpoint(restored, None,
                                                "checkpoints/dqn_linemsg.pt")

def greedy(obs):
    with torch.no_grad():
        return int(restored(torch.as_tensor(obs.reshape(1, -1))).argmax())

greedy_return = evaluate(greedy)
random_return = evaluate(lambda o: int(rng.integers(n_actions)))
print(f"greedy {greedy_return:.1f}, random {random_return:.1f}")

```

Two larger examples ship with the repository (Chapter 12):

- `examples/simple_transformer_example.py` trains a `TransformerPolicyNetwork` with REINFORCE on a memory task;

- `examples/transformer_rl_example.py` trains transformer actor and critic networks with PPO on batched environments.

11.8 Changes in version 1.0

Version 1.0 fixed several defects that changed the computed functions. The most important ones are:

- The MLP stacks of earlier versions applied no activation between hidden layers, so deep “MLPs” were nearly linear. This affected every policy, value and Q-network, encoder and decoder.
- The positional encoding was indexed by batch position instead of sequence position.
- Convolutional networks with activations other than ReLU crashed; the dueling head was combined incorrectly; Q-networks ignored the `action` argument; `CNNEncoder` failed on 64×64 inputs; `CNNDecoder` indexed its kernel sizes incorrectly; bidirectional RNNs read a backward state that had seen only one time step.
- Unknown activation names now raise instead of falling back silently, and the `device` argument is honoured.
- Tabular tools use a per-table random generator instead of the global one; UCB, softmax and `PolicyTable.set_value` were corrected.

Checkpoint compatibility. Because the layer structure changed (activation modules now sit between the linear layers, so the indices of `nn.Sequential` entries differ), model checkpoints written by versions before 1.0 cannot be loaded into 1.0 networks: `load_state_dict` reports missing and unexpected keys. Even where the parameter shapes happen to match, the networks compute different functions. Retrain models with version 1.0; Appendix B lists the other API changes.

Part III

Practice

Chapter 12

Examples, Benchmarks and Development

This chapter is a practical companion to the reference chapters. It lists the example scripts that ship with the repository, shows how to write evaluation loops and record episodes as GIF or MP4 files, explains the benchmark script, and describes the testing, linting and continuous-integration workflow for contributors.

12.1 Example scripts

The directory `examples/` contains runnable, documented scripts. Every script has a `--help` option, a module docstring that explains what it demonstrates, and a `main()` function. Media files are written to `renders/` (relative to the working directory), which Git ignores. Run the scripts from the repository root after installing the package with the extras they need (`pip install ".[all]"` covers all of them).

Table 12.1: Environment examples.

Script	What it shows
<code>quickstart.py</code>	Creates every registered environment with <code>env_lib.make</code> , takes a few random steps and prints the observation shape and return; <code>--render DIR</code> saves one PNG frame per environment.
<code>kuramoto_demo.py</code>	Drives an oscillator network to synchronisation with a phase-alignment controller and a coupling ramp; NumPy or batched PyTorch backend (<code>--backend</code>), several topologies.
<code>linemsg_demo.py</code>	Compares random, duty-cycled and always-relay policies on LineMsg (<code>--policy</code> selects the one to show or record).
<code>wireless_comm_demo.py</code>	Compares random access, slotted ALOHA and a collision-free rotating schedule on WirelessComm.
<code>pistonball_demo.py</code>	Compares a hand-written heuristic with random actions; <code>--manual</code> enables keyboard control in a window.
<code>consensus_demo.py</code>	Compares random actions with the distributed Laplacian controller for consensus or formation control on any topology.
<code>ajlatt_demo.py</code>	Compares random actions with an encircling heuristic for multi-robot target tracking and reports tracking error, covariance and collisions.

The environment examples share a set of options: `--seed`, `--steps` (the episode length), `--episodes` (evaluation episodes per policy), `--save PATH` (record an episode; the suffix `.gif`

Table 12.2: Toolkit examples.

Script	What it shows
<code>plotkit_gallery.py</code>	The three-panel paper figure of Figure 10.1: learning curves with confidence bands, grouped bars and a sweep heatmap, saved as PDF and PNG.
<code>parakit_demo.py</code>	Saving, loading and applying <code>argparse</code> parameters with validation callbacks; <code>--gui</code> opens the Tk editor, <code>--load</code> reads a file or directory, <code>--set KEY=VALUE</code> overrides one value.
<code>simple_transformer_example.py</code>	A <code>TransformerPolicyNetwork</code> trained with REINFORCE on a small memory task (requires PyTorch).
<code>transformer_rl_example.py</code>	PPO with transformer actor and critic networks on batched environments (requires PyTorch; <code>--device auto</code> uses a GPU when available).

or `.mp4` selects the format), `--render-mode` (`human` opens a window) and `--theme dark|light`. The transformer examples accept `--quick` for a run of a few seconds, used as a smoke test in continuous integration.

```
python examples/quickstart.py --render renders/quickstart
python examples/kuramoto_demo.py --topology ring --save renders/kuramoto.gif
python examples/linemsg_demo.py --policy relay --save renders/linemsg.gif
python examples/wireless_comm_demo.py --policy schedule --save renders/wireless.gif
python examples/pistonball_demo.py --episodes 20 --save renders/pistonball.gif
python examples/consensus_demo.py --task formation --save renders/formation.gif
python examples/ajlatt_demo.py --map obstacles05 --save renders/ajlatt.gif
python examples/plotkit_gallery.py --formats pdf png
python examples/simple_transformer_example.py --quick
```

Table 12.3 lists the results that the comparison scripts print with their default arguments. They are useful as sanity checks after modifying an environment and as simple baselines; the environment chapters explain the policies.

Table 12.3: Output of the comparison scripts with default arguments (mean episode return, with the standard deviation over episodes where the script prints one).

Script (setting)	Result
<code>linemsg_demo.py</code> (10 agents, 20 episodes)	random 44.1 ± 4.1 , duty-cycled 85.2 ± 3.0 , always relay 95.0
<code>wireless_comm_demo.py</code> (6×6 , 20 episodes)	random 385 ± 12 , ALOHA 398 ± 16 , rotating schedule 911 ± 17 with no collisions
<code>pistonball_demo.py</code> (20 pistons, 5 episodes)	heuristic 889 ± 22 (all episodes reach the goal), random -342 ± 40 (none)
<code>consensus_demo.py</code> (ring, 8 agents)	Laplacian controller solves the task after 99 steps (return -1968); random actions do not (return -19906)
<code>ajlatt_demo.py</code> (obstacles04, 2 episodes)	heuristic about -7300 with no collisions and target covariance trace about 2; random actions about -45000 with 29 to 37 collisions

12.2 Writing an evaluation loop

All environments share the Gymnasium interface, so one evaluation function serves all of them. The loop below evaluates a policy over several seeded episodes and reports the team return and the per-agent returns from `info["agent_rewards"]`. `np.sum` and `np.any` make it work also for AJLATT, whose reward and `terminated` are per-robot arrays.

```
import numpy as np
import env_lib

def evaluate(env, policy, episodes=5, seed=0):
    """Mean team return and mean per-agent returns over seeded episodes."""
    team, per_agent = [], []
    for episode in range(episodes):
        obs, info = env.reset(seed=seed + episode)
        total, agents = 0.0, 0.0
        while True:
            obs, reward, terminated, truncated, info = env.step(policy(obs))
            total += float(np.sum(reward))
            agents = agents + info["agent_rewards"]
            if np.any(terminated) or truncated:
                break
        team.append(total)
        per_agent.append(agents)
    return np.mean(team), np.mean(per_agent, axis=0)

env = env_lib.make("WirelessComm-v1")
env.action_space.seed(0) # reproducible random actions
mean_return, agent_returns = evaluate(env, lambda obs: env.action_space.sample())
print(f"{mean_return:.1f}", agent_returns.shape) # 16 agents on the 4x4 grid
env.close()
```

Policies that need more than the observation, such as the controllers built into some environments, can close over the environment: `lambda obs: env.unwrapped.laplacian_policy()` for the consensus environment. Use `env.unwrapped` to reach environment-specific methods and attributes through the wrappers that `gymnasium.make` adds.

12.3 Recording episodes

The module `env_lib.utils` provides two functions to turn rendered frames into animations:

```
record_episode(env, policy=None, path=None, *, max_steps=None, seed=None,
               fps=None, render_every=1) -> list[np.ndarray]
save_animation(frames, path, fps=30.0, *, loop=0) -> Path
```

`record_episode` requires an environment created with `render_mode="rgb_array"`. It resets the environment with `seed`, renders the initial state, then steps with `policy(observation)` (uniform random actions from `env.action_space` when no policy is given) until the episode terminates or is truncated, or `max_steps` steps have been taken, rendering every `render_every`-th step. It returns the list of (H, W, 3) uint8 frames and, if `path` is given, writes them with `save_animation` at `fps` frames per second (default: the environment's `metadata["render_fps"]`).

`save_animation` chooses the format from the file suffix: `.gif` is written with Pillow, which is always available; `.mp4`, `.webm`, `.avi`, `.mov` and `.mkv` require the video extra (imageio and

imageio-ffmpeg). For video formats the frames are cropped to even width and height, which most codecs require. Parent directories are created. `loop` is the GIF loop count (0 repeats forever).

```
import env_lib
from env_lib.utils import record_episode, save_animation, set_theme

set_theme("light")          # renderers created from now on use the light theme
env = env_lib.ConsensusEnv(task="formation", formation_shape="wedge",
                           render_mode="rgb_array")
frames = record_episode(env, policy=lambda obs: env.laplacian_policy(),
                        path="renders/formation.gif", seed=0, render_every=2)
save_animation(frames, "renders/formation.mp4", fps=15)  # needs the video extra
env.close()
set_theme("dark")          # back to the default theme
```

`set_theme("dark"|"light")` selects the colour theme of all renderers created afterwards: the dark theme (default) suits screens and slides, the light theme suits papers and printed material. Custom themes can be added with `register_theme`.

For full control, for example to record only part of an episode or to annotate frames, collect the frames yourself with `env.render()`:

```
env = env_lib.make("KuramotoOscillator-v0", render_mode="rgb_array")
obs, info = env.reset(seed=1)
frames = []
for step in range(300):
    obs, reward, terminated, truncated, info = env.step(env.action_space.sample())
    if step % 5 == 0:          # every fifth step
        frames.append(env.render())
    if terminated or truncated:
        break
save_animation(frames, "renders/kuramoto.gif", fps=20)
env.close()
```

GIF files grow quickly with the number of frames and the frame size; rendering every second or fifth step and using MP4 for long episodes keeps the files small. The first `render()` call of an environment builds its figure and takes noticeably longer than later calls.

12.4 Benchmarks

The script `benchmarks/benchmark_envs.py` measures the mean wall-clock time of `env.step` (with pre-sampled random actions and no rendering) and the additional time of `env.render()` in "rgb_array" mode, for representative configurations of every environment. Environments whose optional dependencies are missing are reported as skipped. The script sets `MPLBACKEND=Agg` and `SDL_VIDEODRIVER=dummy` itself, so it runs headless.

```
python benchmarks/benchmark_envs.py          # all environments
python benchmarks/benchmark_envs.py --steps 500 --only kuramoto ajlatt
python benchmarks/benchmark_envs.py --markdown > renders/benchmarks.md
```

Table 12.5 shows typical results. The “before 1.0” column was measured with the implementations that preceded version 1.0 on the same machine; the version 1.0 columns are the output of the script with its default options (x86_64 Linux, Python 3.11, CPU only, PyTorch restricted to one thread). Absolute numbers depend on the hardware; the ratios are more informative.

Table 12.4: Options of `benchmarks/benchmark_envs.py`.

Option	Default	Meaning
<code>--steps</code>	300	Timed steps per case.
<code>--render-steps</code>	40	Timed rendered steps per case; 0 skips the rendering measurement.
<code>--seed</code>	0	Seed of the environment and of the action space.
<code>--only</code>	all	Subset of the groups <code>kuramoto</code> , <code>linemsg</code> , <code>wireless</code> , <code>pistonball</code> , <code>consensus</code> , <code>ajlatt</code> .
<code>--markdown</code>	off	Print a Markdown table instead of plain text.

Table 12.5: Time per step and per rendered frame in milliseconds. A dash marks a configuration that did not exist before version 1.0.

Configuration	Step, before 1.0	Step, 1.0	Frame, 1.0
Kuramoto NumPy, $N = 10$	0.17	0.05	20
Kuramoto NumPy, $N = 50$	2.36	0.05	36
Kuramoto NumPy RK4, $N = 50$	7.1	0.10	36
Kuramoto PyTorch, $N = 50$, 8 systems	10.3	0.55	34
LineMsg, 10 agents	0.019	0.012	15
WirelessComm, 6×6	0.154	0.035	13
WirelessComm, 12×12	0.56	0.044	14
Pistonball, 20 pistons	1.11	0.063	3
Consensus, 8 agents	–	0.066	16
Formation, 32 agents, proximity graph	–	0.20	20
AJLATT, <code>obstacles04</code> , 4 robots	90	6.2	28
AJLATT, <code>obstacles05</code> , 4 robots	67	6.2	26

Before version 1.0 the Kuramoto "rgb_array" renderer failed with matplotlib 3.10 and later, Pistonball drew its screen on every step even without rendering, and an AJLATT frame took about 110 ms. Rendering now costs more than a simulation step in every environment, which is why it is performed only on request (Section 13.9).

12.5 Testing, linting and continuous integration

12.5.1 Running the tests

The test suite lives in `tests/` (`tests/envs/` for the environments, `tests/toolkit/` for the toolkit) and uses `pytest`. After an editable installation with the `dev` extra (Section 2.3):

```
pytest -q                                # whole suite (under a minute)
pytest tests/envs/test_consensus.py -q    # one file
pytest -k "seed or determinism" -q       # tests whose name matches
pytest --cov=env_lib --cov=toolkit --cov-report=term-missing
```

The suite contains more than 700 tests. `tests/conftest.py` sets `MPLBACKEND=Agg` and `SDL_VIDEODRIVER=dummy`, so rendering tests run without a display. Tests of optional features are skipped when the dependency is missing (`pytest.importorskip`). The marker `slow` is reserved for long-running tests and can be deselected with `-m "not slow"`.

The tests check, for every environment, the shapes, dtypes and bounds of the spaces, that observations lie in `observation_space`, determinism under seeding, reward and termination rules against simple reference implementations written in the tests, backward-compatible arguments, the registered ids, rendered frames and, where the multi-agent interface permits it, Gymnasium's `check_env`. For the toolkit they cover input handling and numerical results of `plotkit`, network shapes and the version 1.0 fixes of `neural_toolkit`, and the conversion rules of `parakit`.

12.5.2 Linting and formatting

The code base is formatted and linted with `ruff`; the configuration (line length 100, target Python 3.9, rule sets E, F, W, I, B, UP, NPY) is in `pyproject.toml`.

```
ruff check src tests examples benchmarks # lint
ruff check --fix src tests examples benchmarks # apply automatic fixes
ruff format src tests examples benchmarks # format
ruff format --check src tests examples benchmarks # verify formatting only
```

12.5.3 Continuous integration

The workflow `.github/workflows/ci.yml` runs on pushes to the `main` and `development` branches and on pull requests. It has two jobs:

lint

On Python 3.11: `ruff check` and `ruff format --check` on `src`, `tests`, `examples` and `benchmarks`.

test

On Python 3.9, 3.10, 3.11 and 3.12 with `MPLBACKEND=Agg` and `SDL_VIDEODRIVER=dummy`: installs the CPU build of PyTorch and the package with the `pistonball`, `video` and `dev` extras, runs `pytest -q`, and smoke-tests the examples with `python examples/quickstart.py`

`--steps 5, python examples/simple_transformer_example.py --quick --no-plot` and `python benchmarks/benchmark_envs.py --steps 20 --render-steps 2`.

Running the same commands locally before opening a pull request avoids failed CI runs.

12.5.4 Conventions for contributions

Contributions follow the conventions that the existing code relies on:

- Python 3.9 compatible code with `from __future__ import annotations`, type hints on public functions and NumPy-style docstrings (Parameters, Returns, Raises, Examples).
- No global side effects in library code: no `print`, no `logging.basicConfig`, no calls to the global NumPy or PyTorch random generators, no changes to `matplotlib.rcParams` and no `matplotlib.use`. Randomness comes from `self.np_random` or an explicitly owned generator.
- Arguments are validated early with `ValueError` or `TypeError` and actionable messages; user-facing problems are reported with `warnings.warn(..., stacklevel=2)`; diagnostics go to `logging.getLogger(__name__)`.
- Environments follow the contract of Chapter 3: Gymnasium API, `float32` observations inside `observation_space`, `info["agent_rewards"]`, their own episode limit, and no drawing work unless a render mode is set. Renderers subclass `env_lib.utils.MatplotlibRenderer`, build their artists once and use the colours of the active theme.
- Deprecated arguments keep working and emit a `DeprecationWarning` that names the replacement; every behavioural change is recorded in `CHANGELOG.md`.
- Plain ASCII text in code, output and documentation, without emojis or decorative symbols.

A new environment needs the following additions:

1. an entry in `ENV_SPECS` in `src/env_lib/registration.py`, with a string entry point so that registration does not import optional dependencies;
2. a lazy export in `src/env_lib/__init__.py`;
3. tests in `tests/envs/` and an example script in `examples/`;
4. a chapter in this manual and an entry in `CHANGELOG.md`.

Chapter 13

Troubleshooting

This chapter collects the problems that users most often run into, grouped by symptom, together with their causes and solutions. If a problem is not covered here, run the failing code with warnings enabled (`python -W default ...`), check the installation with the commands of Section 2.4, and report the issue with the full error message and the output of `pip list` at https://github.com/EricDMW/My_Tool_Box/issues.

13.1 Rendering on machines without a display

Symptom. Rendering fails or hangs on a compute cluster, in a container, over SSH or in continuous integration, often with messages such as `cannot connect to X server`, `no display name` or `TclError`.

Solution. Use `render_mode="rgb_array"`, which draws off-screen and never needs a display: the matplotlib renderers draw on an Agg canvas that is independent of the active matplotlib backend, and the Pistonball renderer draws on pygame surfaces without initialising the display. In addition, select non-interactive back ends for any other plotting code:

```
export MPLBACKEND=Agg          # matplotlib: off-screen rendering
export SDL_VIDEODRIVER=dummy   # pygame: no window system
python train.py
```

The same settings can be made at the top of a script, before matplotlib or pygame is imported:

```
import os

os.environ.setdefault("MPLBACKEND", "Agg")
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
```

Frames returned by `env.render()` can then be recorded to GIF or MP4 (Section 12.3) and inspected later. The test suite, the benchmark script and the CI workflow use exactly these settings.

13.2 No window appears in human mode

Symptom. With `render_mode="human"` nothing is shown, and a `RuntimeWarning` states that the active matplotlib backend is non-interactive.

Cause. "human" mode opens a window through matplotlib, which needs an interactive backend such as TkAgg or QtAgg and a display. When matplotlib falls back to Agg (because no GUI toolkit is installed, no display is available, or `MPLBACKEND=Agg` is set), the environment warns once and renders nothing. Pistonball's window is opened by pygame and needs a display, but no matplotlib backend.

Solution. Check which backend is active:

```
python -c "import matplotlib; print(matplotlib.get_backend())"
```

If it prints `agg` (any case), install a GUI toolkit and select it:

- TkAgg needs Tk: `sudo apt-get install python3-tk` (Debian, Ubuntu), `sudo dnf install python3-tkinter` (Fedora), `brew install python-tk` (Homebrew) or `conda install tk`.
- QtAgg needs a Qt binding: `pip install PyQt6` (or `PySide6`).

Then unset `MPLBACKEND` or set it explicitly, for example `export MPLBACKEND=TkAgg`. In a Jupyter notebook, "human" mode is not the right tool: render with `"rgb_array"` and display the frames with `matplotlib.pyplot.imshow` or save an animation. Over SSH, either enable X forwarding (`ssh -X`) or record the episode and copy the file.

13.3 Missing optional dependencies

Optional dependencies are imported only when a feature needs them, and the resulting error names the extra that provides them (Table 13.1).

Table 13.1: Errors caused by missing optional dependencies.

Error message (abbreviated)	Fix
PistonballEnv requires pymunk (and pygame for rendering)	<code>pip install ".[pistonball]"</code>
Pistonball rendering requires pygame	<code>pip install ".[pistonball]"</code>
toolkit.neural_toolkit requires PyTorch	<code>pip install ".[torch]"</code>
No module named 'torch' when creating KuramotoOscillatorEnvTorch or a KuramotoOscillatorTorch-* id	<code>pip install ".[torch]"</code>
Video export requires imageio and imageio-ffmpeg	<code>pip install ".[video]"</code>
The parakit GUI requires Tk (the 'tkinter' module)	system package <code>python3-tk</code> or <code>conda install tk</code> (Section 2.2.3)
No module named 'pytest' or 'ruff'	<code>pip install -e ".[all,dev]"</code>

Note that `pip install ".[dev]"` installs only the development tools; the test suite skips the tests of optional features that are not installed, so a complete test run needs `".[all,dev]"`. If an extra is installed but the error persists, the package was probably installed into a different Python environment than the one running the code; compare `which python` with `pip -V`, and use `python -m pip install ...` to be sure.

13.4 “Out of date” warnings for -v0 environment ids

Symptom. Creating `WirelessComm-v0` or `KuramotoOscillator-v0` with `gymnasium.make` prints

```
DeprecationWarning: WARN: The environment WirelessComm-v0 is out of date.
You should consider upgrading to version `v1`.
```

Cause and solution. `Gymnasium` assumes that a higher version suffix is a newer revision of the same environment. In `env_lib` the suffixes denote preset configurations: `WirelessComm-v1` is the 4×4 grid, not an improved `WirelessComm-v0`. The warning is harmless and can be ignored. `env_lib.make`, which otherwise behaves like `gymnasium.make`, suppresses it. To silence it when calling `gymnasium.make` directly:

```
import warnings
import gymnasium as gym
import env_lib # registers the ids

warnings.filterwarnings("ignore", message=r".*is out of date",
                        category=DeprecationWarning)
env = gym.make("WirelessComm-v0")
```

13.5 AJLATT deprecation warnings

Symptom. Creating an AJLATT environment prints warnings such as AJLATT parameter `'num_Robot'` is deprecated; use `'num_robots'` or parameter `'figID'` no longer has any effect and is ignored.

Cause. The environment is configured by the `AJLATTConfig` dataclass since version 1.0. The legacy keyword names of the former `argparse`-based configuration are still accepted by `AJLATTEnv(...)`, `env_lib.ajlatt_env(...)`, `env_lib.ajlatt_env.make(...)` and `AJLATTConfig.from_kwargs`, but translated with a `DeprecationWarning`.

Solution. Rename the arguments as in Table 13.2; Appendix B shows complete before-and-after examples.

Unknown names raise a `TypeError` in `AJLATTEnv(...)` and `AJLATTConfig.from_kwargs`, and are ignored with a warning by the backward-compatible factory `make`. Python hides `DeprecationWarning` outside the `__main__` module by default; run `python -W default::DeprecationWarning script.py` to see every translation when migrating a code base.

13.6 PyTorch, GPUs and devices

Installing a CPU or a CUDA build. The `torch` extra installs the default PyTorch wheel of the platform, which on Linux is a large CUDA build. For CPU-only machines install the CPU wheel first (`pip install torch --index-url https://download.pytorch.org/whl/cpu`);

Table 13.2: Legacy AJLATT parameters and their replacements.

Legacy argument	Replacement
num_Robot, num_Target	num_robots, num_targets
T_steps	max_episode_steps
maxmum_run	max_episode_steps; the episode is limited to maxmum_run + 1 steps
SIGPERCENT	range_noise_proportional
useupdate	use_update
fix_seed	seed=<int> in the configuration, or <code>env.reset(seed=...)</code>
render=True	render_mode="human" (or "rgb_array")
figID, record, ros, log_dir, episode_num, env_name, ...	no effect; remove them

for a specific CUDA version use the command shown at <https://pytorch.org/get-started/>. Check the result with

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

The Kuramoto PyTorch backend. `KuramotoOscillatorEnvTorch` accepts `device="cpu"`, `"cuda"`, `"cuda:<index>"`, a `torch.device`, or `"auto"`, which selects the first GPU when CUDA is available and the CPU otherwise (the registered id `KuramotoOscillatorTorch-v2` uses `"auto"`). Requesting `"cuda"` on a machine without CUDA raises a `ValueError` that suggests `"auto"` or `"cpu"`. Observations returned by `reset` and `step` are always NumPy arrays; the batched accessors `get_batch_observations()` and `get_batch_rewards()` return tensors on the environment's device. A GPU pays off for large batches of systems; for a single small system the CPU is faster because of the transfer and launch overhead.

neural_toolkit devices. The networks take a `device` argument (default `"cpu"`) that is passed to `Module.to`; `"auto"` is not accepted there. Choose the device explicitly and move the inputs to it:

```
import torch
from toolkit.neural_toolkit import MLPPolicyNetwork

device = "cuda" if torch.cuda.is_available() else "cpu"
policy = MLPPolicyNetwork(8, 4, device=device)
logits = policy(torch.randn(32, 8, device=device))
```

`NetworkUtils.load_checkpoint` loads tensors onto the device of the model by default, so checkpoints written on a GPU load on a CPU-only machine. Checkpoints written before version 1.0 cannot be loaded (Section 11.8).

13.7 Video export

Symptom. Saving a `.mp4` (or `.webm`, `.avi`, `.mov`, `.mkv`) file fails with `Video export requires imageio and imageio-ffmpeg`, or `imageio` reports that it cannot find an `ffmpeg` executable.

Solution. Install the video extra, `pip install "[video]"`. The `imageio-ffmpeg` package ships its own `ffmpeg` binary, so no system installation is needed. GIF export uses Pillow, which is always installed with matplotlib, and is the fallback when a video codec is unavailable. `save_animation` rejects other suffixes with a `ValueError`; frames must all have the same shape (H, W, 3) or (H, W, 4).

13.8 Reproducibility and seeding

The environments are deterministic given their seeds and never use global random state, but a complete experiment contains several independent sources of randomness. Seed each of them:

- *Environment.* Call `env.reset(seed=s)` once at the start of a run; later calls to `env.reset()` without a seed continue the same random stream, so a whole sequence of episodes is reproducible. Resetting with the same seed every episode repeats the same initial state.
- *Action sampling.* `env.action_space.sample()` uses the space's own generator, which `reset` does not seed. Call `env.action_space.seed(s)`. This also applies to `record_episode` without a policy.
- *Construction-time randomness.* Some environments draw random structure in the constructor, independent of `reset`: the Erdős-Rényi graph of `ConsensusEnv` (`graph_seed`; with the default `None` two instances may differ, even in their observation size) and the random topology of the Kuramoto environments (`topology_seed=42`). AJLATT accepts a default seed in its configuration (`seed=...`), used by the first `reset()` without a seed.
- *Networks.* Call `torch.manual_seed(s)` before creating networks and before training. `VariationalEncoder` samples its noise from PyTorch's default generator. On GPUs some operations are non-deterministic; see the PyTorch documentation on reproducibility (`torch.use_deterministic_algorithms`).
- *Tabular tools.* Pass `rng=s` to the tables; each table owns its generator.
- *Your own code.* Use a `numpy.random.default_rng(s)` generator instead of the global `np.random` functions.

Seeded trajectories are reproducible for a fixed version of the package and its dependencies. Across versions, results can change where bugs were fixed; Appendix B lists these changes. The AJLATT environment can reproduce its behaviour before version 1.0 closely with `ci_solver="slsqp"`.

13.9 Performance

- *Render only when needed.* Rendering a frame costs 3 to 36 ms, which is far more than a simulation step (Table 12.5). Train with `render_mode=None`, and record evaluation episodes separately, with `render_every` greater than one for long episodes. "human" mode additionally limits the frame rate, so never train in "human" mode.
- *Create environments once.* Construction loads maps, builds graphs and allocates buffers; reuse an environment across episodes with `reset()` instead of creating a new one per episode. The first `render()` call builds the figure and is slower than later calls.
- *Choose the integrator.* For the Kuramoto environments, Euler integration is about twice as fast as RK4; use RK4 when accuracy at large `dt` matters.

- *Batch on the GPU.* The PyTorch Kuramoto environment simulates `n_agents` systems per step. One batched environment is much cheaper than many separate ones, especially on a GPU.
- *AJLATT.* Keep the default `ci_solver="newton"`, which is several times faster than `"slsqp"`; the cost per step grows with the number of robots because every robot fuses its neighbours' estimates.
- *Small networks on the CPU.* PyTorch uses several threads by default. For the small networks typical of these environments, and when several training processes share a machine, thread contention can make training much slower; limit the threads with `torch.set_num_threads(1)` or the environment variable `OMP_NUM_THREADS=1` and compare.
- *Measure.* `benchmarks/benchmark_envs.py` (Section 12.4) measures step and frame times on your machine; `python -m cProfile -s cumtime script.py` shows where time goes in your own code.

13.10 Other common errors

`step()` raises before the first `reset()`

Every environment requires `reset()` before the first `step()`. Otherwise it raises `env_lib.ResetNeededError`, which can be caught either as a `RuntimeError` or as Gymnasium's `ResetNeeded` error. Environments created with `gymnasium.make` raise Gymnasium's own `ResetNeeded` from the `OrderEnforcing` wrapper.

An episode is longer or shorter than expected

Each environment enforces its own episode limit through a constructor argument (`max_steps`, `max_iter`, `max_cycles` or `max_episode_steps`); pass it to the constructor or to `env_lib.make`. Passing `max_episode_steps` to `gymnasium.make` adds a second `TimeLimit` wrapper, and the shorter of the two limits applies.

`AttributeError` for an environment method

`gymnasium.make` and `env_lib.make` return a wrapped environment. Methods and attributes that only the environment class defines are reached through `env.unwrapped`; for example, the consensus controller is `env.unwrapped.laplacian_policy()`.

`render()` returns `None` with a warning

The environment was created without a render mode. Pass `render_mode="rgb_array"` (or `"human"`) to the constructor or to `make`.

Actions are rejected

Actions must match `env.action_space` in shape. Pistonball in discrete mode expects one value in `{0, 1, 2}` per piston, not a single integer; AJLATT expects an array of shape `(num_robots, 2)`. Appendix B describes the changes from earlier versions.

The parakit window does not open

`ParameterTuner.tune()` raises `ImportError` without Tk and `RuntimeError` without a display; use the headless API (Section 9.2.6).

Figures look different from the examples

`plotkit` applies its style locally and does not change global settings. If other code changed `matplotlib.rcParams` (or called `set_research_style` without `reset_style`), the preset is applied on top of those settings; call `matplotlib.rcdefaults()` to start from the `matplotlib` defaults.

Appendix A

API Reference

This appendix lists the public classes and functions of version 1.0 with their signatures, grouped by module. Signatures omit type annotations; defaults are shown as in the source code. The chapters give the semantics, and the docstrings (`help(obj)` in Python) give full details. Names that start with an underscore and modules not listed here are internal and may change without notice.

A.1 `env_lib`

A.1.1 Package functions

```
env_lib.make(env_id, **kwargs) -> gym.Env
env_lib.list_envs() -> list[str]
env_lib.register_envs()
env_lib.__version__           # "1.0.0"
env_lib.ResetNeededError     # exception class
```

`make` creates a registered environment (a thin wrapper around `gymnasium.make` that suppresses the “out of date” warning of `-v0` ids); keyword arguments go to the environment constructor. `list_envs` returns the registered ids in the order of Table A.1. `register_envs` registers all ids with Gymnasium; it runs automatically on `import env_lib` and is idempotent. `ResetNeededError` is raised by every environment when `step()` or `render()` is called before `reset()`; it subclasses both `RuntimeError` and `gymnasium.error.ResetNeeded`. The environment classes below are exported lazily by `env_lib`, so optional dependencies are imported only when a class is used.

A.1.2 Common environment interface

Every environment class implements the Gymnasium methods

```
reset(*, seed=None, options=None) -> tuple[observation, dict]
step(action) -> tuple[observation, reward, terminated, truncated, dict]
render() -> np.ndarray | None      # (H, W, 3) uint8 in "rgb_array" mode
close()                          # idempotent
```

and has the attributes `action_space`, `observation_space`, `render_mode`, `metadata` and `np_random`. The `info` dictionary always contains `"agent_rewards"`, a float64 array of shape

Table A.1: Registered environment ids and their preset arguments.

Id	Class and preset arguments
KuramotoOscillator-v0	KuramotoOscillatorEnv() (10 oscillators)
KuramotoOscillator-v1	n_oscillators=5
KuramotoOscillator-Constant-v0	n_oscillators=6, coupling_mode="constant"
KuramotoOscillator-FreqSync-Constant-v0	as above, reward_type="frequency_synchronization", target_frequency=2.0
KuramotoOscillatorTorch-v0	KuramotoOscillatorEnvTorch() (10 oscillators)
KuramotoOscillatorTorch-v1	n_oscillators=8, n_agents=4, device="cpu"
KuramotoOscillatorTorch-v2	n_oscillators=10, device="auto", integration_method="rk4"
KuramotoOscillatorTorch-Constant-v0	n_oscillators=6, coupling_mode="constant"
KuramotoOscillatorTorch-FreqSync-Constant-v0	as above, frequency synchronisation, target_frequency=2.0
LineMsg-v0	LineMsgEnv()
WirelessComm-v0	WirelessCommEnv() (6 × 6 grid)
WirelessComm-v1	grid_x=4, grid_y=4
Pistonball-v0	PistonballEnv()
Consensus-v0	ConsensusEnv()
Formation-v0	ConsensusEnv(task="formation")
AJLATT-v0	AJLATTEnv() (Gymnasium's passive environment checker disabled)

(n_agents,). Chapter 3 describes the conventions.

A.1.3 KuramotoOscillatorEnv and KuramotoOscillatorEnvTorch

Module env_lib.kos_env; Chapter 4.

```
KuramotoOscillatorEnv(n_oscillators=10, n_agents=1, dt=0.01, max_steps=1000,
                      coupling_range=(0.0, 5.0), control_input_range=(-1.0, 1.0),
                      natural_freq_range=(0.5, 2.0), device='cpu',
                      render_mode=None, integration_method='euler',
                      reward_type='order_parameter', noise_std=0.0,
                      topology='fully_connected', adj_matrix=None,
                      target_frequency=1.0, coupling_mode='dynamic',
                      constant_coupling_matrix=None, coupling_strength=1.0,
                      normalize_coupling=False, topology_seed=42,
                      sync_threshold=0.99, sync_bonus=10.0)

KuramotoOscillatorEnvTorch(...same arguments..., device='cpu', render_agent=0)
```

KuramotoOscillatorEnv simulates one network with NumPy. KuramotoOscillatorEnvTorch simulates n_agents independent systems in a batch on device ("cpu", "cuda", "cuda:<k>", a torch.device or "auto"); render_agent selects the rendered system. Additional methods of the PyTorch class:

Method	Description
<code>get_batch_observations()</code>	Tensor of the observations of all systems, (<code>n_agents</code> , <code>obs_dim</code>), on the device.
<code>get_batch_rewards(dphases_dt=None, include_bonus=False)</code>	Per-system rewards of the current state, (<code>n_agents</code> ,).

`env_lib.kos_env.register()` is deprecated; registration happens on import.

A.1.4 LineMsgEnv and WirelessCommEnv

Modules `env_lib.linemsg_env` and `env_lib.wireless_comm_env`; Chapter 5.

```
LineMsgEnv(num_agents=10, n_obs_neighbors=1, max_iter=50, render_mode=None,
           action_space_type='discrete')

WirelessCommEnv(grid_x=6, grid_y=6, ddl=2, packet_arrival_probability=0.8,
                success_transmission_probability=0.8, n_obs_neighbors=1,
                max_iter=50, render_mode=None)
```

Both support the render modes "human", "rgb_array" and "ansi" (text). `seed(seed=None)` is kept for backward compatibility and deprecated in favour of `reset(seed=...)`. `WirelessCommEnv` also exposes `access_point_mapping(i, j, agent_action)`, which returns the access point (row, column) used by agent (i, j) (or (None, None)), and `check_transmission_fail(ap_x, ap_y, access_point_profile)`, which is True unless exactly one agent uses the access point. `env_lib.wireless_comm_env.register()` is deprecated.

A.1.5 PistonballEnv

Module `env_lib.pistonball_env`; Chapter 6. Requires the pistonball extra.

```
PistonballEnv(n_pistons=20, time_penalty=-0.1, continuous=True, random_drop=True,
              random_rotate=True, ball_mass=0.75, ball_friction=0.3,
              ball_elasticity=1.5, max_cycles=125, render_mode=None,
              movement_penalty=0.0, movement_penalty_threshold=0.01, kappa=1,
              terminated_condition=True, leftmost_piston_reward=0.0,
              termination_reward=0.5)

ManualPolicy(env, agent_id=0, show_obs=False)
```

Member	Description
<code>observable_mask(ball_x=None)</code>	Boolean (<code>n_pistons</code> ,) mask of the pistons within <code>kappa</code> of the piston under the ball.
<code>get_local_reward_for_piston(i, prev_x, curr_x)</code>	Reference implementation of one piston's local reward.
<code>get_local_reward(prev_position, curr_position)</code>	The ball term of the local reward.
<code>renderer</code>	The <code>PistonballRenderer</code> , or <code>None</code> before the first <code>render()</code> .
<code>seed(seed=None)</code>	Deprecated; use <code>reset(seed=...)</code> .
<code>draw()</code> , <code>draw_background()</code> , <code>draw_pistons()</code> , <code>enable_render()</code>	Deprecated no-ops kept for compatibility; use <code>render()</code> .

`ManualPolicy` is a keyboard controller for "human" mode: calling it returns a joint action in which the selected piston follows the keys (W/S or Up/Down move it, A/D or Left/Right select another piston); its `running` attribute becomes `False` when the user presses Escape or closes the window. The module also exports `PistonballLayout` (screen geometry) and `PistonballRenderer` (the pygame renderer).

A.1.6 ConsensusEnv

Module `env_lib.consensus_env`; Chapter 7.

```
ConsensusEnv(*, n_agents=8, task='consensus', topology='ring', dynamics='single',
             dt=0.1, max_steps=200, arena_size=10.0, max_control=1.0,
             control_cost=0.01, noise_std=0.0, tolerance=0.05, success_bonus=10.0,
             formation_shape='circle', formation_radius=3.0, comm_radius=4.0,
             edge_probability=0.3, graph_seed=None, max_neighbors=None,
             damping=0.5, render_mode=None)
```

Member	Description
<code>laplacian_policy(gain=1.0, velocity_gain=None)</code>	Distributed feedback $u = -\text{gain } L(x - d)$ (plus velocity feedback for double integrators), clipped to the action bounds.
<code>positions</code> , <code>velocities</code>	Copies of the current states, (<code>n_agents</code> , 2).
<code>formation_offsets</code>	Offsets d_i , (<code>n_agents</code> , 2) (zeros for consensus).
<code>adjacency</code> , <code>laplacian</code>	Current communication graph and its Laplacian $L = D - A$.
<code>algebraic_connectivity</code>	Fiedler value of the current Laplacian.
<code>task_error</code> , <code>step_count</code>	Current task error and step within the episode.
<code>observation_layout</code>	Column slices of one observation row.

Module-level helpers and constants:

```
make_topology(topology, n_agents, *, edge_probability=0.3, rng=None,
             max_attempts=1000) -> np.ndarray      # adjacency of a static graph
proximity_adjacency(positions, comm_radius) -> np.ndarray  # disk graph
make_formation(shape, n_agents, radius=1.0) -> np.ndarray  # offsets  $d_i$ 
algebraic_connectivity(adjacency) -> float  # second smallest eigenvalue of  $L$ 
is_connected(adjacency) -> bool
```

```

TOPOLOGIES = ('ring', 'line', 'star', 'complete', 'erdos_renyi', 'proximity')
TASKS = ('consensus', 'formation')
DYNAMICS = ('single', 'double')
FORMATION_SHAPES = ('circle', 'line', 'grid', 'wedge')

```

A.1.7 AJLATTEnv and AJLATTConfig

Module `env_lib.ajlatt_env`; Chapter 8.

```

AJLATTEnv(config=None, render_mode=None, **overrides)
AJLATTConfig(map_name='obstacles04', num_robots=4, num_targets=1,
              max_episode_steps=120, sampling_period=0.5, ..., ci_solver='newton',
              target_policy='auto', ..., seed=None) # dataclass, about 50 fields
TeamRewardWrapper(env) # gymnasium.Wrapper
make(figID=0, *, render_mode=None, **kwargs) -> AJLATTEnv # legacy factory
available_maps() -> list[str]
load_grid_map(map_name, margin2wall=0.5) -> GridMap
load_map(map_name) -> tuple[np.ndarray, list[float], list[float]]

```

`AJLATTEnv` is configured by an `AJLATTConfig` and/or keyword overrides of its fields. `TeamRewardWrapper` returns the summed team reward and a scalar `terminated` flag and keeps the per-robot values in `info["agent_rewards"]` and `info["agent_terminated"]`. `make` (also callable as `env_lib.ajlatt_env(...)`) accepts the legacy parameter names with a `DeprecationWarning` and ignores unknown names with a warning.

Member	Description
<i>AJLATTEnv</i>	
<code>single_action_space</code> , <code>single_observation_space</code>	Per-robot spaces; the joint spaces have a leading <code>num_robots</code> axis.
<code>observation_layout()</code>	Dictionary of slices of the per-robot observation vector.
<code>episode_statistics()</code>	Dictionary of per-step traces of the current episode (<code>robot_cov_trace</code> , <code>target_cov_trace</code> , <code>robot_error</code> , <code>target_error</code> , <code>reward</code>), each (<code>steps + 1</code> , <code>num_robots</code>).
<code>config</code> , <code>MAP</code>	The configuration and the loaded <code>GridMap</code> .
<i>AJLATTConfig</i>	
<code>from_kwargs(strict=True, **kwargs)</code>	Build from keyword arguments, translating legacy names.
<code>add_arguments(parser, prefix="")</code>	Register every scalar field as <code>--<prefix><name></code> on an <code>argparse</code> parser.
<code>from_namespace(namespace, prefix="", strict=False)</code>	Build from parsed arguments.
<code>to_dict()</code> , <code>from_dict(data, strict=True)</code>	Plain-dictionary round trip.
<code>replace(**changes)</code>	Validated copy with some fields replaced.
<code>validate()</code> , <code>observation_dim</code> , <code>field_names()</code>	Range checks, per-robot observation size $7 + 4(n_R - 1) + 5$, field list.

A.1.8 AJLATT maps

Module `env_lib.ajlatt_env.maps`; Chapter 8.

```

GridMap(map_path, margin2wall=0.5)
GridMap.from_array(occupancy, *, mapmin=(0.0, 0.0), mapres=0.4, margin2wall=0.5,
                  name='custom') -> GridMap
DynamicMap(map_dir_path, map_name, map_path=None, margin2wall=0.5)
DynamicMap.generate_map(chosen_idx=None, rot_angs=None, rng=None) -> np.ndarray
resolve_map_path(map_name) -> Path
MAP_DIR                                # directory of the bundled maps

```

GridMap method	Description
<code>is_blocked(start_pos, end_pos)</code>	Whether the straight line between two positions crosses an obstacle.
<code>is_blocked_batch(start_pos, end_pos)</code>	Vectorised <code>is_blocked</code> for arrays of shape (n, 2) or (n, 3).
<code>get_closest_obstacle(odom, ang_res=0.05, fov=pi, r_max=3.0, return_pt=False)</code>	Range and bearing of the closest obstacle cell in a field of view.
<code>get_front_obstacle(odom, r_max=3.0)</code>	Range to the first obstacle straight ahead, or None.
<code>is_collision(pos, margin=None)</code>	Out of bounds or within <code>margin</code> of an occupied cell.
<code>in_bound(pos)</code>	At least <code>margin2wall</code> inside the map boundary.
<code>se2_to_cell(pos), cell_to_se2(cell)</code>	Conversions between positions and cell indices.

The submodules `maps.builder` (`build_map`, `rasterize`, `load_spec`, `save_map`) and `maps.plotting` (`plot_map`) implement the console scripts `ajlatt-build-map` and `ajlatt-plot-map`. The geometry helpers `bresenham2D`, `bresenham_batch`, `se2_to_cell`, `cell_to_se2` and `coord_change2g` are exported as well.

A.2 env_lib.utils

```

record_episode(env, policy=None, path=None, *, max_steps=None, seed=None,
              fps=None, render_every=1) -> list[np.ndarray]
save_animation(frames, path, fps=30.0, *, loop=0) -> Path
set_theme(theme)                                # "dark" (default) or "light"
get_theme(theme=None) -> Theme
register_theme(theme)
Theme(name, background, panel, grid, text, muted, accent, positive, negative,
      warning, categorical, sequential_cmap='viridis', diverging_cmap='RdBu_r')
DARK, LIGHT                                     # the built-in themes
FigureWindow(render_mode, *, figsize=(8.0, 6.0), dpi=100, fps=30.0, title=None,
             theme=None)
MatplotlibRenderer(render_mode, *, figsize=(8.0, 6.0), dpi=100, fps=30.0,
                  title=None, theme=None, blit=True)
figure_to_rgb(fig) -> np.ndarray
style_axes(ax, theme, *, title=None, xlabel=None, ylabel=None, grid=True,
          show_spines=False, fontsize=9.0) -> Axes
with_alpha(color, alpha) -> tuple[float, float, float, float]

```

`record_episode` and `save_animation` are described in Section 12.3. `Theme` is a dataclass of colours with `color(index)` for the cycled categorical colours; `set_theme` affects renderers created afterwards. `FigureWindow` owns one matplotlib figure (off-screen Agg canvas in "rgb_array" mode, a window in "human" mode) with `draw()`, `close()` and `is_open`. `MatplotlibRenderer`

is the base class of the environment renderers: subclasses implement `_build(fig)` (create the artists once) and `_update(...)` (update their data), and users call `render(...)`, `reset()` and `close()`. `figure_to_rgb`, `style_axes` and `with_alpha` are helpers for custom renderers.

A.3 toolkit.plotkit

Chapter 10. All plot functions return the `matplotlib.axes.Axes` they drew on.

```
plot_shadow_curve(y, x=None, y_std=None, labels=None, colors=None, alpha=0.2,
                  ax=None, axis=0, figsize=(10, 6), title=None, xlabel=None,
                  ylabel=None, legend=True, grid=True, style='research',
                  legend_labels=None, x_tick_labels=None, y_tick_labels=None,
                  *, band='std', smoothing=None, **kwargs)
plot_learning_curves(runs, x=None, *, band='ci95', smoothing=None, colors=None,
                     alpha=0.2, ax=None, figsize=(10, 6), title=None,
                     xlabel='Step', ylabel='Return', legend=True, grid=True,
                     style='research', **kwargs)
plot_line(x, y, labels=None, colors=None, ax=None, figsize=(10, 6), title=None,
          xlabel=None, ylabel=None, legend=True, grid=True, style='research',
          **kwargs)
plot_scatter(x, y, labels=None, ...)           # same arguments as plot_line
plot_bar(x, height, labels=None, colors=None, ax=None, figsize=(10, 6),
         title=None, xlabel=None, ylabel=None, legend=True, grid=True,
         style='research', *, width=0.8, yerr=None, value_labels=False,
         horizontal=False, capsize=3.0, **kwargs)
plot_histogram(data, bins='auto', labels=None, colors=None, ax=None,
               figsize=(10, 6), title=None, xlabel=None, ylabel=None,
               legend=True, grid=True, style='research', *, density=False,
               alpha=0.45, histtype='stepfilled', **kwargs)
plot_heatmap(data, xlabels=None, ylabels=None, cmap='viridis', annot=False,
             ax=None, figsize=(8, 6), title=None, xlabel=None, ylabel=None,
             cbar=True, cbar_label=None, style='research', *, fmt=None,
             vmin=None, vmax=None, center=None, robust=False, mask=None,
             nan_color=None, aspect='auto', square=False, linewidths=None,
             linecolor='white', annot_kws=None, cbar_kws=None, **kwargs)
plot_gray_scale(data, xlabels=None, ylabels=None, ax=None, figsize=(8, 6),
                title=None, xlabel=None, ylabel=None, style='research',
                **kwargs)

save_figure(fig_or_ax, path, formats=None, dpi=300, transparent=False,
            **savefig_kwargs) -> list[Path]
style_context(style='research')           # context manager
set_research_style()
reset_style()
get_palette(name='research', n=None) -> list[str]

STYLE_PRESETS           # read-only; 'research', 'presentation', 'minimal', 'default'
RESEARCH_COLORS, RESEARCH_COLOR_LIST      # tab10, dict and list
OKABE_ITO_COLORS, OKABE_ITO_COLOR_LIST    # colour-blind-safe, dict and list
```

`python -m toolkit.plotkit` and the console script `plotkit-gallery` run the gallery (Section 10.11). The module `toolkit.plotkit.core` re-exports the same names for code written against earlier versions.

A.4 toolkit.neural_toolkit

Chapter 11. Requires PyTorch. All networks are `torch.nn.Module` subclasses; the shapes are given in Table 11.1.

A.4.1 Layers

```
get_activation(name) -> nn.Module
build_mlp(input_dim, hidden_dims, output_dim=None, activation='relu',
          dropout=0.0, layer_norm=False, output_activation=None,
          bias=True) -> nn.Sequential
PositionalEncoding(d_model, dropout=0.1, max_len=5000)
ACTIVATION_NAMES    # ('elu', 'gelu', 'identity', 'leaky_relu', 'mish', 'relu',
                    # 'sigmoid', 'silu', 'softplus', 'swish', 'tanh')
```

A.4.2 Policy and value networks

```
MLPPolicyNetwork(input_dim, output_dim, hidden_dims=(256, 256),
                  activation='relu', dropout=0.0, layer_norm=False, device='cpu')
CNPolicyNetwork(input_channels, output_dim, conv_dims=(32, 64, 128),
                 fc_dims=(256, 256), kernel_sizes=None, strides=None,
                 activation='relu', dropout=0.0, layer_norm=False, device='cpu')
RNPolicyNetwork(input_dim, output_dim, hidden_dim=256, num_layers=2,
                 rnn_type='lstm', bidirectional=False, fc_dims=(256,),
                 activation='relu', dropout=0.0, layer_norm=False, device='cpu')
TransformerPolicyNetwork(input_dim, output_dim, d_model=256, nhead=8,
                          num_layers=6, dim_feedforward=1024, dropout=0.1,
                          fc_dims=(256,), activation='relu', layer_norm=True,
                          device='cpu', max_len=5000)
CNNPolicyNetwork = CNPolicyNetwork; RNPPolicyNetwork = RNPolicyNetwork

MLPValueNetwork, CNNValueNetwork, RNNValueNetwork, TransformerValueNetwork
# same arguments as the policy networks, with output_dim=1 by default
```

A.4.3 Q-networks

```
MLPQNetwork(state_dim, action_dim, hidden_dims=(256, 256), activation='relu',
             dropout=0.0, layer_norm=False, device='cpu', continuous_action=False)
DuelingQNetwork(state_dim, action_dim, hidden_dims=(256, 256),
                 activation='relu', dropout=0.0, layer_norm=False, device='cpu')
CNQNetwork(input_channels, action_dim, conv_dims=(32, 64, 128),
            fc_dims=(256, 256), kernel_sizes=None, strides=None,
            activation='relu', dropout=0.0, layer_norm=False, device='cpu')
RNQNetwork(state_dim, action_dim, hidden_dim=256, num_layers=2, rnn_type='lstm',
            bidirectional=False, fc_dims=(256,), activation='relu', dropout=0.0,
            layer_norm=False, device='cpu')
TransformerQNetwork(state_dim, action_dim, d_model=256, nhead=8, num_layers=6,
                     dim_feedforward=1024, dropout=0.1, fc_dims=(256,),
                     activation='relu', layer_norm=True, device='cpu',
                     max_len=5000)
CNNQNetwork = CNQNetwork; RNNQNetwork = RNQNetwork
# forward(state, action=None[, hidden or mask]): (B, A), or Q(s, a) of shape (B,)
```

A.4.4 Encoders and decoders

```
MLPEncoder(input_dim, latent_dim, hidden_dims=(256, 256), activation='relu',
            dropout=0.0, layer_norm=False, device='cpu')
CNNEncoder(input_channels, latent_dim, conv_dims=(32, 64, 128, 256),
            fc_dims=(512, 256), kernel_sizes=None, strides=None,
            activation='relu', dropout=0.0, layer_norm=False, device='cpu',
            pooled_size=4)
RNNEncoder(input_dim, latent_dim, hidden_dim=256, num_layers=2, rnn_type='lstm',
            bidirectional=True, fc_dims=(256,)), activation='relu', dropout=0.0,
            layer_norm=False, device='cpu')
TransformerEncoder(input_dim, latent_dim, d_model=256, nhead=8, num_layers=6,
                   dim_feedforward=1024, dropout=0.1, fc_dims=(256,)),
                   activation='relu', layer_norm=True, device='cpu',
                   max_len=5000)
VariationalEncoder(input_dim, latent_dim, hidden_dims=(256, 256),
                   activation='relu', dropout=0.0, layer_norm=False,
                   device='cpu')           # forward -> (z, mu, logvar)
VariationalEncoder.kl_divergence(mu, logvar) -> Tensor    # static, shape (B,)

MLPDecoder(latent_dim, output_dim, hidden_dims=(256, 256), activation='relu',
            dropout=0.0, layer_norm=False, device='cpu')
CNNDecoder(latent_dim, output_channels, fc_dims=(512, 256),
            conv_dims=(256, 128, 64, 32), kernel_sizes=None, strides=None,
            initial_size=4, activation='relu', dropout=0.0, layer_norm=False,
            device='cpu', output_activation='sigmoid')    # property output_size
RNND decoder(latent_dim, output_dim, hidden_dim=256, num_layers=2,
             rnn_type='lstm', max_seq_len=100, fc_dims=(256,)), activation='relu',
             dropout=0.0, layer_norm=False, device='cpu')
TransformerDecoder(latent_dim, output_dim, d_model=256, nhead=8, num_layers=6,
                   dim_feedforward=1024, max_seq_len=100, dropout=0.1,
                   fc_dims=(256,)), activation='relu', layer_norm=True,
                   device='cpu', max_len=None)
VariationalDecoder(latent_dim, output_dim, hidden_dims=(256, 256),
                   activation='relu', dropout=0.0, layer_norm=False,
                   device='cpu')
# RNND decoder and TransformerDecoder: forward(z, seq_len=None)
```

A.4.5 Factories

```
PolicyFactory.create_policy(policy_type, **kwargs)           # mlp cnn rnn transformer
ValueFactory.create_value_network(network_type, **kwargs)   # mlp cnn rnn transformer
QNetworkFactory.create_q_network(network_type, **kwargs)    # + dueling
EncoderFactory.create_encoder(encoder_type, **kwargs)        # + vae
DecoderFactory.create_decoder(decoder_type, **kwargs)        # + vae
```

The abstract base classes `BasePolicyNetwork`, `BaseValueNetwork`, `BaseQNetwork`, `BaseEncoder` and `BaseDecoder` are exported for subclassing and type checks.

A.4.6 NetworkUtils

All methods are static; Table 11.3 describes them.

```
initialize_weights(module, method='xavier_uniform', gain=1.0)
```

```

count_parameters(model) -> int
count_parameters_by_layer(model) -> dict[str, int]
freeze_layers(model, layer_names)
unfreeze_layers(model, layer_names)
get_grad_norm(model, norm_type=2.0) -> float
clip_grad_norm(model, max_norm, norm_type=2.0) -> float
create_mlp_layers(input_dim, hidden_dims, output_dim, activation='relu',
                  dropout=0.0, layer_norm=False, bias=True) -> nn.Sequential
create_conv_layers(input_channels, conv_dims, kernel_sizes=None, strides=None,
                  padding=None, activation='relu', dropout=0.0,
                  layer_norm=False) -> nn.Sequential
create_transposed_conv_layers(input_channels, conv_dims, kernel_sizes=None,
                              strides=None, padding=None, output_padding=None,
                              activation='relu', dropout=0.0,
                              layer_norm=False) -> nn.Sequential
create_attention_layer(input_dim, num_heads=8, dropout=0.1) -> nn.Module
create_positional_encoding(d_model, max_len=5000) -> nn.Parameter
get_activation(activation) -> nn.Module
apply_weight_decay(model, weight_decay, skip_list=None) -> list[dict]
create_optimizer(model, optimizer_type='adam', lr=0.001, weight_decay=0.0,
                 skip_list=None, **kwargs) -> torch.optim.Optimizer
create_scheduler(optimizer, scheduler_type='step', **kwargs)
save_checkpoint(model, optimizer, epoch, loss, filepath, **extra) -> Path
load_checkpoint(model, optimizer, filepath, map_location=None,
               weights_only=True, strict=True) -> tuple[int, float]
compute_output_size(input_size, kernel_size, stride=1, padding=0,
                   dilation=1) -> int
compute_transposed_output_size(input_size, kernel_size, stride=1, padding=0,
                              output_padding=0, dilation=1) -> int
compute_conv_output_size(input_size, conv_layers) -> tuple[int, int]

```

A.4.7 Tabular tools

```

QTable(state_space_size, action_space_size=None, initial_value=0.0,
       dtype=np.float32, rng=None)
ValueTable(...same arguments...)
PolicyTable(...same arguments...)           # rows start uniform
BaseDiscreteTable                          # abstract base of the three tables
DiscreteEnvironment(state_space_size, action_space_size, rng=None) # abstract

DiscreteTools.create_q_table(state_space_size, action_space_size,
                             initial_value=0.0, rng=None) -> QTable
DiscreteTools.create_value_table(state_space_size, initial_value=0.0,
                                 rng=None) -> ValueTable
DiscreteTools.create_policy_table(state_space_size, action_space_size,
                                  rng=None) -> PolicyTable
DiscreteTools.q_learning_update(q_table, state, action, reward, next_state,
                                gamma=0.99, alpha=0.1, done=False)
DiscreteTools.sarsa_update(q_table, state, action, reward, next_state,
                           next_action, gamma=0.99, alpha=0.1, done=False)
DiscreteTools.expected_sarsa_update(q_table, state, action, reward, next_state,
                                    policy_table=None, gamma=0.99, alpha=0.1,
                                    epsilon=0.0, done=False)
DiscreteTools.value_iteration_update(value_table, state, q_table, gamma=0.99)
DiscreteTools.policy_iteration_update(policy_table, state, q_table)
DiscreteTools.epsilon_greedy_policy(q_table, state, epsilon) -> int

```

```

DiscreteTools.softmax_policy(q_table, state, temperature) -> int
DiscreteTools.boltzmann_policy(q_table, state, temperature) -> int
DiscreteTools.ucb_policy(q_table, state, visit_counts,
                        exploration_constant=1.0) -> int
DiscreteTools.thompson_sampling_policy(q_table, state, visit_counts,
                                       prior_alpha=1.0, prior_beta=1.0) -> int

```

A.5 toolkit.parakit

Chapter 9, Section 9.2.

```

get_parameters(parser) -> dict[str, Any]
save_parameters(values, path_or_dir) -> Path
load_parameters(path) -> dict[str, Any]
apply_parameters(parser, values, strict=True,
                validation_callbacks=None) -> argparse.ArgumentParser
parse_value(action, text, validator=None) -> Any
parse_bool(value) -> bool
format_value(action, value) -> str
describe_type(action) -> str
tunable_actions(parser) -> dict[str, argparse.Action]

ParameterTuner(parser, save_delay=30000, save_path=None, auto_save_enabled=True,
               validation_callbacks=None, inactivity_timeout=10)
ParameterTuner.tune() -> argparse.ArgumentParser
ParameterTuner.tune_parameters(parser, ...same arguments...)    # class method
ParameterAdjuster = ParameterTuner
TK_MISSING_MESSAGE                                             # str: how to install Tk

```

Appendix B

Migration from pre-1.0 versions

Before version 1.0 the code lived in two separate projects, `envlib_project/env_lib` and `toolkit_project/toolkit`, each with its own `setup.py`, and several environments had their own installation files. Version 1.0 turns them into one distribution with a stable API, fixes a number of bugs and changes some behaviour. This appendix explains, module by module, what code written for the earlier versions has to change. The complete list of changes is in `CHANGELOG.md`.

Most user code keeps working: class names, module paths, constructor arguments and their defaults are unchanged, and deprecated arguments are translated with a `DeprecationWarning`. Python shows these warnings only for code in `__main__` by default; run your scripts once with `python -W default::DeprecationWarning` to list everything that needs attention.

B.1 Checklist

1. Uninstall the old distributions and install `my-tool-box` (Section [B.2](#)).
2. Unpack (`observation`, `info`) from every `reset()` call and use the five-tuple of `step()`, including for `AJLATT`.
3. Create `AJLATT` through `env_lib.AJLATTEnv` with the new parameter names and adapt to the joint action and observation spaces (Section [B.7](#)).
4. For `Pistonball` in discrete mode, pass one value in $\{0, 1, 2\}$ per piston (Section [B.6](#)).
5. Treat `terminated` and `truncated` separately, and set episode limits through the constructor arguments (Section [B.3](#)).
6. Retrain `neural_toolkit` models; old checkpoints cannot be loaded (Section [B.9](#)).
7. If your figures relied on side effects of `plotkit` on the global `matplotlib` configuration, call `set_research_style()` explicitly (Section [B.8](#)).

B.2 Installation

Before, each project (and optionally each environment) was installed from its own folder:

```
cd envlib_project && pip install -e .
cd ../toolkit_project && pip install -e .
cd ../envlib_project/env_lib/pistonball_env && pip install -e .    # and similar
```

After, one installation from the repository root provides both import packages; optional features are selected with extras (Chapter 2):

```
pip uninstall -y env_lib toolkit
pip uninstall -y ttenv kos_env linemsg_env pistonball_env wireless_comm_env
pip install -e ".[all]"          # or ".[all,dev]" for development
```

The import names `env_lib` and `toolkit` are unchanged. TensorFlow, seaborn, pandas and scikit-learn are no longer required. Importing `env_lib` or `toolkit` no longer imports PyTorch, pygame, pymunk or Tk; the subpackages and environment classes are loaded on first use, so `import toolkit; toolkit.plotkit.plot_line(...)` and `from env_lib import PistonballEnv` continue to work.

B.3 Changes common to all environments

Reset and step. All environments follow the Gymnasium API. `reset(*, seed=None, options=None)` returns `(observation, info)`, and `step(action)` returns `(observation, reward, terminated, truncated, info)`. The `seed` argument must be passed by keyword.

```
# Before (AJLATT and old-style loops)
# obs = env.reset()
# obs, reward, done, info = env.step(action)

# After
obs, info = env.reset(seed=0)
obs, reward, terminated, truncated, info = env.step(env.action_space.sample())
done = bool(np.any(terminated)) or truncated
```

Seeding. `env.seed(s)` is deprecated where it existed (LineMsg, WirelessComm, Pistonball) and emits a `DeprecationWarning`; use `env.reset(seed=s)`. No environment seeds the global NumPy or PyTorch generators any more (the Kuramoto environments used to call `np.random.seed(42)` and `torch.manual_seed(42)`); code that relied on these side effects for its own random numbers must seed its own generator.

Episode limits and termination. The registered ids no longer add a `TimeLimit` wrapper. Previously, for example, `KuramotoOscillator-v0` was truncated after 1000 steps by the wrapper even if a larger `max_steps` was passed to `gym.make`; now the constructor argument (`max_steps`, `max_iter`, `max_cycles` or `max_episode_steps`) is the only limit, and reaching it sets `truncated=True`. Task success (for example synchronisation of the oscillators) sets `terminated=True`; previously some environments reported both cases as `terminated`.

Registration. Environments are registered when `env_lib` is imported; the `register()` functions of `env_lib.kos_env` and `env_lib.wireless_comm_env` are deprecated no-ops that call the central registration. `env_lib.make(id)` and `env_lib.list_envs()` are new. The id `AJLATT-v1`, an unusable alias of `AJLATT-v0`, was removed.

Observations, rewards and rendering. Observations are `float32` and lie in `observation_space`; per-agent rewards are available in `info["agent_rewards"]`. Rendering happens only on request: in "human" mode the environment renders itself in `reset()` and

`step()`, so explicit `env.render()` calls in "human" loops are no longer needed, and "rgb_array" works without a display. `step()` before the first `reset()` now raises an error instead of resetting implicitly.

B.4 Kuramoto oscillators

- *PyTorch backend.* A sign error made the coupling repulsive ($\sin(\theta_i - \theta_j)$ instead of $\sin(\theta_j - \theta_i)$). Results obtained with `KuramotoOscillatorEnvTorch` before 1.0 are therefore not comparable with new ones. Both backends now produce the same trajectories.
- *Normalisation.* Division of the coupling sum by N is opt-in for both backends (`normalize_coupling=True`). Code that relied on the normalisation of the old PyTorch backend must set this flag.
- *Coupling matrix.* The NumPy backend used to skip entries $K_{ij} \leq 0$; the full matrix is used now. Default settings are not affected.
- *Constant coupling.* `coupling_mode="constant"` without a matrix now uses `coupling_strength * adjacency`, which makes the registered `*-Constant-v0` ids usable (they raised before).
- *Episode end.* Synchronisation ($r > \text{sync_threshold}$) sets `terminated`; `sync_threshold` and `sync_bonus` are constructor arguments. The PyTorch backend accepts `device="auto"`, and its phase coherence and bonus rules now match the NumPy backend.

B.5 LineMsg and WirelessComm

Seeded trajectories are bit-identical to the previous implementations, so existing results remain valid. The text output of the old renderers is available as `render_mode="ansi"`; "human" and "rgb_array" show the new dashboards. `LineMsg` accepts actions either as an integer in `Discrete(2**N)` or as an array of N bits, and `action_space_type="multibinary"` supports more than 62 agents. `WirelessComm` validates its arguments and actions and reports per-agent outcomes (idle, success, collision, lost) in `info["outcomes"]`.

B.6 Pistonball

Discrete actions. The discrete action space is `MultiDiscrete([3] * n_pistons)`: one value per piston, 0 moves the piston down, 1 keeps it still and 2 moves it up. Earlier versions declared the same array format as `Box(0, 2, (n_pistons,), int32)`, and some earlier documentation used a single base-3 integer in `Discrete(3**n_pistons)`. Code that sends arrays keeps working; code that encodes a single integer must decode it:

```
import numpy as np
import env_lib

env = env_lib.PistonballEnv(n_pistons=5, continuous=False)
obs, info = env.reset(seed=0)

# Before: one integer for all pistons (base-3 encoding)
code = 13
# After: one value per piston; decode an old integer action like this
action = np.array([(code // 3**i) % 3 for i in range(env.n_pistons)]) # [1 1 1 0 0]
```

```
obs, reward, terminated, truncated, info = env.step(action)
```

Other changes. Actions are validated (shape, range, NaN). The screen is only drawn when rendering is requested; the methods `draw()`, `draw_background()`, `draw_pistons()` and `enable_render()` are deprecated no-ops, and the PNG sprites were replaced by a vector renderer. The environment works with pymunk 6 and 7.

B.7 AJLATT

AJLATT was rewritten as a Gymnasium environment configured by a dataclass. The old factory called `argparse.parse_args()` internally, which failed in any script with its own command-line arguments.

Creating the environment. The legacy factory still works; the class and the configuration dataclass are the preferred interface:

```
import env_lib
from env_lib import AJLATTConfig, AJLATTEnv

# Before (still works, with DeprecationWarnings for the legacy names)
env = env_lib.ajlatt_env(map_name="obstacles04", num_Robot=4, T_steps=100)

# After
env = AJLATTEnv(map_name="obstacles04", num_robots=4, max_episode_steps=100)

# Or with an explicit configuration object
config = AJLATTConfig(map_name="obstacles05", num_robots=3)
env = AJLATTEnv(config, render_mode="rgb_array")
```

Table 13.2 lists the renamed parameters. Rendering is off by default; the old `render=True` flag becomes `render_mode="human"`, and `render_mode="rgb_array"` records frames without a display.

Spaces, rewards and termination. The spaces are *joint*: `action_space` has shape `(num_robots, 2)` (linear and angular velocity of each robot) and `observation_space` shape `(num_robots, obs_dim)`, both float32. The per-robot spaces used by the old version are available as `single_action_space` and `single_observation_space`. The reward and `terminated` are per-robot arrays; wrap the environment in `TeamRewardWrapper` for a scalar team reward and termination flag.

```
import numpy as np
from env_lib.ajlatt_env import TeamRewardWrapper

env = AJLATTEnv(num_robots=4)
print(env.single_action_space.shape, env.action_space.shape)    # (2,) (4, 2)

obs, info = env.reset(seed=0)                                     # before: obs = env.reset()
actions = np.stack([env.single_action_space.sample() for _ in range(4)])
obs, reward, terminated, truncated, info = env.step(actions)
print(reward.shape, terminated.shape)                             # (4,) (4,)
```

```
team_env = TeamRewardWrapper(AJLATTEnv(num_robots=4))
obs, info = team_env.reset(seed=0)
obs, team_reward, done, truncated, info = team_env.step(actions)  # float, bool
```

Behaviour changes. The following changes affect results:

- The target-velocity entries of the observation contain the commanded target velocity instead of the configured constant.
- Covariance intersection uses a Newton solver by default; its objective is never worse than that of the previous SLSQP solver. Pass `ci_solver="slsqp"` to reproduce results of earlier versions (to about 10^{-6}).
- Target policies are per environment instance and selected with `target_policy`; they used to be shared class attributes that were not reset between episodes. Unknown map names raise an error instead of prompting with `input()`.
- Covariance traces were appended to lists that were never cleared (a memory leak). Per-episode traces are returned by `env.episode_statistics()`.
- Dynamic maps are regenerated at every `reset()` and no longer need scikit-image.

Training scripts. The PPO, LLM and debugging options that used to live in the AJLATT parameter module were removed from the environment package. To expose the environment options on a script's own parser, use `AJLATTConfig.add_arguments`:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--lr", type=float, default=3e-4)  # your own options
AJLATTConfig.add_arguments(parser, prefix="env.")      # --env.num_robots, ...
args = parser.parse_args(["--env.num_robots", "3"])
config = AJLATTConfig.from_namespace(args, prefix="env.")
env = AJLATTEnv(config)
```

Maps. The map tools moved to `env_lib.ajlatt_env.maps` and are available as the console scripts `ajlatt-build-map` and `ajlatt-plot-map` instead of the scripts `build_map.py` and `plot_map.py`. The old builder wrote transposed grids; rebuild custom maps from their specification with the new builder. Maps whose grid is one row shorter than the header declares are padded with a wall row and a warning (three bundled maps had this problem and were fixed).

B.8 plotkit

The function names and positional arguments are unchanged; new options are keyword-only arguments appended to the signatures.

- *No global side effects.* The plot functions used to call `plt.style.use("default")` and change `rcParams`, which also affected later, unrelated figures. They now apply their style locally. Code that relied on the side effect should call `set_research_style()` once (and `reset_style()` to undo it). `set_research_style` no longer sets `figure.dpi=300`, which made on-screen figures very large; it still sets `savefig.dpi=300`.

- *Fixed input handling.* `plot_line(x, [y1, y2])` plots both series (it plotted only the first), and a flat list of numbers is one series (it was treated as several). Several bar series are grouped side by side instead of overlapping.
- *New features.* `band` and `smoothing` in `plot_shadow_curve`, NaN-robust statistics, `plot_learning_curves`, `plot_histogram`, `save_figure`, style presets, the Okabe-Ito palette and the gallery command. The heatmap functions no longer need seaborn.

```
from toolkit.plotkit import plot_line, reset_style, set_research_style

# Before: calling any plotkit function changed the global style as a side effect.
# After: opt in explicitly when plain matplotlib code should use the research style.
set_research_style()
ax = plot_line([0, 1, 2], [[0, 1, 4], [0, 2, 3]], labels=["a", "b"]) # two series
reset_style()
```

B.9 neural_toolkit

Retrain models. Earlier MLP stacks applied no activation between hidden layers, and several other defects changed the computed functions (Section 11.8). The layer structure of every network changed, so checkpoints written before 1.0 cannot be loaded; retrain and save new checkpoints.

API differences. Besides the fixes, the following details changed:

- Unknown activation names raise `ValueError` instead of silently falling back to a default, and the `device` argument is honoured (modules are moved to it at construction).
- Q-networks honour the `action` argument: `q(s, a)` returns $Q(s, a)$ with shape `(B,)` for integer actions.
- `NetworkUtils.load_checkpoint` loads with `weights_only=True` and maps tensors to the model's device by default. Checkpoints that contain arbitrary Python objects in the extra entries need `weights_only=False` (only for trusted files). `save_checkpoint` accepts `optimizer=None` and extra keyword entries, and returns the path.
- The method `create_positional_encoding` of `NetworkUtils` returns the frozen table as an `nn.Parameter` of shape `(1, max_len, d_model)`; use `PositionalEncoding` for a module that adds it to its input.
- The tabular tools draw random numbers from a per-table generator (`rng` argument) instead of the global NumPy state.

Names from earlier documentation. The documentation of earlier versions described some names that never existed in the code. Table B.1 lists the actual equivalents.

B.10 parakit

`ParameterTuner` (and its alias `ParameterAdjuster`) keeps its constructor and `tune_parameters` class method, but its behaviour was corrected:

- `tune()` installs the accepted values as the parser's defaults, as it was documented to do; previously the returned parser was unchanged.

Table B.1: Names used in pre-1.0 documentation and their actual equivalents.

Documented before 1.0	Use
<i>Static methods of NetworkUtils</i>	
<code>count_trainable_parameters</code>	<code>count_parameters</code> (counts trainable parameters)
<code>init_weights</code>	<code>initialize_weights</code>
<code>clip_gradients</code>	<code>clip_grad_norm</code>
<code>get_device, to_device</code>	plain PyTorch: <code>torch.device(...)</code> , <code>module.to(device)</code>
<i>Encoder arguments</i>	
<code>output_dim</code> of <code>CNNEncoder</code> and <code>RNNEncoder</code>	<code>latent_dim</code>

- Boolean strings (the text "False" used to be stored as a string, which is true in a boolean context), lists (`nargs`), `None` values and `store_true` options are parsed correctly.
- Validation callbacks receive the *converted* value. Callbacks written for the old convention, which received text and returned (`is_valid`, `converted_value`, `message`), keep working as long as they also accept an already converted value (for example `float(value)` of a float); their converted value is ignored.
- `import toolkit.parakit` works without Tk; the window imports Tk only when it opens. The library no longer calls `logging.basicConfig`, so configure logging in your script if you want to see its log messages.
- New headless functions (`get_parameters`, `save_parameters`, `load_parameters`, `apply_parameters`, `parse_value`) make the GUI optional (Section 9.2).

```
import argparse
from toolkit.parakit import apply_parameters, load_parameters, save_parameters

parser = argparse.ArgumentParser()
parser.add_argument("--learning_rate", type=float, default=1e-3)

# Before: validators received text and returned a 3-tuple
def validate_lr_old(value):
    lr = float(value)
    return (0 < lr <= 1), lr, "learning rate must be in (0, 1]"

# After: validators receive the converted value; a bool or (ok, message) suffices
def validate_lr(value):
    return 0 < value <= 1, "learning rate must be in (0, 1]"

path = save_parameters({"learning_rate": 0.01}, "runs/params")
apply_parameters(parser, load_parameters(path),
                 validation_callbacks={"learning_rate": validate_lr_old})
apply_parameters(parser, {"learning_rate": "0.02"},
                 validation_callbacks={"learning_rate": validate_lr})
```